

Using ML techniques to predict RT side effect as patients who affected by cancer and took RT after removing cancer part could be affected by some RT side effect which can influence their quality of life such as arm_lymphoedema side effect. The data that had been collected contain clinical and genetic data

```
##### 'breast_Arm_Lymphoedema24m' #####
```

1) Data collection

The data is stores as csv file and previously had been taken from the hospitals of 9 regions, and all of these pateints had breast cancer before and acure with RT afterward the operation.

importing some libraries in order to upload data

```
# importing Pandas and NumPy library
import pandas as pd
import numpy as np
from numpy import sqrt
import statsmodels.api as sm
import statsmodels.formula.api as smf

# for visualising and metrics
from sklearn.decomposition import PCA
from sklearn import metrics
# for color heatmap
from matplotlib import cm

# for error
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import cross_val_score
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.preprocessing import MinMaxScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.feature_selection import RFE
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import MinMaxScaler
from sklearn.pipeline import Pipeline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, confusion_matrix,
import matplotlib.pyplot as plt
from sklearn.metrics import roc_auc_score
from numpy import sqrt, argmax

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

# Step 1: Dynamically compute class weights
from collections import Counter
import numpy as np

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_curve, auc, precision_recall_
average_precision_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier, ExtraTre
GradientBoostingClassifier, AdaBoostClassifier
from xgboost import XGBClassifier
from sklearn.discriminant_analysis import QuadraticDiscrimina
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neural_network import MLPClassifier
import matplotlib.cm as cm

## This with pipeline of standarise scaling
from tqdm import tqdm
```

```
# for ML models
# For statistic Method
import scipy.stats as stats
import statsmodels.api as sm
from statsmodels.formula.api import ols

from scipy.stats import chi2_contingency

# for calculate geometric mean
from numpy import argmax
from numpy import interp
```

```
from tabulate import tabulate

#from sklearn import feature_selection
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import chi2

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import KFold

from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_curve, auc, roc_auc_score
from sklearn.metrics import RocCurveDisplay
from sklearn.metrics import precision_recall_curve
from sklearn.metrics import average_precision_score
from sklearn.metrics import accuracy_score, precision_score,
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import classification_report

# Importing ML models
from sklearn import linear_model
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier as KNN
from xgboost import XGBClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.discriminant_analysis import QuadraticDiscrimina
from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import label_binarize
from sklearn.multiclass import OneVsRestClassifier
from sklearn.ensemble import VotingClassifier
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.optimizers import Adam
```

```
# Import Keras for implementing autoencoders
import keras
from imblearn.over_sampling import SMOTE

from sklearn.ensemble import ExtraTreesClassifier, RandomForest
from sklearn.model_selection import RepeatedStratifiedKFold #
from sklearn.ensemble import HistGradientBoostingClassifier

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import MinMaxScaler
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, Stratifi

import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.preprocessing import MinMaxScaler
from collections import Counter
import matplotlib.pyplot as plt
```

✓ uploading the data

```
# Read the data from original file (use relative paths)
data1 = pd.read_csv('/content/FinalRTmerged_data28.12.24.csv')
#data1 = pd.read_csv('/content/FinalRTmerged_data8.10.25.csv')
data2= pd.read_csv('/content/ArmlymphedemaSNPs.csv')
```

```
list(data1.columns)
```

```

breast_pain_medication_name_84m',
'breast_pain_medication_dose_and_freq_84m',
'breast_swollen_arm_84m',
'breast_swollen_arm_interferes_with_normal_activity_84m',
'breast_left_arm_circumference_mm_84m',
'breast_right_arm_circumference_mm_84m',
'breast_arm_measuring_date_84m',
'Site_84m',
'Visit_96m',
'Sequence_Num_96m',
'Date_96m',
'CRFversion_96m',
'CRFstatus_96m',
'breast_atrophy_96m',
'breast_nipple_retraction_96m',
'breast_nipple_retraction MV Indicator_96m',
'breast_oedema_96m',
'breast_skin_ulceration_96m',
'breast_telangiectasia_tumour_bed_96m',
'breast_telangiectasia_outside_tumour_bed_96m',
'breast_skin_induration_tumour_bed_96m',
'breast_skin_induration_outside_tumour_bed_96m',
'breast_erythema_96m',
'breast_arm_lymphodema_96m',
'breast_skin_hyperpigmentation_96m',
'breast_pneumontis_96m',
'breast_pain_96m',
'breast_pain_severity_96m',
'breast_pain_medication_96m',
'breast_pain_medication_name_96m',
'breast_pain_medication_dose_and_freq_96m',
'breast_swollen_arm_96m',
'breast_swollen_arm_interferes_with_normal_activity_96m',
'breast_left_arm_circumference_mm_96m',
'breast_right_arm_circumference_mm_96m',
'breast_arm_measuring_date_96m',
'Site_96m'

```

```

# Select the dependent variable here
dependent1 = 'breast_arm_lymphodema_24m'

```

✦ implementing functions

```

# to show all rows and columns in dataframe
def max_val(df):
    pd.set_option('display.max_columns', None)
    pd.set_option('display.max_rows', None)

```

```

from IPython.display import display, HTML

def bold(text):
    display(HTML(f"<p style='font-size:16pt;'>{text}</p>"))

```

```

# To check the percentage of null value in each feature
def nullP(df):
    count_null= ((pd.isnull(df).sum()/len(df))*100).round()
    print(count_null.sort_values(ascending=False))
    count_null.to_csv('count_null_all.csv')

```

```

# calculate uniq data in X coloumn
def Cuniq(df, x):
    print(df[x].sort_values().value_counts())

```

```
# This function is used to print the uniques values in each f
def uniq(data):
    for col in list(data):
        print(col, "Unique Values are:\n ", data[col].unique())
        print("=====
```

```
# This function is used to Remove all features that contain X

def X_per_removed(data1, dep, perc):

    min_count = int(((100 - perc) / 100) * data1.shape[0] + 1

    cols_to_keep = [
        col for col in data1.columns
        if col == dep or data1[col].count() >= min_count
    ]

    feat_data = data1[cols_to_keep]

    print(feat_data.shape)
    print(list(feat_data.columns))

    return feat_data
```

```
def remove_duplicate(df):

    duplicate_cols = []
    cols = df.columns
    for i in range(len(cols)):
        for j in range(i + 1, len(cols)):

            col1 = df[cols[i]]
            col2 = df[cols[j]]

            # Compare ignoring NaNs
            same = ((col1 == col2) | col1.isna() | col2.isna())

            if same:

                # Keep column with fewer NaNs
                if col1.isna().sum() <= col2.isna().sum():
                    duplicate_cols.append(cols[j])
                else:
                    duplicate_cols.append(cols[i])
```

```
print("Columns to drop:")
print(set(duplicate_cols))

# Drop duplicates
n_df = df.drop(columns=set(duplicate_cols))
print(n_df.shape)
```

```
import numpy as np
import pandas as pd

def pairwise_corr(df, col=None, threshold=0.70):

    # Save ID/target column if provided
    keep_col = None
    if col is not None and col in df.columns:
        keep_col = df[[col]]
        X = df.drop(columns=[col])
    else:
        X = df.copy()

    # Numeric columns only for correlation
    numeric_features = X.select_dtypes(include=np.number).columns
    X_numeric = X[numeric_features]

    corr_matrix = X_numeric.corr().abs()

    mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
    tri_df = corr_matrix.mask(mask)

    to_drop = []
    correlated_pairs = []

    for c in tri_df.columns:
        for index, value in tri_df[c].items():

            if pd.notna(value) and value > threshold:

                correlated_pairs.append((c, index, value))

            # Drop lower-variance feature
            if X_numeric[c].var() > X_numeric[index].var():
                to_drop.append(index)
            else:
                to_drop.append(c)

    to_drop = list(set(to_drop))
```

```

reduced_X = X.drop(columns=to_drop, errors='ignore')

# Add col back to final dataset
if keep_col is not None:
    reduced_X = pd.concat([keep_col, reduced_X], axis=1)

print(f"Highly correlated feature pairs > {threshold}:")
for c1, c2, corr in correlated_pairs:
    print(f"{c1} and {c2}: {corr:.2f}")

print("\nRemoved features:")
print(to_drop)

print("\nFinal shape:", reduced_X.shape)

return reduced_X

```

Start coding or [generate](#) with AI.

```

# VIF measures how much a feature (column) is correlated with
#A high VIF (e.g., > 5 or 10) means the feature is highly cor
#This can make models like regression unstable or hard to int
import pandas as pd
from statsmodels.stats.outliers_influence import variance_inf

def calculate_vif(df, threshold):
    # Store Subject_Id separately
    subject_ids = df['Subject Id']

    # Drop Subject_Id and non-numeric, drop rows with missing
    df_snp = df.drop(columns=['Subject Id'])
    df_snp = df_snp.select_dtypes(include=[float, int])
    df_snp = df_snp.dropna()

    # Keep matching Subject_Id values (after dropna)
    subject_ids = subject_ids.loc[df_snp.index]

    dropped = True
    while dropped:
        vif_data = pd.DataFrame()
        vif_data["SNP"] = df_snp.columns
        vif_data["VIF"] = [variance_inflation_factor(df_snp.v
                                                    for i in range(df_snp.shape[1]))]

        max_vif = vif_data["VIF"].max()
        if max_vif > threshold:
            drop_snp = vif_data.sort_values("VIF", ascending=

```



```

        df_snp = df_snp.drop(columns=[drop_snp])
        print(f"Dropped {drop_snp} with VIF {max_vif:.2f}")
    else:
        dropped = False

# Add Subject_Id back safely
df_snp['Subject Id'] = subject_ids.values

return df_snp

```

Using Cross Validation to Get the Best Value of k in KNN model
 # The function below shows us the best value for K in KNN using

```

def BestK(X, y, k_values=range(1, 21)):
    scores = []
    scaler = StandardScaler()
    # Keep only the rows in y that match X's indices
    y = y.loc[y.index.intersection(X.index)]
    X = X.loc[y.index] # Also make sure X only keeps matching rows

    X_scaled = scaler.fit_transform(X)

    for k in k_values:
        knn = KNeighborsClassifier(n_neighbors=k)
        cv_scores = cross_val_score(knn, X_scaled, y, cv=10,
                                     scores.append(np.mean(cv_scores)))

    best_index = np.argmax(scores)
    best_k = k_values[best_index]
    print("Best K:", best_k)
    print("Best Score:", scores[best_index])
    return best_k

```

✓ 2) Data Cleaning visualization

Which involves presenting data in a visual format such as charts and graphs to make it easier to understand and to identify patterns and trends in the data.

✓ Navigating & Labeling**

(2067, 507, 5087, 31)

	Subject Id	date	height_cm	weight_at_cancer_diagnosis_kg	age	bra_cup_size	band_size	smoker	smoking_duration_yrs
0	RQ00001-2	24/06/2014	164.0	78.0	62	5.0	8.0	1.0	30.0
1	RQ00002-0	08/07/2014	162.0	57.0	47	3.0	3.0	3.0	30.0
2	RQ00004-9	17/06/2014	171.0	82.0	56	5.0	9.0	0.0	NaN
3	RQ00005-4	06/08/2014	156.0	76.0	63	5.0	8.0	0.0	NaN
4	RQ00006-5	02/07/2014	164.0	59.0	68	5.0	5.0	1.0	17.0

5 rows × 507 columns

```
print(list(data1.columns))
```

```
['Subject Id', 'date', 'height_cm', 'weight_at_cancer_diagnosis_kg', 'age', 'bra_cup_size', 'band_size', 'smoker', 'smoking_
```

```
data2.head()
```

	Subject_Id	rs2101893	rs77405610	rs67342194	rs2954011	rs1788750	rs10764743	rs4859721	rs7195405	rs6009631	...	rs1
0	RQ22067-0	1.564	0.192	0.999	1.174	0.877	0.069	0.128	1.002	0.003	...	
1	RQ34365-4	1.802	0.000	0.000	1.991	2.000	0.966	1.972	1.028	0.007	...	
2	RQ34616-4	1.769	0.000	0.066	0.198	0.000	0.986	1.998	0.008	0.463	...	
3	RQ34720-1	1.672	0.773	0.993	1.307	0.995	1.856	1.103	1.971	0.845	...	
4	RQ14799-8	1.984	0.000	0.000	0.009	0.992	0.668	1.151	0.006	0.011	...	

5 rows × 31 columns

```
# Show the length of data
len(data2)
```

5087

```
# Rename the subject id column to the id in order to be merged
data2.rename(columns = {'Subject_Id':'Subject Id'}, inplace =
```

```
data2 = data2[data2['Subject Id'].isin(data1['Subject Id'])]
```

```
data2.shape
```

(1912, 31)

```
data1.describe()
```

	height_cm	weight_at_cancer_diagnosis_kg	age	bra_cup_size	band_size	smoker	smoking_duration_yrs
count	2057.000000	2053.000000	2067.000000	1997.000000	1975.000000	2045.000000	823.000000
mean	162.794361	70.110083	58.248670	3.899349	6.031899	0.749144	23.407047
std	6.820660	14.919588	11.048345	1.250805	1.887004	1.048788	13.296206
min	140.000000	36.000000	23.000000	1.000000	1.000000	0.000000	1.000000
25%	158.000000	60.000000	50.000000	3.000000	5.000000	0.000000	10.000000
50%	163.000000	68.000000	58.000000	4.000000	6.000000	0.000000	23.000000
75%	168.000000	78.000000	66.000000	5.000000	7.000000	1.000000	34.000000
max	187.000000	187.000000	90.000000	11.000000	10.000000	3.000000	64.000000

8 rows × 373 columns

data2.describe()

	rs2101893	rs77405610	rs67342194	rs2954011	rs1788750	rs10764743	rs4859721	rs7195405	rs6009631	rs...
count	1912.000000	1912.000000	1912.000000	1912.000000	1912.000000	1912.000000	1912.000000	1912.000000	1912.000000	1912.000000
mean	1.779041	0.110039	0.493405	1.112940	1.049048	0.985759	1.731134	0.680379	0.144159	0.000000
std	0.250131	0.226129	0.596734	0.593418	0.678163	0.613357	0.431270	0.640959	0.299560	0.000000
min	0.251000	0.000000	0.000000	0.000000	0.000000	0.000000	0.029000	0.000000	0.000000	0.000000
25%	1.705750	0.000000	0.000000	0.812250	0.850000	0.516500	1.282500	0.013000	0.000000	0.000000
50%	1.871000	0.001000	0.028000	1.076000	0.996000	0.997000	1.979000	0.913500	0.005000	0.000000
75%	1.954000	0.094250	0.995000	1.495000	1.835250	1.359250	1.994000	1.012000	0.071250	0.000000
max	2.000000	1.792000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000	1.981000	1.000000

8 rows × 30 columns

2.2.2) Data selection

```
data1.columns = data1.columns.str.strip() # Removes leading/trailing spaces
```

```
print(data1[dependent1].isna().sum()) # How many values missing
print(len(data1)) # Total rows
```

```
376
2067
```

```
print(data1[dependent1].dtype)
```

float64

```
# Show the unique value of specific column
data1[dependent1].unique()
```

array([0., nan, 1., 2.])

Data preprocessing

such as Data formatting, Data cleaning, Data anonymization and Data sampling.

```
# Manipulate unclear values
data1 = data1.replace('?', np.nan) # replace ? with nan
data1 = data1.replace('x', np.nan) # replace x with nan
data1 = data1.replace('0.0%', 0) # replace 0.0% with 0
data1 = data1.replace('0', 0) # replace 0 char with num
```

```
data1 = data1.replace('1', 1) # replace 1 char with num
data1 = data1.replace('2', 2) # replace 2 char with num
data1 = data1.replace('3', 3) # replace 3 char with num
data1 = data1.replace('^\\s+', np.nan, regex=True) # replace
```

```
# Handling some missing values
data1["smoker"].fillna(0, inplace=True) # notice from data sh
smokers have value 0 in smoking
data1["alcohol_intake"].fillna(0, inplace=True) # notice from
alcohol drinker dont have alcoholintaker
```

```
nullP(data1)
```

```
breast_nipple_retraction MV Indicator_96m      100.0
CRFstatus_96m                                100.0
breast_swollen_arm_interferes_with_normal_activity_96m  100.0
breast_swollen_arm_interferes_with_normal_activity_84m  100.0
breast_pain_medication_dose_and_freq_84m            100.0
...
ace_inhibitor                                     0.0
age                                                0.0
height_cm                                         0.0
date                                              0.0
Subject Id                                       0.0
Length: 507, dtype: float64
```

```
data1.shape
```

```
(2067, 507)
```

```
#Remove all features that contain 50% or more null
n_data1= X_per_removed(data1,dependent1, 50)
```

```
(2067, 182)
['Subject Id', 'date', 'height_cm', 'weight_at_cancer_diagnosis_kg', 'age', 'bra_cup_size', 'band_size', 'smoker', 'alcohol_
```

```
n_data1= remove_duplicate(n_data1)
```

```
Columns to drop:
{'Site_postRT', 'Site_baseline', 'Site_12m', 'CRFversion_12m', 'Site_24m', 'Sequence Num_baseline', 'Sequence Num_postRT', '
(2067, 171)
```

Correct datatypes in dataframe

```
n_data1.dtypes.to_csv('data1_dtypes.csv')
```

```
#uniq(n_data1)
```

```
# 4. ID columns
n_data1['Subject Id'] = n_data1['Subject Id'].astype('string')
```

```
# 5. Date columns
date_cols = [col for col in n_data1.columns if 'date' in col.
```

```
for col in date_cols:
    n_data1[col] = pd.to_datetime(n_data1[col], errors='coerc
```

```
# 6. Keep real continuous numeric columns
continuous_cols = [
    'height_cm',
    'weight_at_cancer_diagnosis_kg',
    'age',
    'monopause_age_yrs',
    'nodes_involved',
    'nodes_examined',
    'tumour_size_mm',
    'ki_67_status_pc',
    'radio_breast_dose_Gy',
    'radio_photon_dose_MV',
    'radio_breast_fractions',
    'radio_breast_ct_volume_cm3',
    'radio_breast_internal_mammary_volume_cm3',
    'radio_skin_max_dose_Gy',
    'radio_heart_mean_dose_Gy',
    'radio_ipsilateral_lung_mean_Gy',
    'radio_breast_fractions_dose_per_fraction_Gy',
    'radio_breast_fractions_per_week',
    'radio_photon_boostdose_precise_Gy',
    'radio_photon_boost_fractions',
    'radio_photon_boost_fractions_per_week',
    'radio_photon_boost_dose_per_fraction_Gy',
    'breast_left_arm_circumfrence_mm_baseline',
    'breast_right_arm_circumfrence_mm_baseline',
    'breast_left_arm_circumfrence_mm_postRT',
    'breast_right_arm_circumfrence_mm_postRT',
    'breast_left_arm_circumfrence_mm_12m',
    'breast_right_arm_circumfrence_mm_12m',
    'breast_left_arm_circumfrence_mm_24m',
    'breast_right_arm_circumfrence_mm_24m'
]

continuous_cols = [col for col in continuous_cols if col in n_data1.columns]

for col in continuous_cols:
    n_data1[col] = pd.to_numeric(n_data1[col], errors='coerce')
```

```
# 7. Everything else with low unique values becomes category
exclude_cols = ['Subject Id'] + date_cols + continuous_cols

cat_cols = [col for col in n_data1.columns if col not in exclude_cols]
```

```
for col in cat_cols:
    n_data1[col] = n_data1[col].astype('category')
```

```
# 8. Check final result
print(n_data1.dtypes)
```

```
Subject Id                string[python]
date                    datetime64[ns]
height_cm               float64
weight_at_cancer_diagnosis_kg  float64
age                     int64
...
breast_pneumontis_24m    category
breast_pain_24m          category
breast_swollen_arm_24m   category
breast_left_arm_circumfrence_mm_24m  float64
breast_right_arm_circumfrence_mm_24m  float64
Length: 171, dtype: object
```

```
# 8. Check final result
n_data1.dtypes.to_csv('data1_dtypes_clean2.csv')
```

```
# Remove Columns With Only One Unique Non-NaN Value
constant_cols = []
```

```
for col in n_data1.columns:

    unique_non_nan = n_data1[col].dropna().nunique()

    if unique_non_nan <= 1:
        constant_cols.append(col)
```

```
print("Constant columns:")
print(constant_cols)
```

```
# Drop them
n_data1 = n_data1.drop(columns=constant_cols)
n_data1.shape
```

```
Constant columns:
['Sequence Num_x', 'Visit_y', 'Sequence Num_y', 'Visit_baseline', 'CRFversion_baseline', 'Visit_12m', 'Sequence Num_12m', 'V
(2067, 162)'
```

```
# same result as above
#df_clean = df.loc[:, df.nunique(dropna=True) > 1]
```

```
print(list(n_data1.columns))
```

```
['Subject Id', 'date', 'height_cm', 'weight_at_cancer_diagnosis_kg', 'age', 'bra_cup_size', 'band_size', 'smoker', 'alcohol_
```

```
# no need to add it as it will be removed with pairwise corre.
#df['bmi'] = (df['weight_at_cancer_diagnosis_kg'] / df['heigh
```

```
# remove features with any post RT data
n_data11= n_data1[['Subject Id', 'date', 'height_cm', 'weight_
```

```
n_data11= pairwisecorr(n_data11, dependent1, threshold=0.7)
```

```
Highly correlated feature pairs > 0.7:
weight_at_cancer_diagnosis_kg and radio_breast_ct_volume_cm3: 0.73
weight_at_cancer_diagnosis_kg and breast_left_arm_circumfrence_mm_baseline: 0.81
weight_at_cancer_diagnosis_kg and breast_right_arm_circumfrence_mm_baseline: 0.82
radio_breast_dose_Gy and radio_breast_fractions: 0.98
radio_breast_dose_Gy and radio_skin_max_dose_Gy: 0.76
radio_breast_dose_Gy and radio_breast_fractions_dose_per_fraction_Gy: 0.84
radio_breast_fractions and radio_skin_max_dose_Gy: 0.74
radio_breast_fractions and radio_breast_fractions_dose_per_fraction_Gy: 0.82
```

```
radio_photon_boost_fractions_per_week and radio_photon_boost_dose_per_fraction_Gy: 0.87
breast_left_arm_circumfrence_mm_baseline and breast_right_arm_circumfrence_mm_baseline: 0.97

Removed features:
['breast_left_arm_circumfrence_mm_baseline', 'weight_at_cancer_diagnosis_kg', 'radio_breast_dose_Gy', 'radio_breast_fraction']

Final shape: (2067, 109)
```

```
print(list(n_data11.columns))
```

```
['breast_arm_lymphodema_24m', 'Subject Id', 'date', 'height_cm', 'age', 'bra_cup_size', 'band_size', 'smoker', 'alcohol_intake', 'rs11631993', 'rs13132305', 'rs179547', 'rs2101893', 'rs2954011', 'rs4859721', 'rs7195405', 'rs77405610', 'rs67342194', 'rs1788750', 'rs10764743', 'rs61912468', 'rs61912469', 'rs200456488']
```

```
print(list(data2.columns))
```

```
['Subject Id', 'rs2101893', 'rs77405610', 'rs67342194', 'rs2954011', 'rs1788750', 'rs10764743', 'rs4859721', 'rs7195405', 'rs77405610', 'rs67342194', 'rs1788750', 'rs10764743', 'rs61912468', 'rs61912469', 'rs200456488']
```

```
data2.shape
```

```
(1912, 31)
```

```
data2 = calculate_vif(data2, 5)
```

```
Dropped rs7179547 with VIF 24982.99
Dropped rs11631993 with VIF 100.95
Dropped rs2101893 with VIF 33.66
Dropped rs13132305 with VIF 23.93
Dropped rs2954011 with VIF 23.18
Dropped rs4859721 with VIF 12.01
Dropped rs200456488 with VIF 7.88
```

```
data2.shape
```

```
(1912, 24)
```

```
data2= pairwise_corr(data2, dependent1, threshold=0.7)
```

```
Highly correlated feature pairs > 0.7:
rs61912468 and rs61912469: 0.80
```

```
Removed features:
['rs61912469']
```

```
Final shape: (1912, 23)
```

```
data2.head()
```

	rs77405610	rs67342194	rs1788750	rs10764743	rs7195405	rs6009631	rs2954007	rs57288348	rs35035095	rs72977387	...	rs61912468
0	0.192	0.999	0.877	0.069	1.002	0.003	0.730	0.000	0.005	0.093	...	0.800
1	0.000	0.000	2.000	0.966	1.028	0.007	1.809	0.002	0.982	0.150	...	0.800
2	0.000	0.066	0.000	0.986	0.008	0.463	0.005	0.011	0.947	0.010	...	0.800
3	0.773	0.993	0.995	1.856	1.971	0.845	0.937	0.006	0.028	0.034	...	0.800
4	0.000	0.000	0.992	0.668	0.006	0.011	0.002	0.003	1.968	0.120	...	0.800

5 rows × 23 columns

```
data2 = data2.drop_duplicates(subset='Subject Id')
```

2.2.4) Data transformation such as Scaling, Decomposition and Aggregation


```
# Mapping data1 and data2 together by the id column

n_df = pd.merge(n_data11, data2,on = 'Subject Id', how='left')
#n_df = data1

print(n_df.shape)
```

```
(2067, 131)
```

```
# no need because already done this for both data
#uncorrelated_df= pairwise_corr(n_df, dependent1, threshold=0.1)
```

```
uncorrelated_df= n_df
```

```
# If I want to use clinic data only
#n_df= data1
```

```
# from this dataset we can divide the features into clinic, surgery
print(list(uncorrelated_df))
```

```
['breast_arm_lymphodema_24m', 'Subject Id', 'date', 'height_cm', 'age', 'bra_cup_size', 'band_size', 'smoker', 'alcohol_intake']
```

```
# Drop date-type features
#n_df = n_df.drop(columns=n_df.select_dtypes(include=['datetime']))
```

```
#n_df.columns = n_df.columns.to_flat_index()
```

```
Cuniq(uncorrelated_df, dependent1)
```

```
breast_arm_lymphodema_24m
0.0    1628
1.0     54
2.0      9
Name: count, dtype: int64
```

From above we can see that Grade 3 atrophy is contain 24 cases, g1 and g2 in clinics have the same procedure, so thought doing binary classification is the right option for Erythema.

```
# We transform the arm_lymphodema_24 column values to 0 and 1

reduced_df = uncorrelated_df.replace(dict(breast_arm_lymphodema_24m={2:1}))
```

```
Cuniq(reduced_df, dependent1)
```

```
breast_arm_lymphodema_24m
0.0    1628
1.0     63
Name: count, dtype: int64
```

```
# select categorical features in order to transform them to numeric
cat_columns = reduced_df.select_dtypes(['category']).columns
# trans the values to numeric and apply that to the data
```

```
#reduced_df[cat_columns] = reduced_df[cat_columns].apply(lambda x: x.cat.rename_categories(['no', 'yes']))

reduced_df[cat_columns] = (reduced_df[cat_columns].
                           apply(lambda x: x.cat.codes.replac
```

```
Cuniq(reduced_df, dependent1)
```

```
breast_arm_lymphodema_24m
0.0    1628
1.0     63
Name: count, dtype: int64
```

```
print(list(reduced_df.columns))
```

```
['breast_arm_lymphodema_24m', 'Subject Id', 'date', 'height_cm', 'age', 'bra_cup_size', 'band_size', 'smoker', 'alcohol_intake', 'sys_blood_pressure', 'diabetes', 'hypertension', 'depression', 'nodes_involved', 'nodes_examined', 'surgery_axilla_type', 'tumour_side_of_primary', 'radio_treated_breast', 'her2_status', 'sys_antiher2', 'sys_tamoxifen', 'sys_aromatase', 'radio_skin_delineation', 'radio_3d', 'radio_boost_sequence', 'radio_photon_boost_fractions']
```

✓ Removing Correlated Features

Feature selection: is very important to remove irrelevant features result in a better performance model, to easier understand the model and to run the model fast**

Pairwise correlated feature selection

```
Highly correlated feature pairs > 0.7:
diabetes and antidiabetic: 0.90
hypertension and other_antihypertensive_drug: 0.80
depression and antidepressant: 0.86
nodes_involved and n_stage: 0.86
nodes_examined and surgery_axilla_type: 0.73
tumour_side_of_primary and radio_treated_breast: 1.00
her2_status and sys_antiher2: 0.85
sys_tamoxifen and sys_aromatase: 0.85
radio_skin_delineation and radio_3d: 0.75
radio_boost_sequence and radio_photon_boost_fractions: 0.84
```

```
Removed features:
['tumour_side_of_primary', 'surgery_axilla_type', 'other_antihypertensive_drug', 'sys_antiher2', 'antidepressant', 'sys_tamoxifen', 'sys_aromatase', 'radio_skin_delineation', 'radio_3d', 'radio_boost_sequence', 'radio_photon_boost_fractions']
```

```
Final shape: (2067, 121)
```

```
# remove all date columns
datetime_cols = df_clean.select_dtypes(include=['datetime64'])
print(datetime_cols)
if not datetime_cols.empty:
    df_clean = df_clean.drop(columns=datetime_cols)
```

```
Index(['date', 'Date', 'surgery_date', 'radio_breast_startdate',
       'radio_breast_enddate', 'radio_boost_startdate', 'radio_boost_enddate'],
      dtype='object')
```

```
print(list(df_clean.columns))
```

```
['breast_arm_lymphodema_24m', 'Subject Id', 'height_cm', 'age', 'bra_cup_size', 'band_size', 'smoker', 'alcohol_intake', 'sys_blood_pressure', 'diabetes', 'hypertension', 'depression', 'nodes_involved', 'nodes_examined', 'surgery_axilla_type', 'tumour_side_of_primary', 'radio_treated_breast', 'her2_status', 'sys_antiher2', 'sys_tamoxifen', 'sys_aromatase', 'radio_skin_delineation', 'radio_3d', 'radio_boost_sequence', 'radio_photon_boost_fractions']
```

```
reduced_df_clinic=reduced_df[['breast_arm_lymphodema_24m', 'Subject Id', 'height_cm', 'age', 'bra_cup_size', 'band_size', 'smoker', 'alcohol_intake', 'sys_blood_pressure', 'diabetes', 'hypertension', 'depression', 'nodes_involved', 'nodes_examined', 'surgery_axilla_type', 'tumour_side_of_primary', 'radio_treated_breast', 'her2_status', 'sys_antiher2', 'sys_tamoxifen', 'sys_aromatase', 'radio_skin_delineation', 'radio_3d', 'radio_boost_sequence', 'radio_photon_boost_fractions']]

reduced_df_snp= reduced_df[['breast_arm_lymphodema_24m', 'Subject Id', 'height_cm', 'age', 'bra_cup_size', 'band_size', 'smoker', 'alcohol_intake', 'sys_blood_pressure', 'diabetes', 'hypertension', 'depression', 'nodes_involved', 'nodes_examined', 'surgery_axilla_type', 'tumour_side_of_primary', 'radio_treated_breast', 'her2_status', 'sys_antiher2', 'sys_tamoxifen', 'sys_aromatase', 'radio_skin_delineation', 'radio_3d', 'radio_boost_sequence', 'radio_photon_boost_fractions']]
```

(2067, 131)

(2067, 107)

', 99)

```
nullP(df_combined_20)
```

```

breast_arm_lymphodema_24m      18.0
her2_status                    13.0
tumour_quadrant                9.0
nodes_examined                9.0
nodes_involved                 9.0
...
radio_breast_enddate           0.0
radio_skin_delineation         0.0
breast_skin_hyperpigmentation_baseline  0.0
breast_pain_baseline           0.0
breast_arm_lymphodema_baseline  0.0
Length: 107, dtype: float64

```

```

def cross_validated_feature_selection(df, col, splits):

    print('Original Data shape is', df.shape)

    # Drop rows only where target is missing
    df = df.dropna(subset=[col])

    cols_to_drop = [col]
    if 'Subject Id' in df.columns:
        cols_to_drop.append('Subject Id')
    X = df.drop(columns=cols_to_drop)
    y = df[col]

    print('Data shape after dropping missing target:', df.shape)

    X_numeric = X.select_dtypes(include=np.number)

    # Fill missing feature values
    X_numeric = X_numeric.fillna(X_numeric.median()) # filling

    X_scaled = pd.DataFrame(MinMaxScaler().fit_transform(X_numeric),
                            columns=X_numeric.columns,
                            index=X_numeric.index)

    pipeline = Pipeline([('clf', RandomForestClassifier(n_estimators=100,
                                                         max_depth=10, min_samples_split=10))])

    skf = StratifiedKFold(n_splits=splits, shuffle=True, random_state=42)

    auc_scores = cross_val_score(pipeline, X_scaled, y, cv=skf,
                                  scoring='roc_auc')

    print(f"✅ Mean AUC with final selected features: {auc_scores.mean()}")

```

```
cross_validated_feature_selection(df_clean_20, dependent1, 20)
```

```

Original Data shape is (2067, 99)
Data shape after dropping missing target: (1691, 99)
✅ Mean AUC with final selected features: 0.8825

```

3) Feature Selection Methods

(2067, 99)

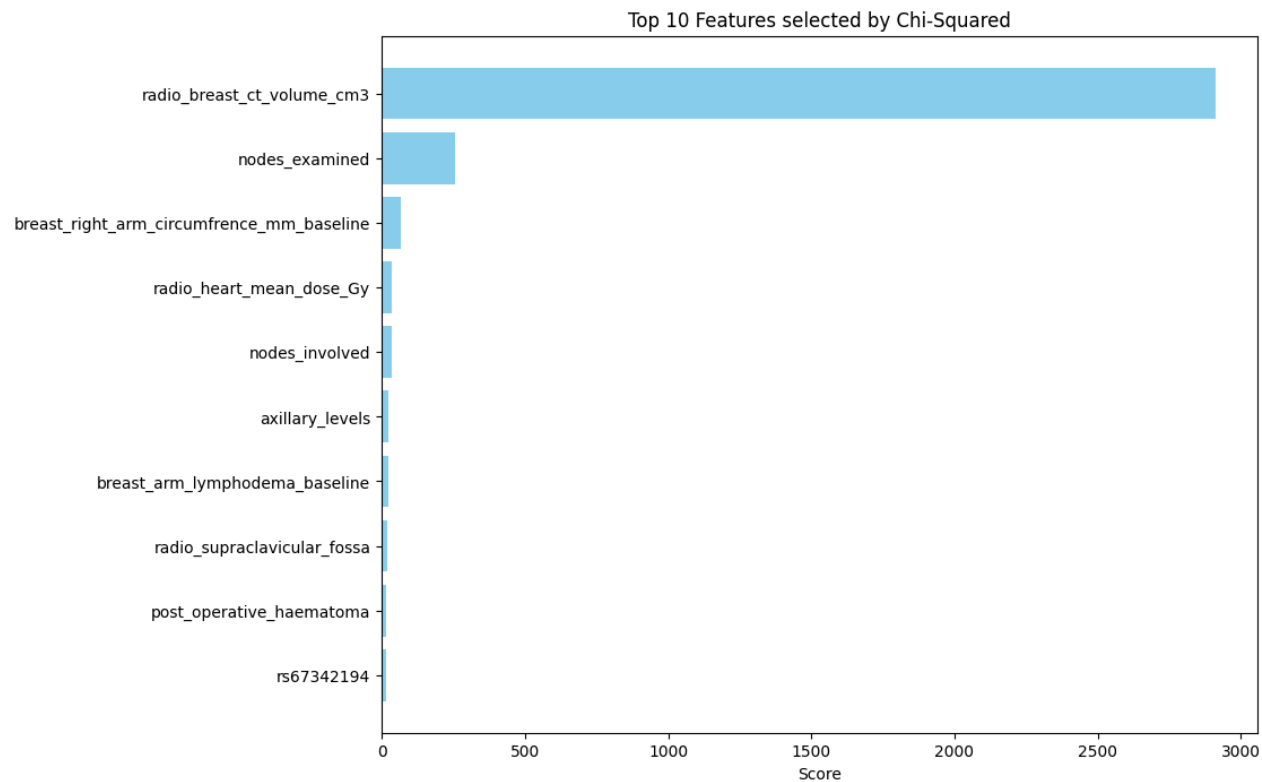
First Method:Features selction

3.3.1) Univariate Selection — Statistical tests may be used to pick certain features that have the best relationship to the performance variable. The scikit-learn library provides the SelectKBest class that can be used to select a specific number of features in a suite of different statistical tests. The below following example uses the chi-squared (chi2) statistical test for non-negative features to select 10 of the best features from the Mobile Price Range Prediction Dataset.

not scaled

```
feature_sets_univariate_nonscaled = {}

for i in k:
    freq_df_nonscaled_auc= featureselection1(df_clean_20, dep
    feature_sets_univariate_nonscaled[i] = freq_df_nonscaled_
```

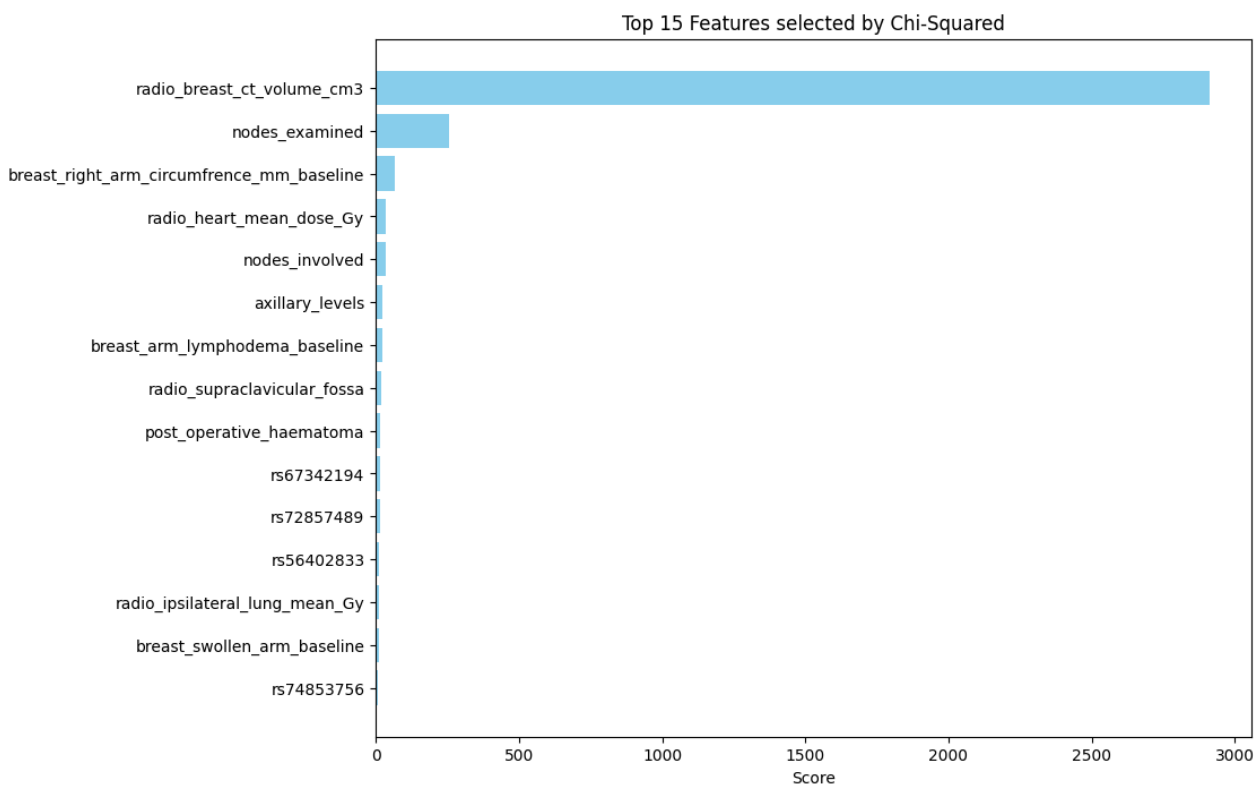



	TopFeat	Score
43	radio_breast_ct_volume_cm3	2912.71
24	nodes_examined	256.382
74	breast_right_arm_circumfrence_mm_baseline	68.0867
46	radio_heart_mean_dose_Gy	36.5315
23	nodes_involved	35.9625
54	axillary_levels	23.2121
69	breast_arm_lymphodema_baseline	21.675
55	radio_supraclavicular_fossa	21
25	post_operative_haematoma	17.321
76	rs67342194	16.1009

List of top 10 features:

['radio_breast_ct_volume_cm3', 'nodes_examined', 'breast_right_arm_circumfrence_mm_baseline', 'radio_heart_mean_dose_Gy', 'breast_arm_lymphodema_baseline', 'radio_supraclavicular_fossa', 'post_operative_haematoma', 'rs67342194', 'axillary_levels', 'nodes_involved']

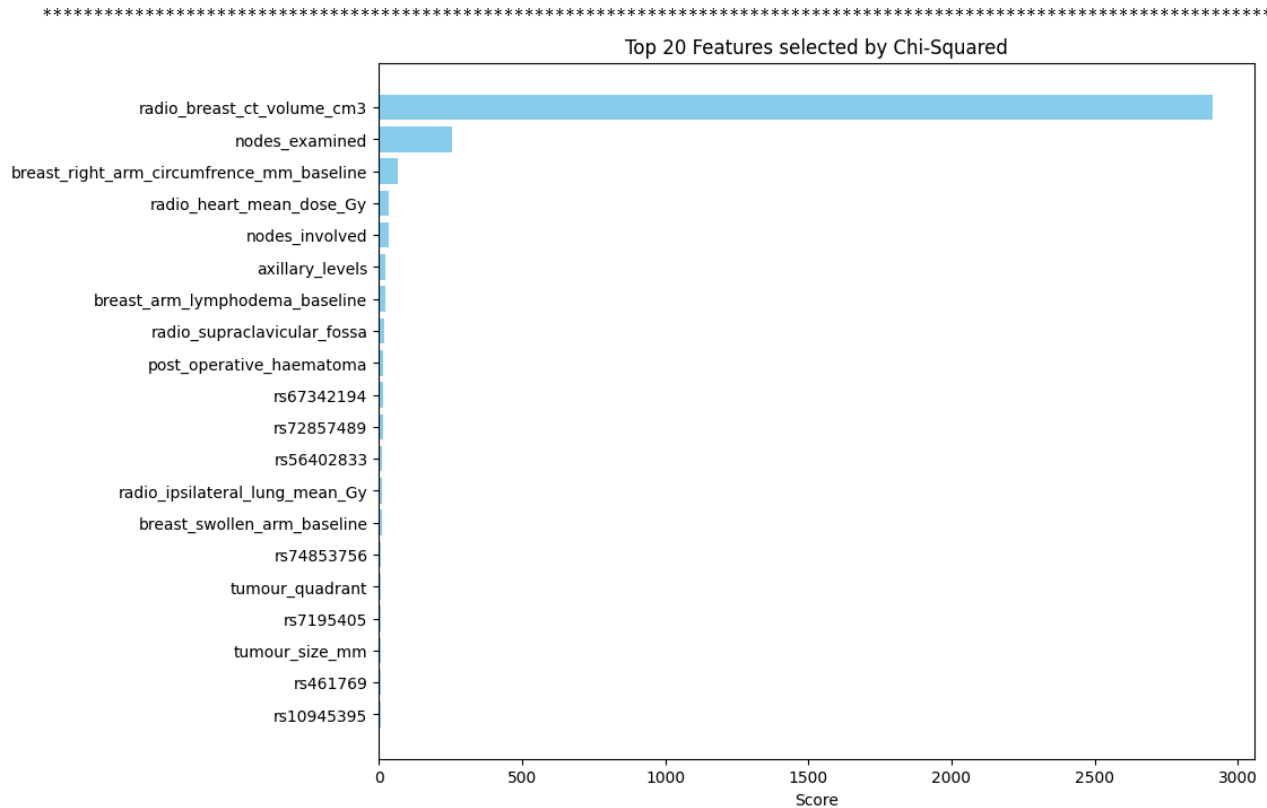
Mean AUC score using top 10 features: 0.8299



	TopFeat	Score
43	radio_breast_ct_volume_cm3	2912.71
24	nodes_examined	256.382
74	breast_right_arm_circumfrence_mm_baseline	68.0867
46	radio_heart_mean_dose_Gy	36.5315
23	nodes_involved	35.9625
54	axillary_levels	23.2121
69	breast_arm_lymphodema_baseline	21.675
55	radio_supraclavicular_fossa	21
25	post_operative_haematoma	17.321
76	rs67342194	16.1009
94	rs72857489	14.8763
85	rs56402833	13.0817
49	radio_ipsilateral_lung_mean_Gy	11.9184
73	breast_swollen_arm_baseline	11.83
90	rs74853756	8.1429

List of top 15 features:
['radio_breast_ct_volume_cm3', 'nodes_examined', 'breast_right_arm_circumfrence_mm_baseline', 'radio_heart_mean_dose_Gy', 'breast_arm_lymphodema_baseline', 'radio_supraclavicular_fossa', 'post_operative_haematoma', 'rs67342194', 'rs72857489', 'rs56402833', 'radio_ipsilateral_lung_mean_Gy', 'breast_swollen_arm_baseline', 'rs74853756', 'tumour_quadrant', 'rs7195405']

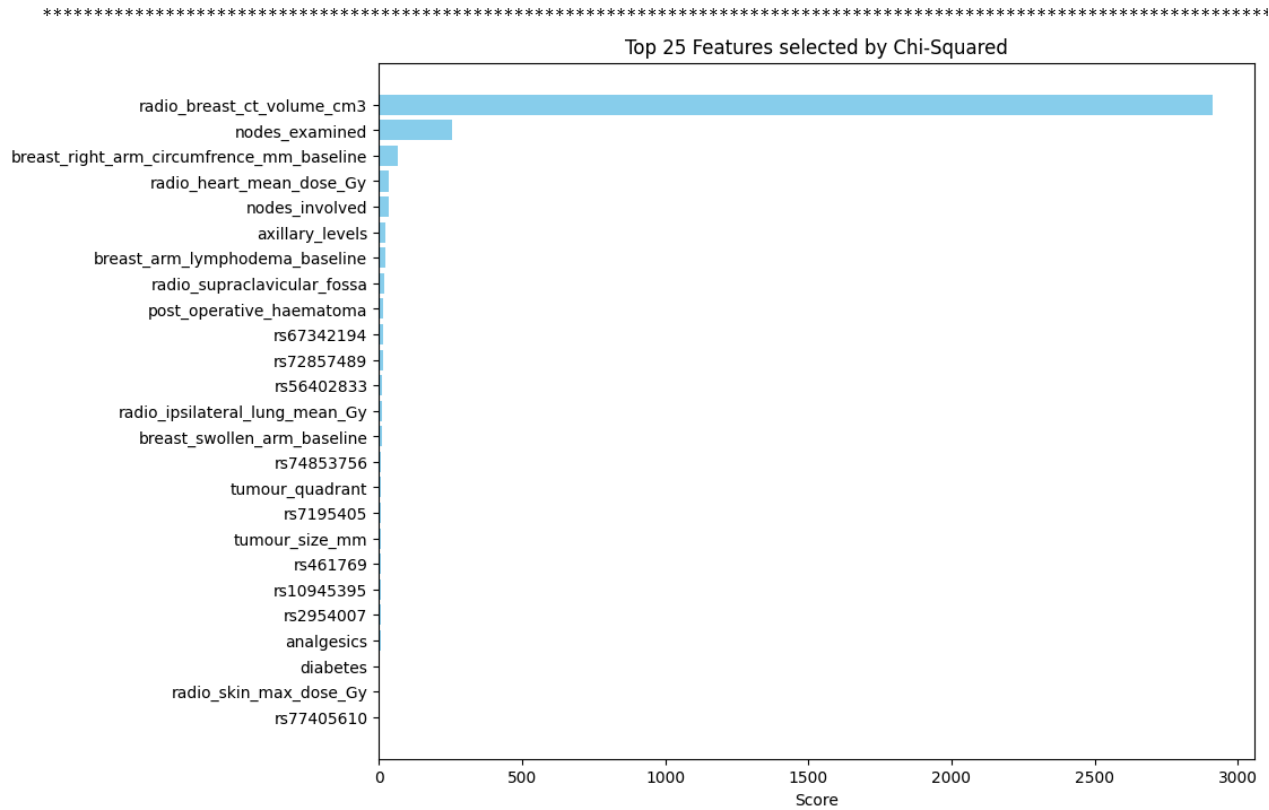
Mean AUC score using top 15 features: 0.8710



	TopFeat	Score
43	radio_breast_ct_volume_cm3	2912.71
24	nodes_examined	256.382
74	breast_right_arm_circumfrence_mm_baseline	68.0867
46	radio_heart_mean_dose_Gy	36.5315
23	nodes_involved	35.9625
54	axillary_levels	23.2121
69	breast_arm_lymphodema_baseline	21.675
55	radio_supraclavicular_fossa	21
25	post_operative_haematoma	17.321
76	rs67342194	16.1009
94	rs72857489	14.8763
85	rs56402833	13.0817
49	radio_ipsilateral_lung_mean_Gy	11.9184
73	breast_swollen_arm_baseline	11.83
90	rs74853756	8.1429
29	tumour_quadrant	8.0807
79	rs7195405	6.61387
33	tumour_size_mm	6.39883
92	rs461769	5.93069
91	rs10945395	5.8235

List of top 20 features:
['radio_breast_ct_volume_cm3', 'nodes_examined', 'breast_right_arm_circumfrence_mm_baseline', 'radio_heart_mean_dose_Gy', 'breast_arm_lymphodema_baseline', 'radio_supraclavicular_fossa', 'post_operative_haematoma', 'rs67342194', 'rs72857489', 'rs56402833', 'radio_ipsilateral_lung_mean_Gy', 'breast_swollen_arm_baseline', 'rs74853756', 'tumour_quadrant', 'rs7195405', 'tumour_size_mm', 'rs461769', 'rs10945395']

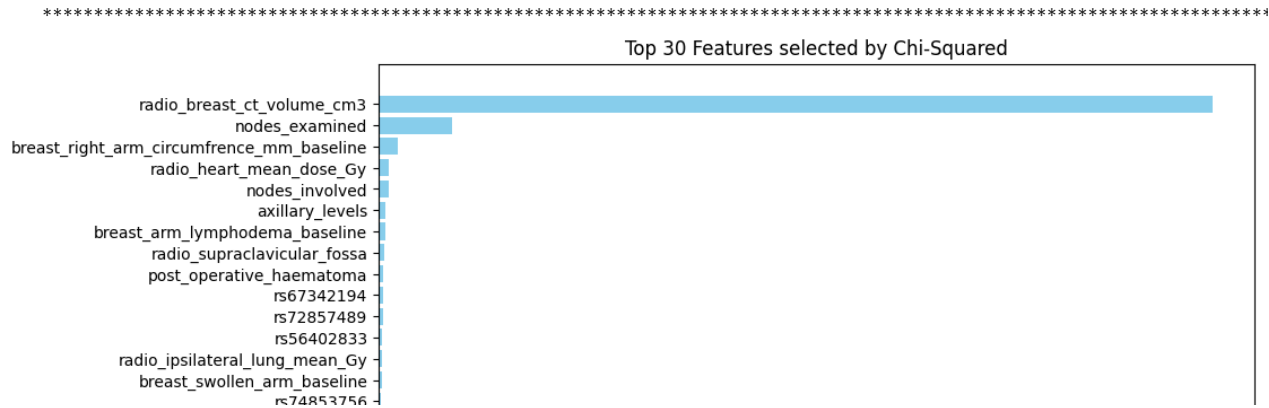
Mean AUC score using top 20 features: 0.9213

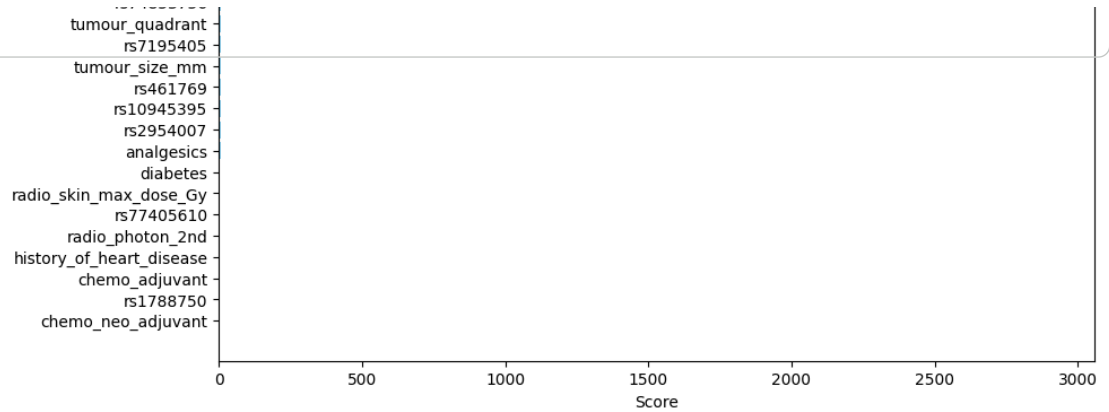


	TopFeat	Score
43	radio_breast_ct_volume_cm3	2912.71
24	nodes_examined	256.382
74	breast_right_arm_circumfrence_mm_baseline	68.0867
46	radio_heart_mean_dose_Gy	36.5315
23	nodes_involved	35.9625
54	axillary_levels	23.2121
69	breast_arm_lymphodema_baseline	21.675
55	radio_supraclavicular_fossa	21
25	post_operative_haematoma	17.321
76	rs67342194	16.1009
94	rs72857489	14.8763
85	rs56402833	13.0817
49	radio_ipsilateral_lung_mean_Gy	11.9184
73	breast_swollen_arm_baseline	11.83
90	rs74853756	8.1429
29	tumour_quadrant	8.0807
79	rs7195405	6.61387
33	tumour_size_mm	6.39883
92	rs461769	5.93069
91	rs10945395	5.8235
81	rs2954007	5.74137
18	analgesics	5.6
7	diabetes	5.28125
45	radio_skin_max_dose_Gy	5.01585
75	rs77405610	4.27777

List of top 25 features:
['radio_breast_ct_volume_cm3', 'nodes_examined', 'breast_right_arm_circumfrence_mm_baseline', 'radio_heart_mean_dose_Gy', 'nodes_involved', 'axillary_levels', 'breast_arm_lymphodema_baseline', 'radio_supraclavicular_fossa', 'post_operative_haematoma', 'rs67342194', 'rs72857489', 'rs56402833', 'radio_ipsilateral_lung_mean_Gy', 'breast_swollen_arm_baseline', 'rs74853756', 'tumour_quadrant', 'rs7195405', 'tumour_size_mm', 'rs461769', 'rs10945395', 'rs2954007', 'analgesics', 'diabetes', 'radio_skin_max_dose_Gy', 'rs77405610']

Mean AUC score using top 25 features: 0.9184





	TopFeat	Score
43	radio_breast_ct_volume_cm3	2912.71
24	nodes_examined	256.382
74	breast_right_arm_circumfrence_mm_baseline	68.0867
46	radio_heart_mean_dose_Gy	36.5315
23	nodes_involved	35.9625
54	axillary_levels	23.2121
69	breast_arm_lymphodema_baseline	21.675
55	radio_supraclavicular_fossa	21
25	post_operative_haematoma	17.321
76	rs67342194	16.1009
94	rs72857489	14.8763
85	rs56402833	13.0817
49	radio_ipsilateral_lung_mean_Gy	11.9184
73	breast_swollen_arm_baseline	11.83
90	rs74853756	8.1429
29	tumour_quadrant	8.0807
79	rs7195405	6.61387
33	tumour_size_mm	6.39883
92	rs461769	5.93069
91	rs10945395	5.8235
81	rs2954007	5.74137
18	analgesics	5.6
7	diabetes	5.28125
45	radio_skin_max_dose_Gy	5.01585
75	rs77405610	4.27777
58	radio_photon_2nd	4.05235
8	history_of_heart_disease	4
39	chemo_adjuvant	3.77978
77	rs1788750	3.43468
38	chemo_neo_adjuvant	3.2

List of top 30 features:

['radio_breast_ct_volume_cm3', 'nodes_examined', 'breast_right_arm_circumfrence_mm_baseline', 'radio_heart_mean_dose_Gy', ' '

Mean AUC score using top 30 features: 0.9086

Takeaways2: Based on the features that only has less than 20% missing values:

- Mean AUC score using top 10 features: 0.8299
- Mean AUC score using top 15 features: 0.8710
- Mean AUC score using top 20 features: 0.9213
- Mean AUC score using top 25 features: 0.9184
- Mean AUC score using top 30 features: 0.9086

```
from collections import Counter

def get_stable_features(feature_sets, threshold_ratio=0.8):
    """
    Select features appearing in at least threshold_ratio of

    Parameters
    -----
    feature_sets : dict
        Format:
        {
            k: (dataframe, auc_score)
        }

    threshold_ratio : float
        Minimum proportion of sets a feature must appear in.
        Example:
            0.8 = 80%

    Returns
    -----
    list
        Stable feature names only
    """

    feature_counter = Counter()

    n_sets = len(feature_sets)

    for k, (df, auc) in feature_sets.items():

        # Get top-k features based on score
        top_features = (
            df.sort_values('Score', ascending=False)
                .head(k)['TopFeat']
                .tolist()
        )

        feature_counter.update(top_features)
```

```
threshold = int(threshold_ratio * n_sets)

stable_features = [
    feature for feature, count in feature_counter.items()
    if count >= threshold
]

return stable_features
```

```
stable_features_univariate_nonscaled = get_stable_features(
    feature_sets_univariate_nonscaled,
    threshold_ratio=0.8
)

print(stable_features_univariate_nonscaled)
print ('Number of features', len(stable_features_univariate_nonscaled))
```

```
['radio_breast_ct_volume_cm3', 'nodes_examined', 'breast_right_arm_circumfrence_mm_baseline', 'radio_heart_mean_dose_Gy', 'r
Number of features 15
```

```
cross_validated_feature_selection(df_clean_20[stable_features_
```

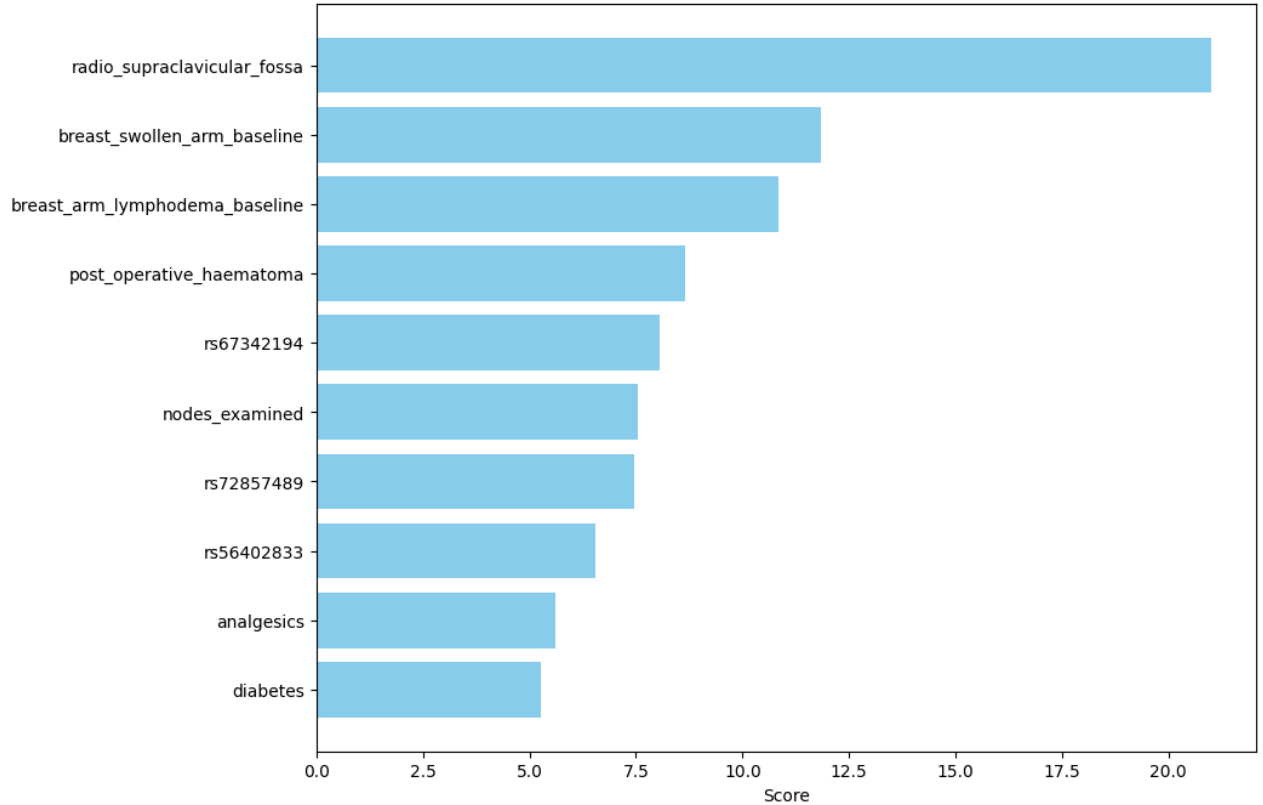
```
Original Data shape is (2067, 16)
Data shape after dropping missing target: (1691, 16)
✅ Mean AUC with final selected features: 0.8561
```

▼ scaled

```
feature_sets_univariate_scaled = {}

for i in k:
    freq_df_scaled_auc= featureselection1_scaled(df_clean_20,
    feature_sets_univariate_scaled[i] = freq_df_scaled_auc
```


Top 10 Features selected by Chi-Squared



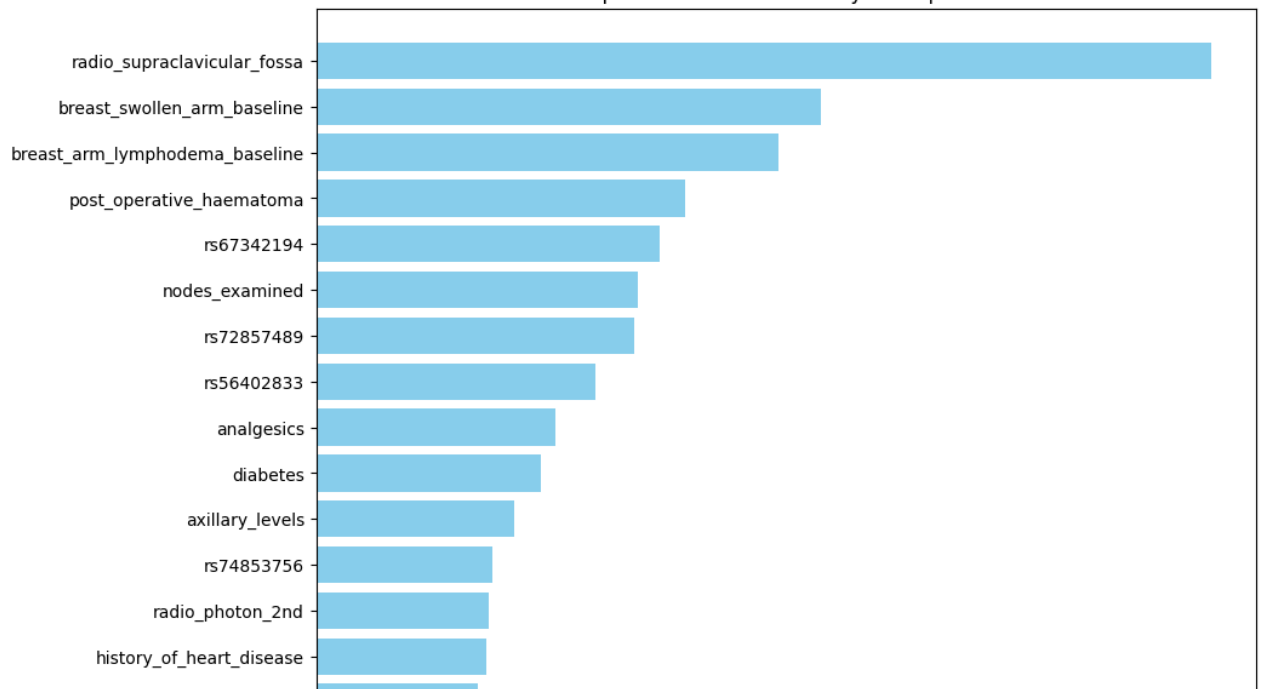
	TopFeat	Score
55	radio_supraclavicular_fossa	21
73	breast_swollen_arm_baseline	11.83
69	breast_arm_lymphodema_baseline	10.8375
25	post_operative_haematoma	8.6605
76	rs67342194	8.05045
24	nodes_examined	7.54064
94	rs72857489	7.45677
85	rs56402833	6.54087
18	analgesics	5.6
7	diabetes	5.28125

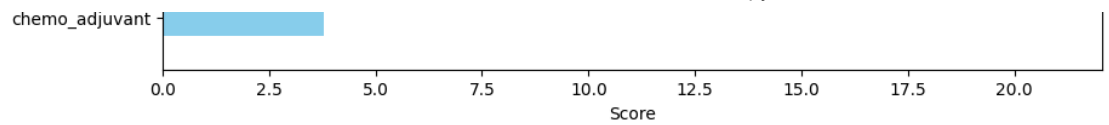
List of top 10 features:

['radio_supraclavicular_fossa', 'breast_swollen_arm_baseline', 'breast_arm_lymphodema_baseline', 'post_operative_haematoma']

Mean AUC score using top 10 features: 0.8274

Top 15 Features selected by Chi-Squared





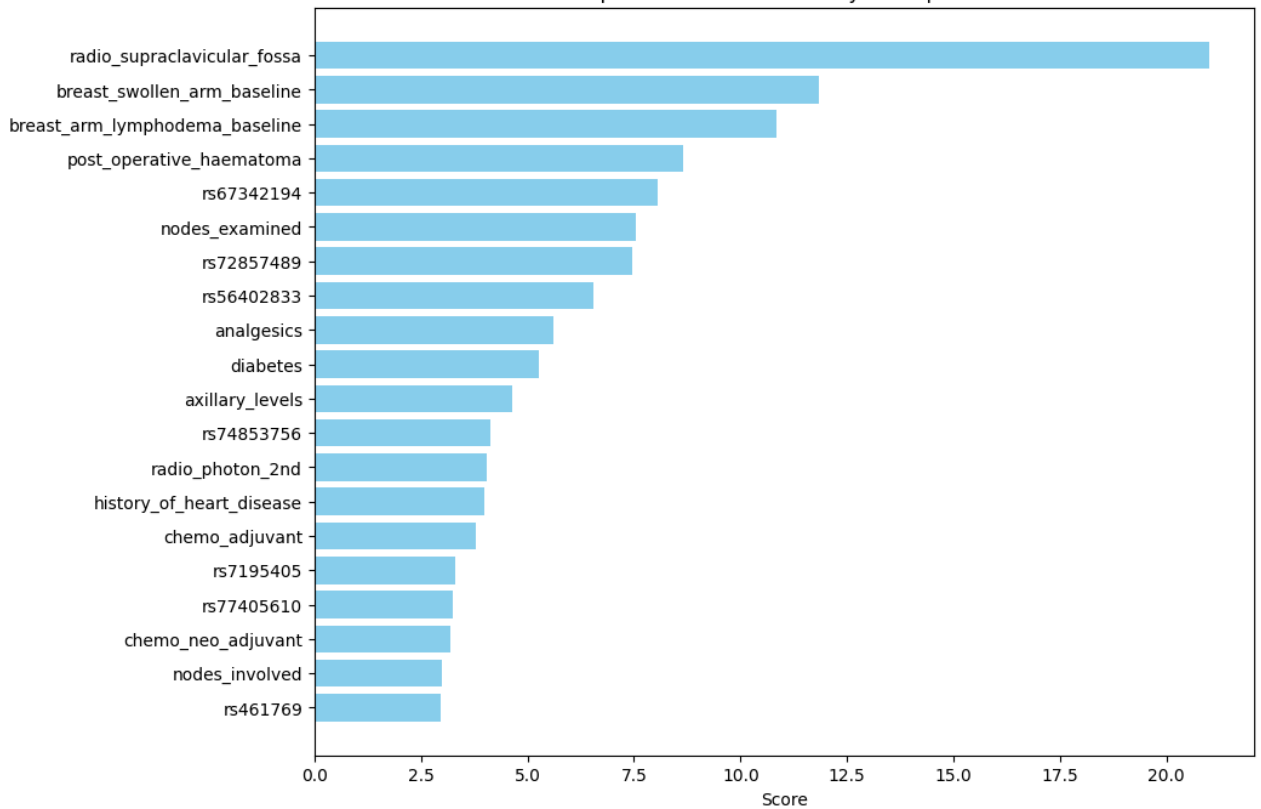
	TopFeat	Score
55	radio_supraclavicular_fossa	21
73	breast_swollen_arm_baseline	11.83
69	breast_arm_lymphodema_baseline	10.8375
25	post_operative_haematoma	8.6605
76	rs67342194	8.05045
24	nodes_examined	7.54064
94	rs72857489	7.45677
85	rs56402833	6.54087
18	analgesics	5.6
7	diabetes	5.28125
54	axillary_levels	4.64242
90	rs74853756	4.1209
58	radio_photon_2nd	4.05235
8	history_of_heart_disease	4
39	chemo_adjuvant	3.77978

List of top 15 features:

['radio_supraclavicular_fossa', 'breast_swollen_arm_baseline', 'breast_arm_lymphodema_baseline', 'post_operative_haematoma', 'rs67342194', 'nodes_examined', 'rs72857489', 'rs56402833', 'analgesics', 'diabetes', 'axillary_levels', 'rs74853756', 'radio_photon_2nd', 'history_of_heart_disease', 'chemo_adjuvant']

Mean AUC score using top 15 features: 0.8577

Top 20 Features selected by Chi-Squared



	TopFeat	Score
55	radio_supraclavicular_fossa	21
73	breast_swollen_arm_baseline	11.83
69	breast_arm_lymphodema_baseline	10.8375
25	post_operative_haematoma	8.6605
76	rs67342194	8.05045
24	nodes_examined	7.54064
94	rs72857489	7.45677
85	rs56402833	6.54087
18	analgesics	5.6
7	diabetes	5.28125
54	axillary_levels	4.64242
90	rs74853756	4.1209
58	radio_photon_2nd	4.05235
8	history_of_heart_disease	4
39	chemo_adjuvant	3.77978
79	rs7195405	3.30694
75	rs77405610	3.26049
38	chemo_neo_adjuvant	3.2
22	nodes_involved	2.80688

```

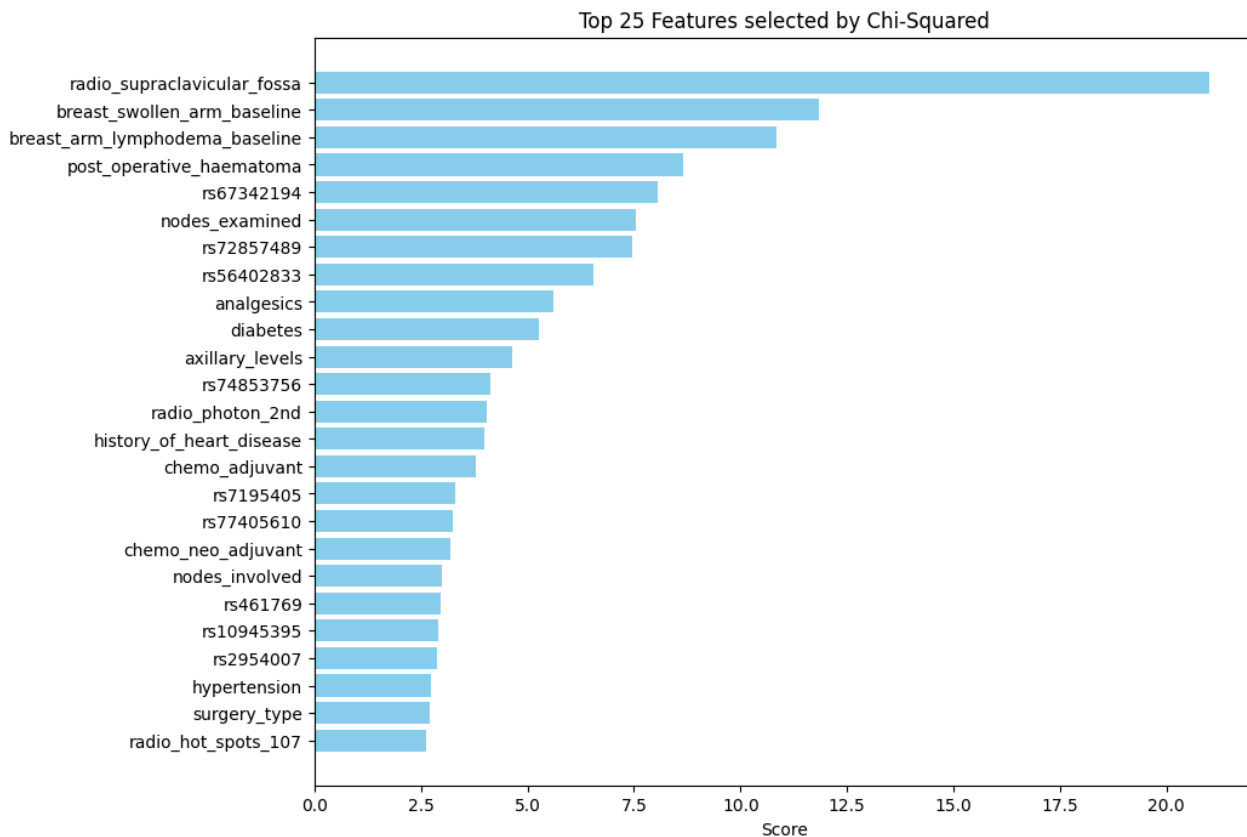
+-----+-----+
| 25 | nodes_involved | 2.99688 |
| 92 | rs461769        | 2.96534 |
+-----+-----+

```

List of top 20 features:

['radio_supraclavicular_fossa', 'breast_swollen_arm_baseline', 'breast_arm_lymphodema_baseline', 'post_operative_haematoma']

Mean AUC score using top 20 features: 0.8671



```

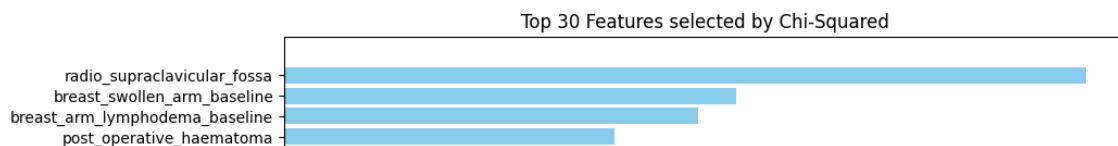
+-----+-----+-----+
| | TopFeat | | Score |
+-----+-----+-----+
| 55 | radio_supraclavicular_fossa | | 21 |
| 73 | breast_swollen_arm_baseline | | 11.83 |
| 69 | breast_arm_lymphodema_baseline | | 10.8375 |
| 25 | post_operative_haematoma | | 8.6605 |
| 76 | rs67342194 | | 8.05045 |
| 24 | nodes_examined | | 7.54064 |
| 94 | rs72857489 | | 7.45677 |
| 85 | rs56402833 | | 6.54087 |
| 18 | analgesics | | 5.6 |
| 7 | diabetes | | 5.28125 |
| 54 | axillary_levels | | 4.64242 |
| 90 | rs74853756 | | 4.1209 |
| 58 | radio_photon_2nd | | 4.05235 |
| 8 | history_of_heart_disease | | 4 |
| 39 | chemo_adjuvant | | 3.77978 |
| 79 | rs7195405 | | 3.30694 |
| 75 | rs77405610 | | 3.26049 |
| 38 | chemo_neo_adjuvant | | 3.2 |
| 23 | nodes_involved | | 2.99688 |
| 92 | rs461769 | | 2.96534 |
| 91 | rs10945395 | | 2.91175 |
| 81 | rs2954007 | | 2.88947 |
| 12 | hypertension | | 2.738 |
| 22 | surgery_type | | 2.7075 |
| 48 | radio_hot_spots_107 | | 2.62656 |
+-----+-----+-----+

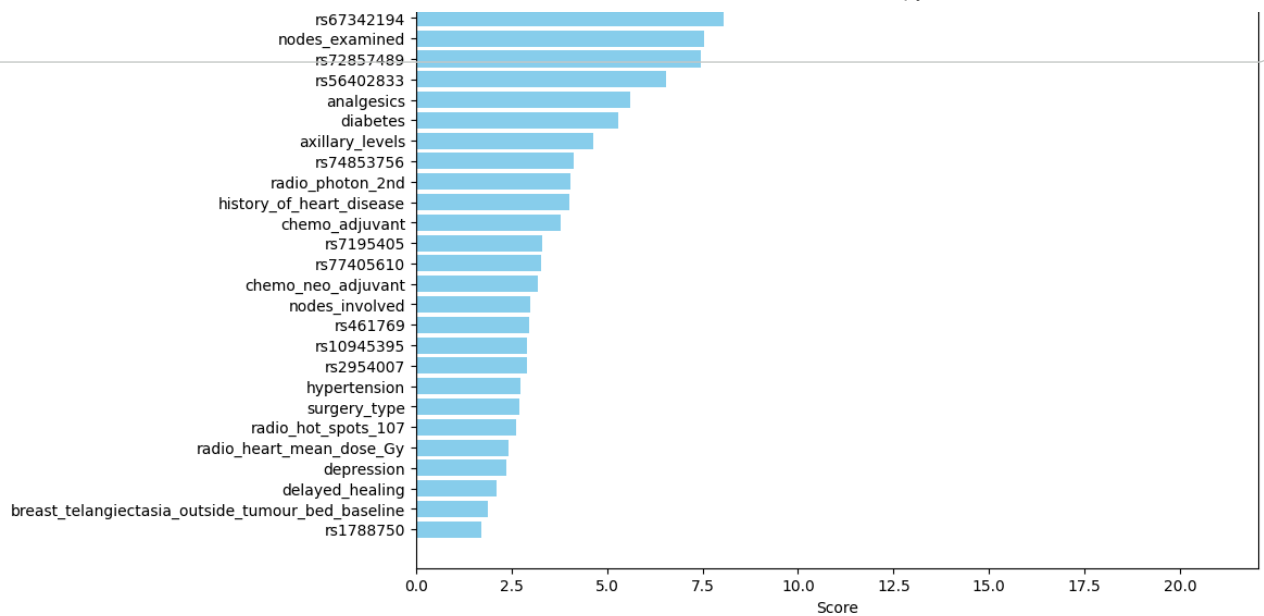
```

List of top 25 features:

['radio_supraclavicular_fossa', 'breast_swollen_arm_baseline', 'breast_arm_lymphodema_baseline', 'post_operative_haematoma']

Mean AUC score using top 25 features: 0.8914





	TopFeat	Score
55	radio_supraclavicular_fossa	21
73	breast_swollen_arm_baseline	11.83
69	breast_arm_lymphodema_baseline	10.8375
25	post_operative_haematoma	8.6605
76	rs67342194	8.05045
24	nodes_examined	7.54064
94	rs72857489	7.45677
85	rs56402833	6.54087
18	analgesics	5.6
7	diabetes	5.28125
54	axillary_levels	4.64242
90	rs74853756	4.1209
58	radio_photon_2nd	4.05235
8	history_of_heart_disease	4
39	chemo_adjuvant	3.77978
79	rs7195405	3.30694
75	rs77405610	3.26049
38	chemo_neo_adjuvant	3.2
23	nodes_involved	2.99688
92	rs461769	2.96534
91	rs10945395	2.91175
81	rs2954007	2.88947
12	hypertension	2.738
22	surgery_type	2.7075
48	radio_hot_spots_107	2.62656
46	radio_heart_mean_dose_Gy	2.43057
13	depression	2.3625
28	delayed_healing	2.1125
65	breast_telangiectasia_outside_tumour_bed_baseline	1.875
77	rs1788750	1.71734

List of top 30 features:

['radio_supraclavicular_fossa', 'breast_swollen_arm_baseline', 'breast_arm_lymphodema_baseline', 'post_operative_haematoma']

Mean AUC score using top 30 features: 0.9223

Takeaways: After scaling Based on the features that only has less than 20% missing values:

- Mean AUC score using top 10 features: 0.8274
- Mean AUC score using top 15 features: 0.8577
- Mean AUC score using top 20 features: 0.8671
- Mean AUC score using top 25 features: 0.8914
- Mean AUC score using top 30 features: 0.9223

AUC Dropped After Scaling overall, for example 0.9213 → 0.8671 (20 features) This often happens because:

- the unscaled method overfit to magnitude-driven leakage
- huge-valued variables dominated
- CV became overly optimistic

The scaled version likely reflects reality better because:

- χ^2 becomes biased toward large numeric ranges
- scores become distorted

But still has data leakage which means:

- feature selection sees the ENTIRE dataset
- including future validation folds

```
stable_features_univariate_scaled = get_stable_features(
    feature_sets_univariate_scaled,
    threshold_ratio=0.8
)
```

```
print(stable_features_univariate_scaled)
print('Number of features', len(stable_features_univariate_scaled))
```

```
['radio_supraclavicular_fossa', 'breast_swollen_arm_baseline', 'breast_arm_lymphodema_baseline', 'post_operative_haematoma',
Number of features 15]
```

```
cross_validated_feature_selection(df_clean_20[stable_features_scaled])
```

```
Original Data shape is (2067, 16)
Data shape after dropping missing target: (1691, 16)
✅ Mean AUC with final selected features: 0.8591
```

inside CV

```
# This is the one I took for my feature selection heatmap # TL
# without seeing any features_quick way to see the results
for i in k:
    Mean_auc_scores_CV_inside_pipeline= featureselection1_pipeline
```

```
Mean AUC score using top 10 features (Inside CV): 0.83
Mean AUC score using top 15 features (Inside CV): 0.83
Mean AUC score using top 20 features (Inside CV): 0.83
Mean AUC score using top 25 features (Inside CV): 0.86
Mean AUC score using top 30 features (Inside CV): 0.87
```

- Mean AUC score using top 10 features (no leakage): 0.83
- Mean AUC score using top 15 features (no leakage): 0.83
- Mean AUC score using top 20 features (no leakage): 0.83

- Mean AUC score using top 25 features (no leakage): 0.86
- Mean AUC score using top 30 features (no leakage): 0.87

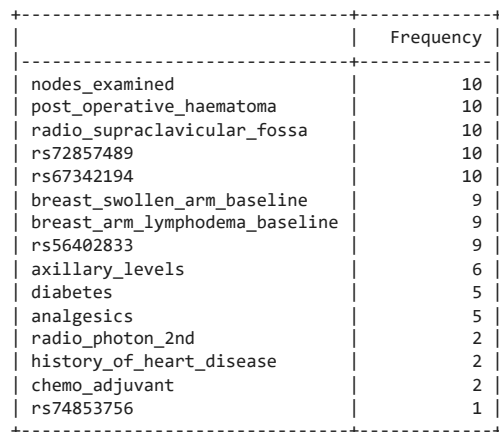
Takeaways4, in above analysis this is a real and true feature selection as only happens in the training set,,, But there is an issue because it is not only model building I need to look at the important features and observe their importance.

✓ features frequency inside Pipeline

```
feature_sets_univariate_scaled_inside_pipeline = {}

for i in k:
    freq_df_scaled_auc= featureselection1_scaled_inside_pipeline[i]
    feature_sets_univariate_scaled_inside_pipeline[i] = freq_df_scaled_auc
```

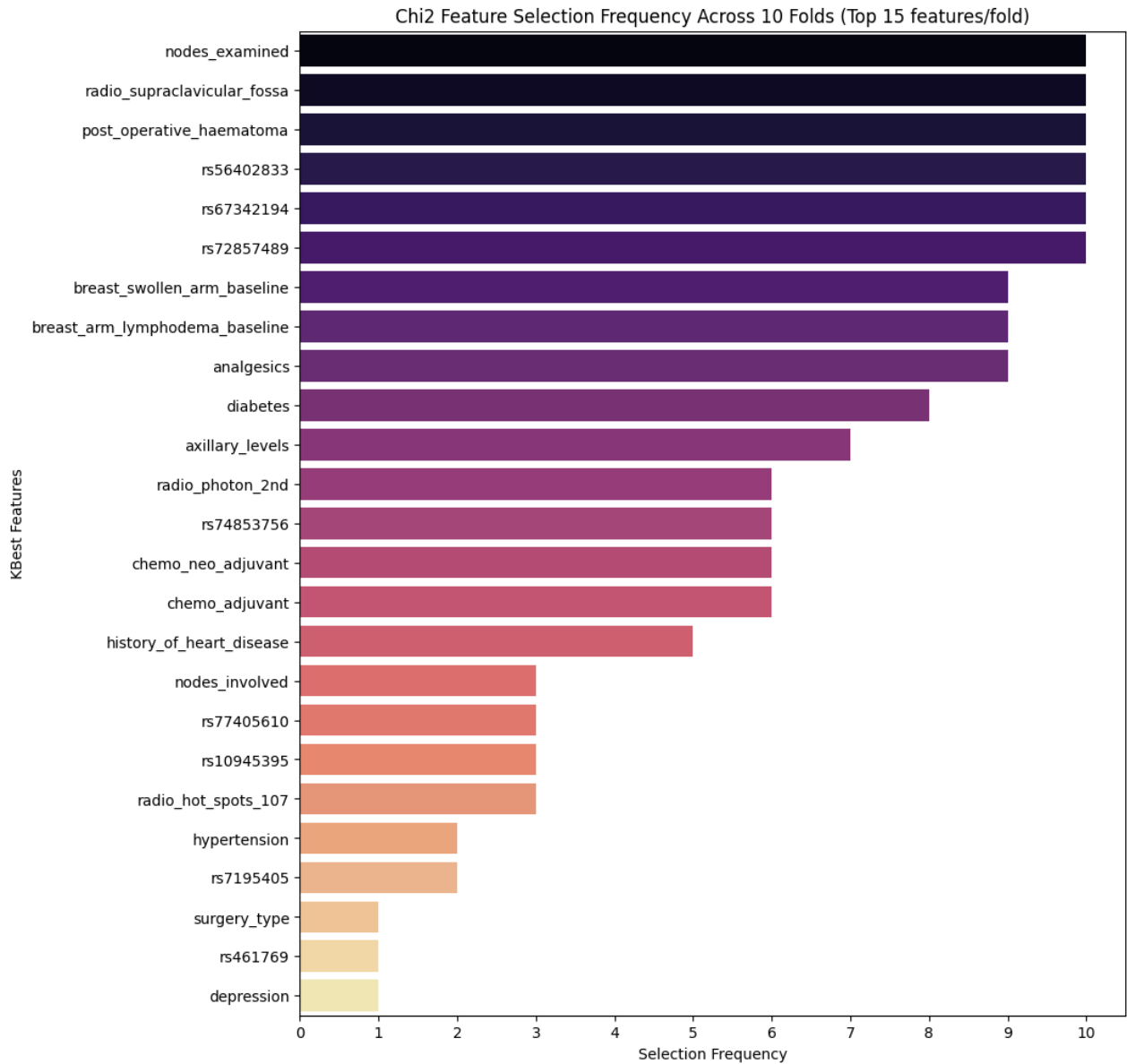

Mean AUC using top 10 features across 10 folds: 0.8317



37/153

Fold 2: Selected features: ['diabetes', 'hypertension', 'analgesics', 'nodes_examined', 'post_operative_haematoma', 'chemo_neo_adjuvant', 'chemo_adjuvant', 'history_of_heart_disease', 'rs77405610', 'rs10945395', 'radio_hot_spots_107', 'hypertension', 'rs7195405', 'surgery_type', 'rs461769', 'depression']
Fold 3: Selected features: ['diabetes', 'history_of_heart_disease', 'nodes_examined', 'post_operative_haematoma', 'chemo_neo_adjuvant', 'chemo_adjuvant', 'analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'chemo_neo_adjuvant', 'chemo_adjuvant', 'history_of_heart_disease', 'rs77405610', 'rs10945395', 'radio_hot_spots_107', 'hypertension', 'rs7195405', 'surgery_type', 'rs461769', 'depression']
Fold 4: Selected features: ['diabetes', 'history_of_heart_disease', 'analgesics', 'nodes_examined', 'post_operative_haematoma', 'chemo_neo_adjuvant', 'chemo_adjuvant', 'history_of_heart_disease', 'rs77405610', 'rs10945395', 'radio_hot_spots_107', 'hypertension', 'rs7195405', 'surgery_type', 'rs461769', 'depression']
Fold 5: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'analgesics', 'nodes_examined', 'post_operative_haematoma', 'chemo_neo_adjuvant', 'chemo_adjuvant', 'history_of_heart_disease', 'rs77405610', 'rs10945395', 'radio_hot_spots_107', 'hypertension', 'rs7195405', 'surgery_type', 'rs461769', 'depression']
Fold 6: Selected features: ['diabetes', 'analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'chemo_neo_adjuvant', 'chemo_adjuvant', 'history_of_heart_disease', 'rs77405610', 'rs10945395', 'radio_hot_spots_107', 'hypertension', 'rs7195405', 'surgery_type', 'rs461769', 'depression']
Fold 7: Selected features: ['diabetes', 'history_of_heart_disease', 'analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'chemo_neo_adjuvant', 'chemo_adjuvant', 'history_of_heart_disease', 'rs77405610', 'rs10945395', 'radio_hot_spots_107', 'hypertension', 'rs7195405', 'surgery_type', 'rs461769', 'depression']
Fold 8: Selected features: ['diabetes', 'analgesics', 'nodes_examined', 'post_operative_haematoma', 'chemo_neo_adjuvant', 'chemo_adjuvant', 'history_of_heart_disease', 'rs77405610', 'rs10945395', 'radio_hot_spots_107', 'hypertension', 'rs7195405', 'surgery_type', 'rs461769', 'depression']
Fold 9: Selected features: ['depression', 'analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'chemo_neo_adjuvant', 'chemo_adjuvant', 'history_of_heart_disease', 'rs77405610', 'rs10945395', 'radio_hot_spots_107', 'hypertension', 'rs7195405', 'surgery_type', 'rs461769', 'depression']
Fold 10: Selected features: ['analgesics', 'nodes_examined', 'post_operative_haematoma', 'chemo_neo_adjuvant', 'chemo_adjuvant', 'history_of_heart_disease', 'rs77405610', 'rs10945395', 'radio_hot_spots_107', 'hypertension', 'rs7195405', 'surgery_type', 'rs461769', 'depression']

Mean AUC using top 15 features across 10 folds: 0.8333

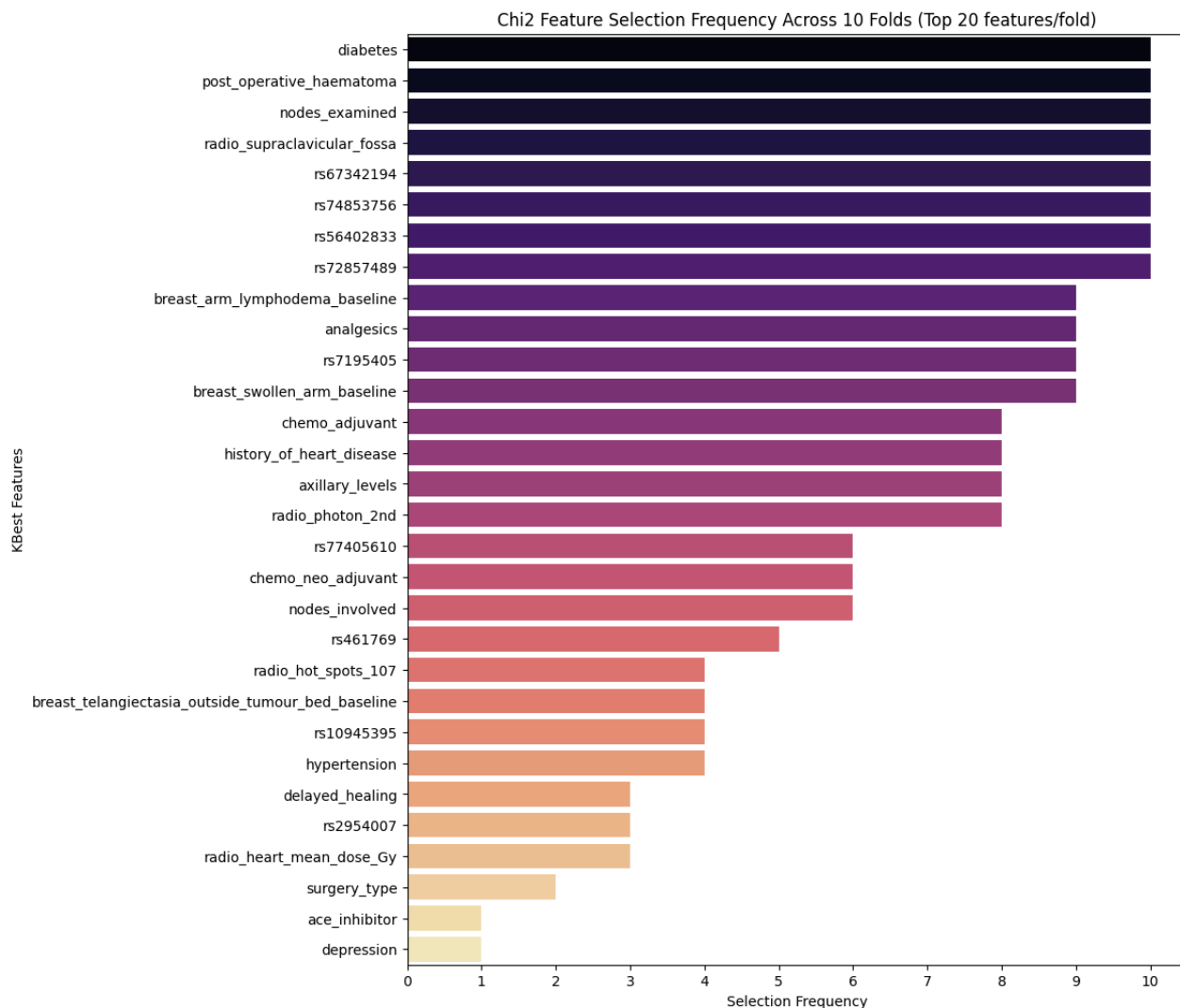


	Frequency
nodes_examined	10
radio_supraclavicular_fossa	10
post_operative_haematoma	10
rs56402833	10
rs67342194	10
rs72857489	10
breast_swollen_arm_baseline	9
breast_arm_lymphodema_baseline	9
analgesics	9
diabetes	8
axillary_levels	7
radio_photon_2nd	6
rs74853756	6
chemo_neo_adjuvant	6
chemo_adjuvant	6
history_of_heart_disease	5
nodes_involved	3
rs77405610	3
rs10945395	3
radio_hot_spots_107	3
hypertension	2
rs7195405	2
surgery_type	1
rs461769	1

```
| depression | 1 |
+-----+-----+
Top 15 most frequently Features across 10 folds: ['nodes_examined', 'radio_supraclavicular_fossa', 'post_operative_haematoma', 'diabetes', 'history_of_heart_disease', 'analgesics', 'surgery_type', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs67342194', 'rs74853756', 'rs56402833', 'rs72857489']

Fold 1: Selected features: ['diabetes', 'history_of_heart_disease', 'analgesics', 'surgery_type', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs67342194', 'rs74853756', 'rs56402833', 'rs72857489', 'breast_arm_lymphodema_baseline', 'analgesics', 'rs7195405', 'breast_swollen_arm_baseline', 'chemo_adjuvant', 'history_of_heart_disease', 'axillary_levels', 'radio_photon_2nd', 'rs77405610', 'chemo_neo_adjuvant', 'nodes_involved', 'rs461769', 'radio_hot_spots_107', 'breast_telangiectasia_outside_tumour_bed_baseline', 'rs10945395', 'hypertension', 'delayed_healing', 'rs2954007', 'radio_heart_mean_dose_Gy', 'surgery_type', 'ace_inhibitor', 'depression']
Fold 2: Selected features: ['diabetes', 'hypertension', 'ace_inhibitor', 'analgesics', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs67342194', 'rs74853756', 'rs56402833', 'rs72857489', 'breast_arm_lymphodema_baseline', 'analgesics', 'rs7195405', 'breast_swollen_arm_baseline', 'chemo_adjuvant', 'history_of_heart_disease', 'axillary_levels', 'radio_photon_2nd', 'rs77405610', 'chemo_neo_adjuvant', 'nodes_involved', 'rs461769', 'radio_hot_spots_107', 'breast_telangiectasia_outside_tumour_bed_baseline', 'rs10945395', 'hypertension', 'delayed_healing', 'rs2954007', 'radio_heart_mean_dose_Gy', 'surgery_type', 'ace_inhibitor', 'depression']
Fold 3: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs67342194', 'rs74853756', 'rs56402833', 'rs72857489', 'breast_arm_lymphodema_baseline', 'analgesics', 'rs7195405', 'breast_swollen_arm_baseline', 'chemo_adjuvant', 'history_of_heart_disease', 'axillary_levels', 'radio_photon_2nd', 'rs77405610', 'chemo_neo_adjuvant', 'nodes_involved', 'rs461769', 'radio_hot_spots_107', 'breast_telangiectasia_outside_tumour_bed_baseline', 'rs10945395', 'hypertension', 'delayed_healing', 'rs2954007', 'radio_heart_mean_dose_Gy', 'surgery_type', 'ace_inhibitor', 'depression']
Fold 4: Selected features: ['diabetes', 'history_of_heart_disease', 'analgesics', 'surgery_type', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs67342194', 'rs74853756', 'rs56402833', 'rs72857489', 'breast_arm_lymphodema_baseline', 'analgesics', 'rs7195405', 'breast_swollen_arm_baseline', 'chemo_adjuvant', 'history_of_heart_disease', 'axillary_levels', 'radio_photon_2nd', 'rs77405610', 'chemo_neo_adjuvant', 'nodes_involved', 'rs461769', 'radio_hot_spots_107', 'breast_telangiectasia_outside_tumour_bed_baseline', 'rs10945395', 'hypertension', 'delayed_healing', 'rs2954007', 'radio_heart_mean_dose_Gy', 'surgery_type', 'ace_inhibitor', 'depression']
Fold 5: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'analgesics', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs67342194', 'rs74853756', 'rs56402833', 'rs72857489', 'breast_arm_lymphodema_baseline', 'analgesics', 'rs7195405', 'breast_swollen_arm_baseline', 'chemo_adjuvant', 'history_of_heart_disease', 'axillary_levels', 'radio_photon_2nd', 'rs77405610', 'chemo_neo_adjuvant', 'nodes_involved', 'rs461769', 'radio_hot_spots_107', 'breast_telangiectasia_outside_tumour_bed_baseline', 'rs10945395', 'hypertension', 'delayed_healing', 'rs2954007', 'radio_heart_mean_dose_Gy', 'surgery_type', 'ace_inhibitor', 'depression']
Fold 6: Selected features: ['diabetes', 'history_of_heart_disease', 'analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs67342194', 'rs74853756', 'rs56402833', 'rs72857489', 'breast_arm_lymphodema_baseline', 'analgesics', 'rs7195405', 'breast_swollen_arm_baseline', 'chemo_adjuvant', 'history_of_heart_disease', 'axillary_levels', 'radio_photon_2nd', 'rs77405610', 'chemo_neo_adjuvant', 'nodes_involved', 'rs461769', 'radio_hot_spots_107', 'breast_telangiectasia_outside_tumour_bed_baseline', 'rs10945395', 'hypertension', 'delayed_healing', 'rs2954007', 'radio_heart_mean_dose_Gy', 'surgery_type', 'ace_inhibitor', 'depression']
Fold 7: Selected features: ['diabetes', 'history_of_heart_disease', 'analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs67342194', 'rs74853756', 'rs56402833', 'rs72857489', 'breast_arm_lymphodema_baseline', 'analgesics', 'rs7195405', 'breast_swollen_arm_baseline', 'chemo_adjuvant', 'history_of_heart_disease', 'axillary_levels', 'radio_photon_2nd', 'rs77405610', 'chemo_neo_adjuvant', 'nodes_involved', 'rs461769', 'radio_hot_spots_107', 'breast_telangiectasia_outside_tumour_bed_baseline', 'rs10945395', 'hypertension', 'delayed_healing', 'rs2954007', 'radio_heart_mean_dose_Gy', 'surgery_type', 'ace_inhibitor', 'depression']
Fold 8: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs67342194', 'rs74853756', 'rs56402833', 'rs72857489', 'breast_arm_lymphodema_baseline', 'analgesics', 'rs7195405', 'breast_swollen_arm_baseline', 'chemo_adjuvant', 'history_of_heart_disease', 'axillary_levels', 'radio_photon_2nd', 'rs77405610', 'chemo_neo_adjuvant', 'nodes_involved', 'rs461769', 'radio_hot_spots_107', 'breast_telangiectasia_outside_tumour_bed_baseline', 'rs10945395', 'hypertension', 'delayed_healing', 'rs2954007', 'radio_heart_mean_dose_Gy', 'surgery_type', 'ace_inhibitor', 'depression']
Fold 9: Selected features: ['diabetes', 'history_of_heart_disease', 'depression', 'analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs67342194', 'rs74853756', 'rs56402833', 'rs72857489', 'breast_arm_lymphodema_baseline', 'analgesics', 'rs7195405', 'breast_swollen_arm_baseline', 'chemo_adjuvant', 'history_of_heart_disease', 'axillary_levels', 'radio_photon_2nd', 'rs77405610', 'chemo_neo_adjuvant', 'nodes_involved', 'rs461769', 'radio_hot_spots_107', 'breast_telangiectasia_outside_tumour_bed_baseline', 'rs10945395', 'hypertension', 'delayed_healing', 'rs2954007', 'radio_heart_mean_dose_Gy', 'surgery_type', 'ace_inhibitor', 'depression']
Fold 10: Selected features: ['diabetes', 'analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs67342194', 'rs74853756', 'rs56402833', 'rs72857489', 'breast_arm_lymphodema_baseline', 'analgesics', 'rs7195405', 'breast_swollen_arm_baseline', 'chemo_adjuvant', 'history_of_heart_disease', 'axillary_levels', 'radio_photon_2nd', 'rs77405610', 'chemo_neo_adjuvant', 'nodes_involved', 'rs461769', 'radio_hot_spots_107', 'breast_telangiectasia_outside_tumour_bed_baseline', 'rs10945395', 'hypertension', 'delayed_healing', 'rs2954007', 'radio_heart_mean_dose_Gy', 'surgery_type', 'ace_inhibitor', 'depression']

Mean AUC using top 20 features across 10 folds: 0.8322
```



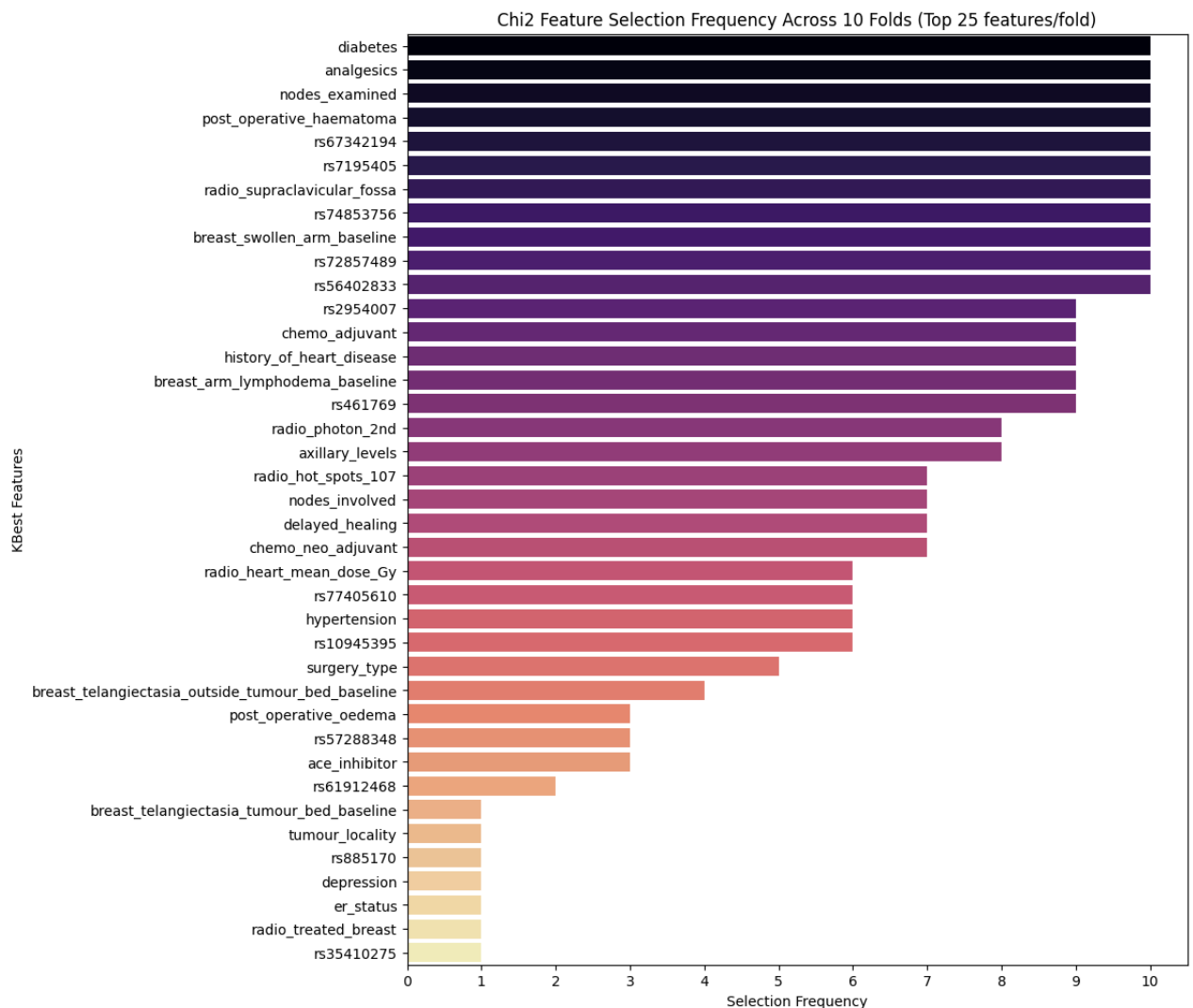
	Frequency
diabetes	10
post_operative_haematoma	10
nodes_examined	10
radio_supraclavicular_fossa	10
rs67342194	10
rs74853756	10
rs56402833	10
rs72857489	10
breast_arm_lymphodema_baseline	9
analgesics	9
rs7195405	9
breast_swollen_arm_baseline	9
chemo_adjuvant	8
history_of_heart_disease	8
axillary_levels	8
radio_photon_2nd	8
rs77405610	6
chemo_neo_adjuvant	6
nodes_involved	6
rs461769	5
radio_hot_spots_107	4

breast_telangiectasia_outside_tumour_bed_baseline	4
rs10945395	4
hypertension	4
delayed_healing	3
rs2954007	3
radio_heart_mean_dose_Gy	3
surgery_type	2
ace_inhibitor	1
depression	1

Top 20 most frequently Features across 10 folds: ['diabetes', 'post_operative_haematoma', 'nodes_examined', 'radio_supraclav

Fold 1: Selected features: ['diabetes', 'history_of_heart_disease', 'analgesics', 'surgery_type', 'nodes_examined', 'post_op
 Fold 2: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'ace_inhibitor', 'analgesics', 'nodes_ir
 Fold 3: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'analgesics', 'nodes_examined', 'post_op
 Fold 4: Selected features: ['diabetes', 'history_of_heart_disease', 'analgesics', 'surgery_type', 'nodes_involved', 'nodes_e
 Fold 5: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'ace_inhibitor', 'analgesics', 'surgery_
 Fold 6: Selected features: ['diabetes', 'history_of_heart_disease', 'analgesics', 'surgery_type', 'nodes_involved', 'nodes_e
 Fold 7: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'analgesics', 'surgery_type', 'nodes_inv
 Fold 8: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'ace_inhibitor', 'analgesics', 'nodes_ir
 Fold 9: Selected features: ['diabetes', 'history_of_heart_disease', 'depression', 'analgesics', 'nodes_involved', 'nodes_exa
 Fold 10: Selected features: ['diabetes', 'hypertension', 'analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_f

Mean AUC using top 25 features across 10 folds: 0.8638



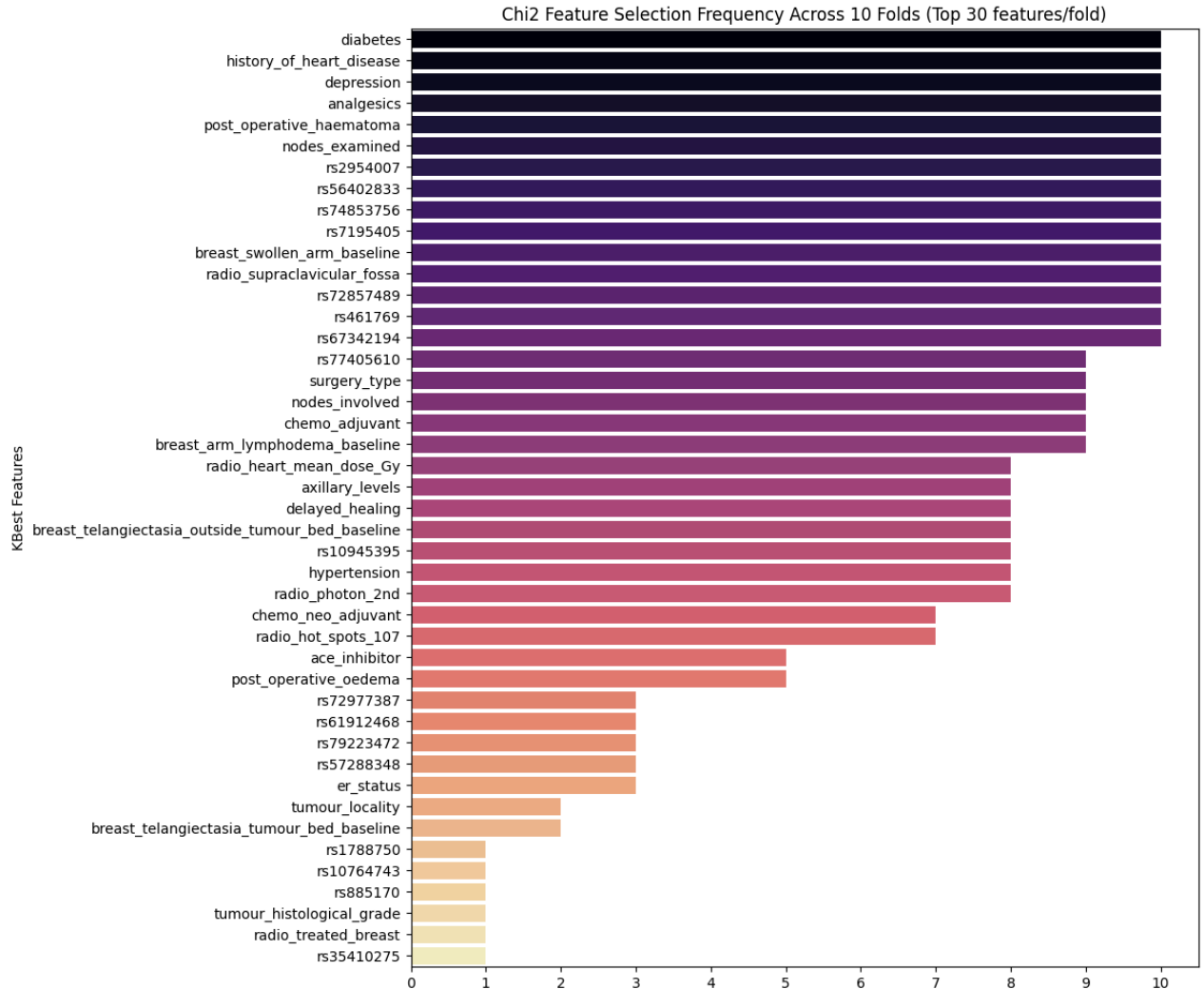
	Frequency
diabetes	10
analgesics	10
nodes_examined	10
post_operative_haematoma	10
rs67342194	10
rs7195405	10
radio_supraclavicular_fossa	10
rs74853756	10
breast_swollen_arm_baseline	10
rs72857489	10
rs56402833	10
rs2954007	9
chemo_adjuvant	9

history_of_heart_disease	9
breast_arm_lymphodema_baseline	9
rs461769	9
radio_photon_2nd	8
axillary_levels	8
radio_hot_spots_107	7
nodes_involved	7
delayed_healing	7
chemo_neo_adjuvant	7
radio_heart_mean_dose_Gy	6
rs77405610	6
hypertension	6
rs10945395	6
surgery_type	5
breast_telangiectasia_outside_tumour_bed_baseline	4
post_operative_oedema	3
rs57288348	3
ace_inhibitor	3
rs61912468	2
breast_telangiectasia_tumour_bed_baseline	1
tumour_locality	1
rs885170	1
depression	1
er_status	1
radio_treated_breast	1
rs35410275	1

Top 25 most frequently Features across 10 folds: ['diabetes', 'analgesics', 'nodes_examined', 'post_operative_haematoma', 'r

Fold 1: Selected features: ['diabetes', 'history_of_heart_disease', 'depression', 'ace_inhibitor', 'analgesics', 'surgery_ty
 Fold 2: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'depression', 'ace_inhibitor', 'analgesi
 Fold 3: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'depression', 'analgesics', 'nodes_exami
 Fold 4: Selected features: ['diabetes', 'history_of_heart_disease', 'depression', 'analgesics', 'surgery_type', 'nodes_invol
 Fold 5: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'depression', 'ace_inhibitor', 'analgesi
 Fold 6: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'depression', 'analgesics', 'surgery_ty
 Fold 7: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'depression', 'ace_inhibitor', 'analgesi
 Fold 8: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'depression', 'ace_inhibitor', 'analgesi
 Fold 9: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'depression', 'analgesics', 'surgery_ty
 Fold 10: Selected features: ['diabetes', 'history_of_heart_disease', 'hypertension', 'depression', 'analgesics', 'surgery_ty

Mean AUC using top 30 features across 10 folds: 0.8717



Selection Frequency

	Frequency
diabetes	10
history_of_heart_disease	10
depression	10
analgesics	10
post_operative_haematoma	10
nodes_examined	10
rs2954007	10
rs56402833	10
rs74853756	10
rs7195405	10
breast_swollen_arm_baseline	10
radio_supraclavicular_fossa	10
rs72857489	10
rs461769	10
rs67342194	10
rs77405610	9
surgery_type	9
nodes_involved	9
chemo_adjuvant	9
breast_arm_lymphodema_baseline	9
radio_heart_mean_dose_Gy	8
axillary_levels	8
delayed_healing	8
breast_telangiectasia_outside_tumour_bed_baseline	8
rs10945395	8
hypertension	8
radio_photon_2nd	8
chemo_neo_adjuvant	7
radio_hot_spots_107	7
ace_inhibitor	5
post_operative_oedema	5
rs72977387	3
rs61912468	3
rs79223472	3
rs57288348	3
er_status	3
tumour_locality	2
breast_telangiectasia_tumour_bed_baseline	2
rs1788750	1
rs10764743	1
rs885170	1
tumour_histological_grade	1
radio_treated_breast	1
rs35410275	1

Top 30 most frequently Features across 10 folds: ['diabetes', 'history_of_heart_disease', 'depression', 'analgesics', 'post_

Keyways:

- Mean AUC using top 15 features across 10 folds: 0.8317
- Mean AUC using top 20 features across 10 folds: 0.8333
- Mean AUC using top 20 features across 10 folds: 0.8322
- Mean AUC using top 25 features across 10 folds: 0.8638
- Mean AUC using top 30 features across 10 folds: 0.8717

```
def get_frequent_features(feature_sets, freq_threshold=8):
    """
    Extract stable features from feature frequency tables.

    Works with BOTH formats:

    Format 1:
    {
        k: freq_df
    }

    Format 2:
    {
        k: (freq_df, auc)
    }

    Parameters
    -----
    feature_sets : dict
        Feature frequency results

    freq_threshold : int
        Minimum frequency required

    Returns
    -----
    list
        Stable feature names
    """

    stable_features = set()

    for k_value, value in feature_sets.items():

        # Handle both dictionary formats
        if isinstance(value, tuple):
            freq_df = value[0]
        else:
            freq_df = value

        selected = freq_df[
```

```

        freq_df['Frequency'] >= freq_threshold
    ]

    stable_features.update(
        selected.index.tolist()
    )

    return list(stable_features)

```

```

stable_features_univariate_scaled_inside_pipeline = get_frequency_based_feature_sets_univariate_scaled_inside_pipeline(
    feature_sets_univariate_scaled_inside_pipeline,
    freq_threshold=8
)

print(stable_features_univariate_scaled_inside_pipeline)
print(len(stable_features_univariate_scaled_inside_pipeline))

```

```

['radio_photon_2nd', 'rs2954007', 'breast_telangiectasia_outside_tumour_bed_baseline', 'breast_swollen_arm_baseline', 'rs72827

```

```

cross_validated_feature_selection(df_clean_20[stable_features_

```

```

Original Data shape is (2067, 28)
Data shape after dropping missing target: (1691, 28)
✅ Mean AUC with final selected features: 0.8971

```

▼ Nested CV

```

'\n      "ExtraTrees": ExtraTreesClassifier(\n          n_estimators=100, random_state=5\n      ),\n      "Gradient Boosting": GradientBoostingClassifier(\n          n_estimators=50, random_state=5\n      ),\n      "AdaBoost": AdaBoostClassifier(\n          n_estimators=50, random_state=5\n      ),\n      "XGBoost": XGBClassifier(\n          random_state=5,\n          use_label_encoder=False,\n          eval_metric="logloss"\n      ),\n      "DecisionTree": DecisionTreeClassifier(\n          max_depth=5,\n          random_state=5,\n          class_weight="balanced"\n      ),\n      "QDA": make_pipeline(\n          StandardScaler(),\n          QuadraticDiscriminantAnalysis()\n      ),\n      "GaussianNR": make_pipeline(\n          StandardScaler(),\n          GaussianNB()\n      ),\n      "SVC": make_pipeline(\n          StandardScaler(),\n          SVC()\n      )

```

```

from itertools import combinations

def jaccard_stability(fold_details, model_name):

    df_model = fold_details[fold_details["model"] == model_name]

    feature_sets = df_model["selected_features"].tolist()

    scores = []

    for a, b in combinations(feature_sets, 2):
        intersection = len(set(a) & set(b))
        union = len(set(a) | set(b))
        scores.append(intersection / union)

    return np.mean(scores)

```

```
def nested_cv_feature_selection_all_features(
    df,
    target,
    k_list,
    outer_splits,
    inner_splits
):

    df = df.dropna()

    # Same style as your second function
    X = df.drop(columns=[target, 'Subject Id'])
    y = df[target]

    # Convert categorical variables to numeric dummy variable
    X = pd.get_dummies(X, drop_first=True)

    outer_cv = StratifiedKFold(
        n_splits=outer_splits,
        shuffle=True,
        random_state=0
    )

    inner_cv = StratifiedKFold(
        n_splits=inner_splits,
        shuffle=True,
        random_state=1
    )

    results = {name: [] for name in get_models(X, y).keys()}
    fold_details = []

    for fold_id, (train_idx, test_idx) in enumerate(
        tqdm(outer_cv.split(X, y), desc="Outer CV"),
        start=1
    ):

        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        best_k = None
        best_score = -np.inf

        # =====
        # INNER LOOP: choose best k
        # =====
        for k in k_list:
```

```

inner_scores = []

for inner_train_idx, inner_val_idx in inner_cv.sp

    X_tr = X_train.iloc[inner_train_idx]
    X_val = X_train.iloc[inner_val_idx]
    y_tr = y_train.iloc[inner_train_idx]
    y_val = y_train.iloc[inner_val_idx]

    pipeline = Pipeline([
        ('scaler', MinMaxScaler()),
        ('feature_selection', SelectKBest(score_f
        ('model', RandomForestClassifier(
            n_estimators=100,
            max_depth=10,
            min_samples_split=5,
            min_samples_leaf=1,
            bootstrap=True,
            random_state=5
        ))
    ])

    pipeline.fit(X_tr, y_tr)

    y_pred = pipeline.predict_proba(X_val)[: , 1]
    inner_scores.append(roc_auc_score(y_val, y_pr

mean_score = np.mean(inner_scores)

if mean_score > best_score:
    best_score = mean_score
    best_k = k

# =====
# OUTER LOOP: train final models
# =====
models = get_models(X_train, y_train)

for name, model in models.items():

    pipeline = Pipeline([
        ('scaler', MinMaxScaler()),
        ('feature_selection', SelectKBest(score_func=
        ('model', model)
    ])

    pipeline.fit(X_train, y_train)

```

```
y_pred = pipeline.predict_proba(X_test)[: , 1]
auc_score = roc_auc_score(y_test, y_pred)

selected_mask = pipeline.named_steps[
    "feature_selection"
].get_support()

selected_features = X_train.columns[selected_mask]

results[name].append(auc_score)

fold_details.append({
    "fold": fold_id,
    "model": name,
    "best_k": best_k,
    "inner_auc": best_score,
    "outer_auc": auc_score,
    "selected_features": selected_features
})

summary = {
    model: {
        "mean_auc": np.mean(scores),
        "std_auc": np.std(scores)
    }
    for model, scores in results.items()
}

summary = pd.DataFrame(summary).T.sort_values(
    by="mean_auc",
    ascending=False
)

fold_details = pd.DataFrame(fold_details)

all_features = []

for features in fold_details["selected_features"]:
    all_features.extend(features)

feature_counts = Counter(all_features)

feature_frequency = pd.DataFrame({
    "feature": feature_counts.keys(),
    "count": feature_counts.values()
})
```

```
feature_frequency["percentage"] = (
    100 * feature_frequency["count"] / len(fold_details)
)

feature_frequency = feature_frequency.sort_values(
    by="percentage",
    ascending=False
).reset_index(drop=True)
```

```
summary1, fold_details1, feature_frequency1 = nested_cv_featu
```

Outer CV: 10it [01:09, 6.90s/it]

summary1

	mean_auc	std_auc	
RandomForest	0.854289	0.082492	

fold_details1

	fold	model	best_k	inner_auc	outer_auc	selected_features	
0	1	RandomForest	30	0.828021	0.897059	[diabetes, history_of_heart_disease, depressio...	
1	2	RandomForest	30	0.853369	0.944853	[diabetes, history_of_heart_disease, hypertens...	
2	3	RandomForest	30	0.844309	0.819853	[diabetes, history_of_heart_disease, hypertens...	
3	4	RandomForest	30	0.868881	0.812500	[diabetes, history_of_heart_disease, depressio...	
4	5	RandomForest	30	0.833113	0.933824	[diabetes, history_of_heart_disease, hypertens...	
5	6	RandomForest	20	0.873088	0.671569	[diabetes, history_of_heart_disease, analgesic...	
6	7	RandomForest	30	0.851114	0.960784	[diabetes, history_of_heart_disease, hypertens...	
7	8	RandomForest	20	0.853911	0.848039	[diabetes, history_of_heart_disease, hypertens...	
8	9	RandomForest	20	0.841122	0.862745	[diabetes, history_of_heart_disease, depressio...	
9	10	RandomForest	30	0.854568	0.791667	[diabetes, history_of_heart_disease, hypertens...	

Next steps: [Generate code with fold_details1](#) [New interactive sheet](#)

feature_frequency1

	feature	count	percentage	
0	diabetes	10	100.0	
1	history_of_heart_disease	10	100.0	
2	nodes_examined	10	100.0	
3	analgesics	10	100.0	
4	post_operative_haematoma	10	100.0	
5	rs72857489	10	100.0	
6	rs74853756	10	100.0	
7	rs56402833	10	100.0	
8	rs67342194	10	100.0	
9	rs7195405	10	100.0	
10	breast_swollen_arm_baseline	10	100.0	
11	radio_supraclavicular_fossa	10	100.0	
12	nodes_involved	9	90.0	
13	breast_arm_lymphodema_baseline	9	90.0	
14	rs2954007	8	80.0	
15	rs77405610	8	80.0	
16	rs461769	8	80.0	
17	depression	8	80.0	
18	radio_photon_2nd	8	80.0	
19	chemo_adjuvant	8	80.0	
20	axillary_levels	8	80.0	
21	chemo_neo_adjuvant	7	70.0	
22	rs10945395	7	70.0	
23	breast_telangiectasia_outside_tumour_bed_baseline	7	70.0	
24	radio_heart_mean_dose_Gy	6	60.0	
25	hypertension	6	60.0	
26	surgery_type	6	60.0	
27	radio_hot_spots_107	5	50.0	
28	delayed_healing	5	50.0	
29	ace_inhibitor	4	40.0	
30	post_operative_oedema	4	40.0	
31	er_status	3	30.0	
32	tumour_locality	2	20.0	
33	rs61912468	2	20.0	
34	rs57288348	2	20.0	
35	rs79223472	2	20.0	
36	breast_telangiectasia_tumour_bed_baseline	1	10.0	
37	rs1788750	1	10.0	
38	rs10764743	1	10.0	
39	rs72977387	1	10.0	
40	rs885170	1	10.0	
41	tumour_histological_grade	1	10.0	
42	radio_treated_breast	1	10.0	
43	rs35410275	1	10.0	

Next steps:

[Generate code with feature_frequency1](#)[New interactive sheet](#)

```
# Features that appeared in 8 folds and more across 10 folds
stable_features_nested1 = feature_frequency1[feature_frequenc
```

```
print (stable_features_nested1)
```

```
['diabetes', 'history_of_heart_disease', 'nodes_examined', 'analgesics', 'post_operative_haematoma', 'rs72857489', 'rs748537
```

```
print(len(stable_features_nested1))
```

```
21
```

```
jaccard_stability(fold_details1, "RandomForest")
```

```
np.float64(0.647321191223846)
```

```
cross_validated_feature_selection(df_clean_20[stable_features.
```

```
Original Data shape is (2067, 22)
```

```
Data shape after dropping missing target: (1691, 22)
```

```
✅ Mean AUC with final selected features: 0.8812
```

Second feature selectin Method

3.3.3) Feature Importance — You can gain the significance of each feature of your dataset by using the Model Characteristics property. Feature value gives you a score for every function of your results, the higher the score the more significant or appropriate the performance variable is. Feature importance is the built-in class that comes with Tree Based Classifiers, we will use the Extra Tree Classifier to extract the top 10 features for the dataset.

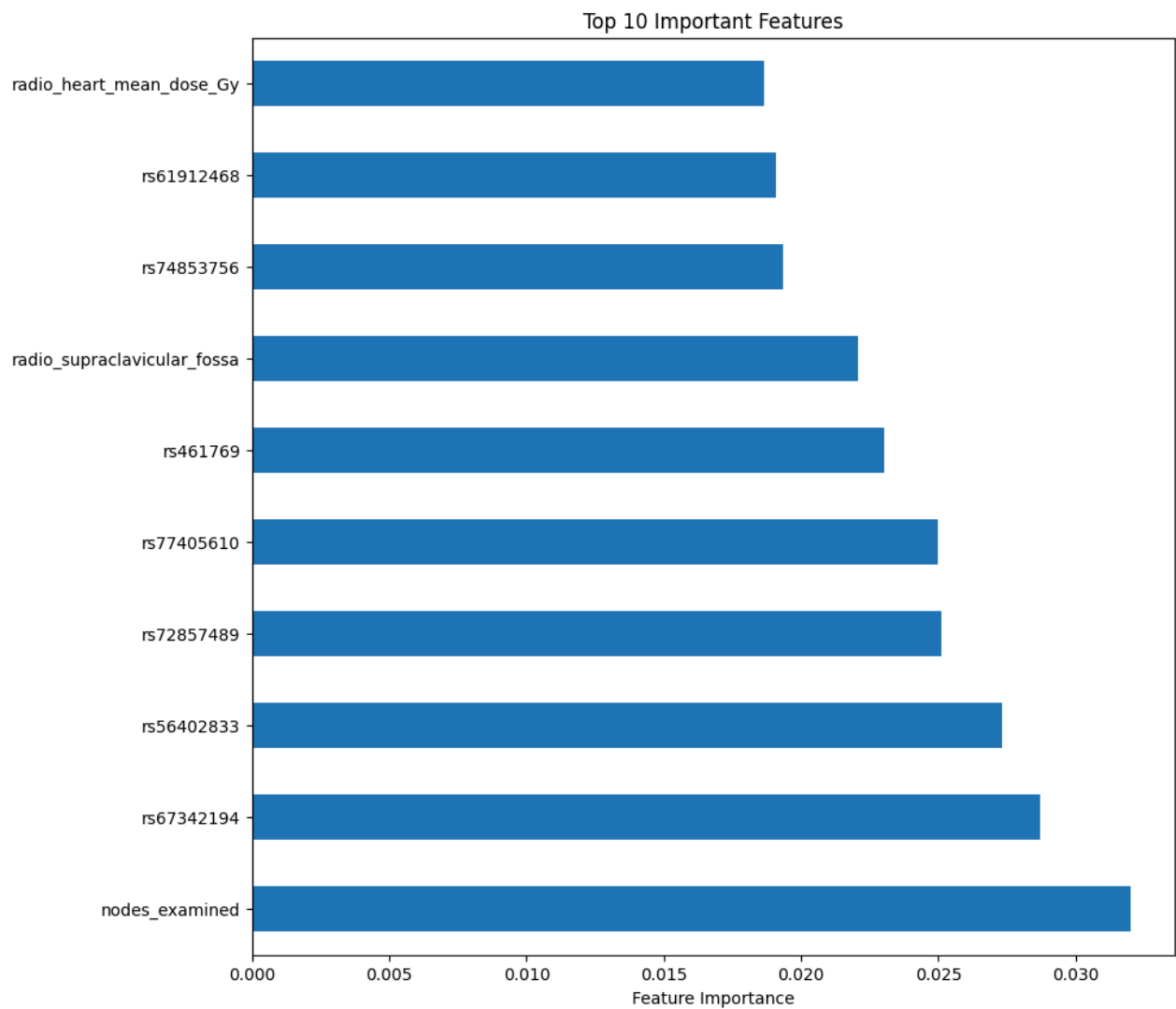
outside CV

```
feature_sets_featimportance = {}
```

```
for i in k:
```

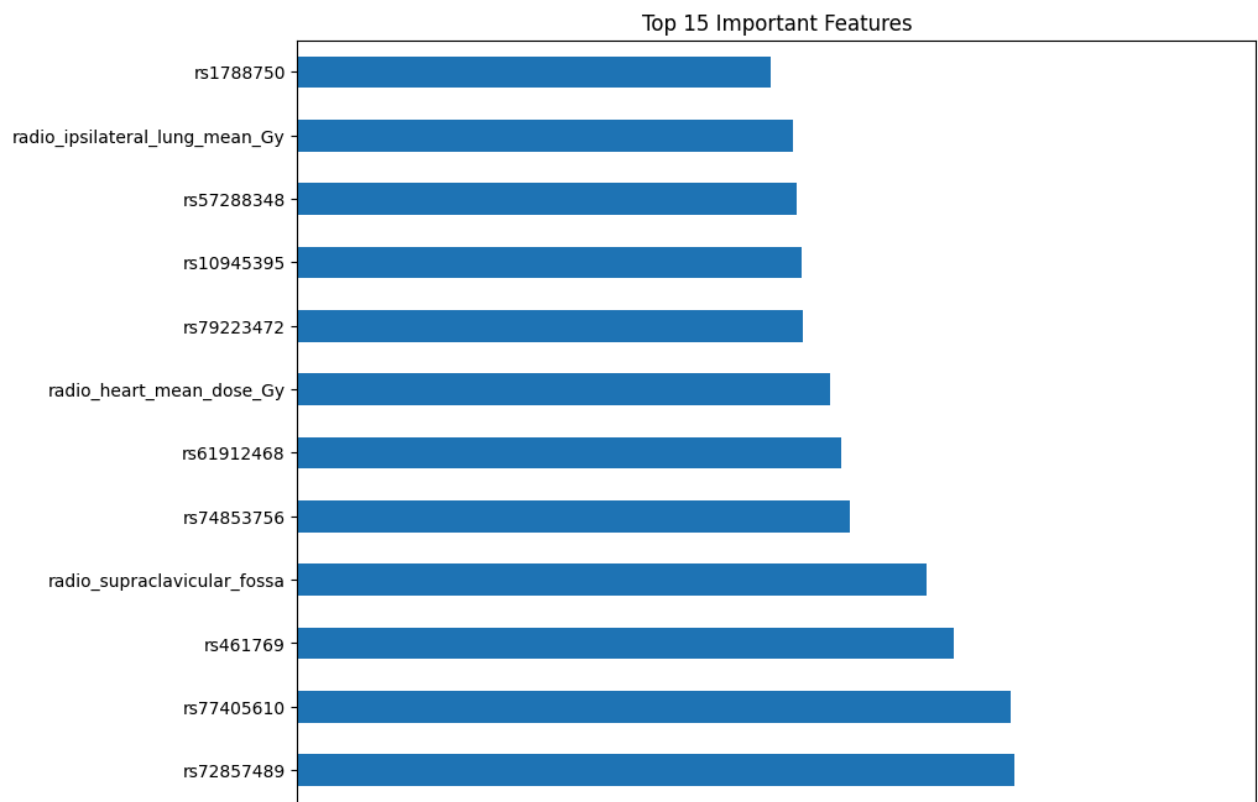
```
    freq_df2_auc= featureselection2(df_clean_20, dependent1, l
    feature_sets_featimportance[i] = freq_df2_auc
```

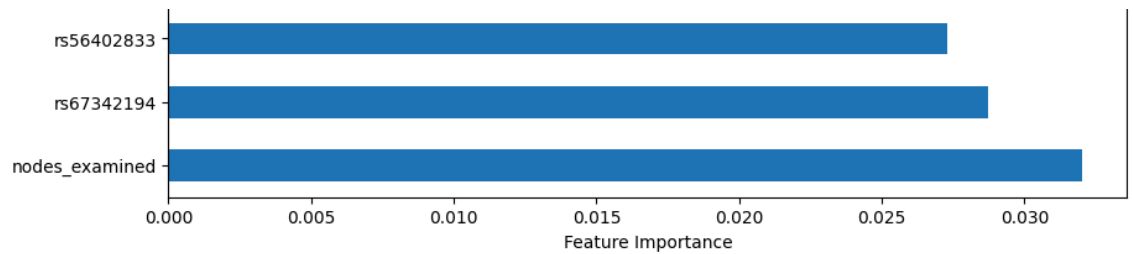

Choosing 10 Features:



Important 10 features: ['nodes_examined', 'rs67342194', 'rs56402833', 'rs72857489', 'rs77405610', 'rs461769', 'radio_suprac
Mean AUC score using top 10 features: 0.8794

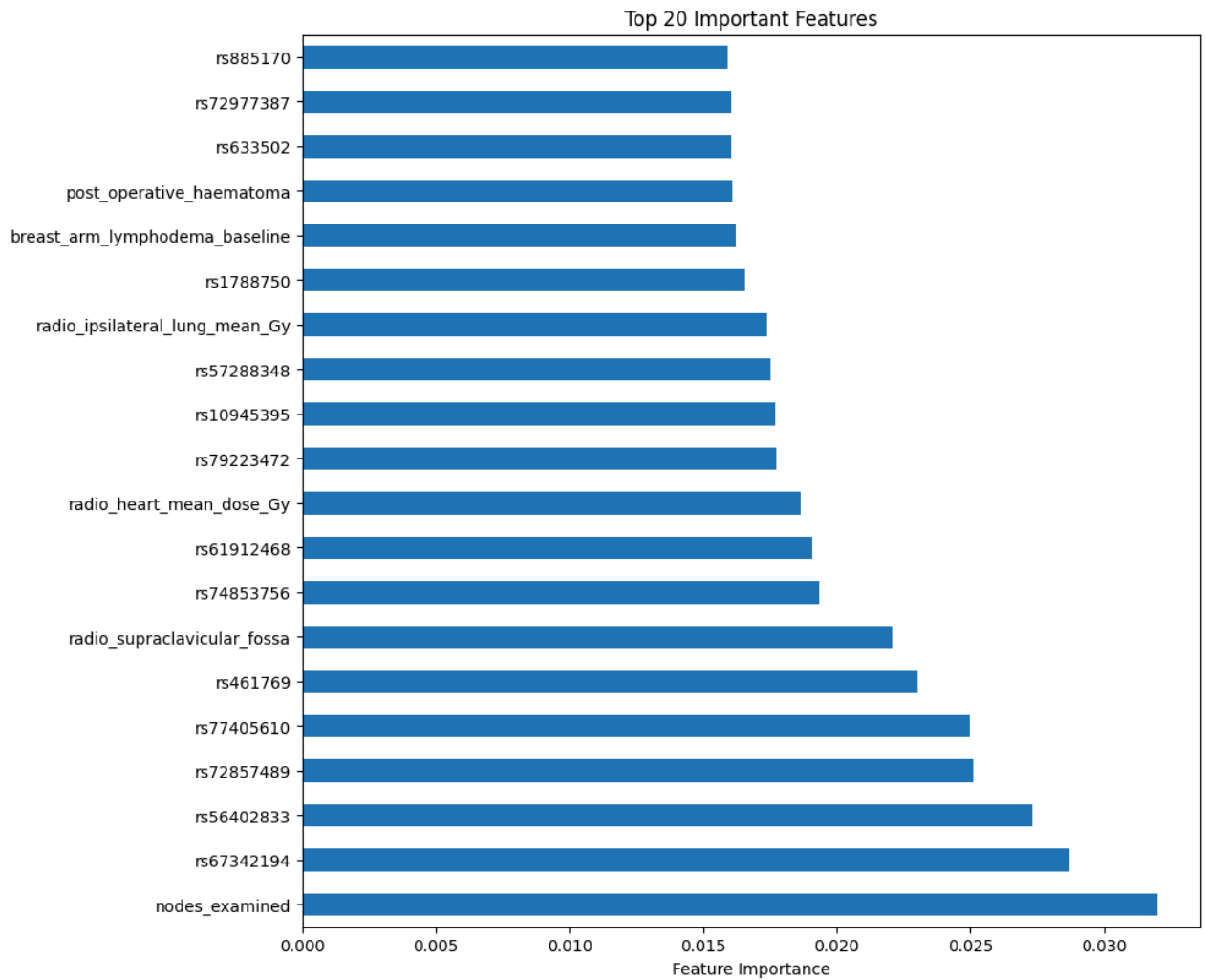
Choosing 15 Features:





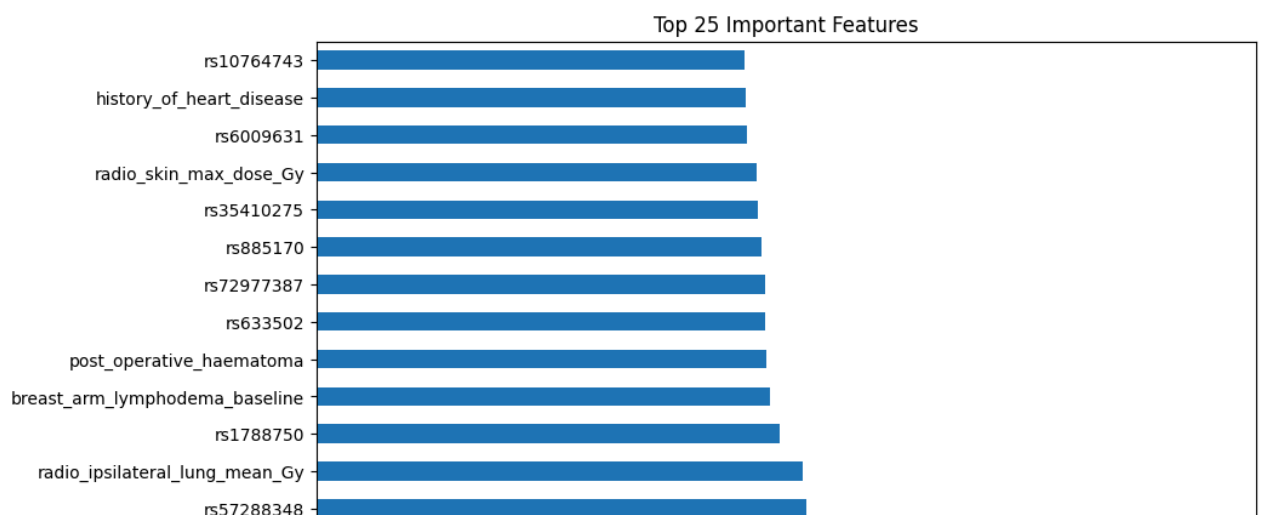
Important 15 features: ['nodes_examined', 'rs67342194', 'rs56402833', 'rs72857489', 'rs77405610', 'rs461769', 'radio_suprac
Mean AUC score using top 15 features: 0.9031

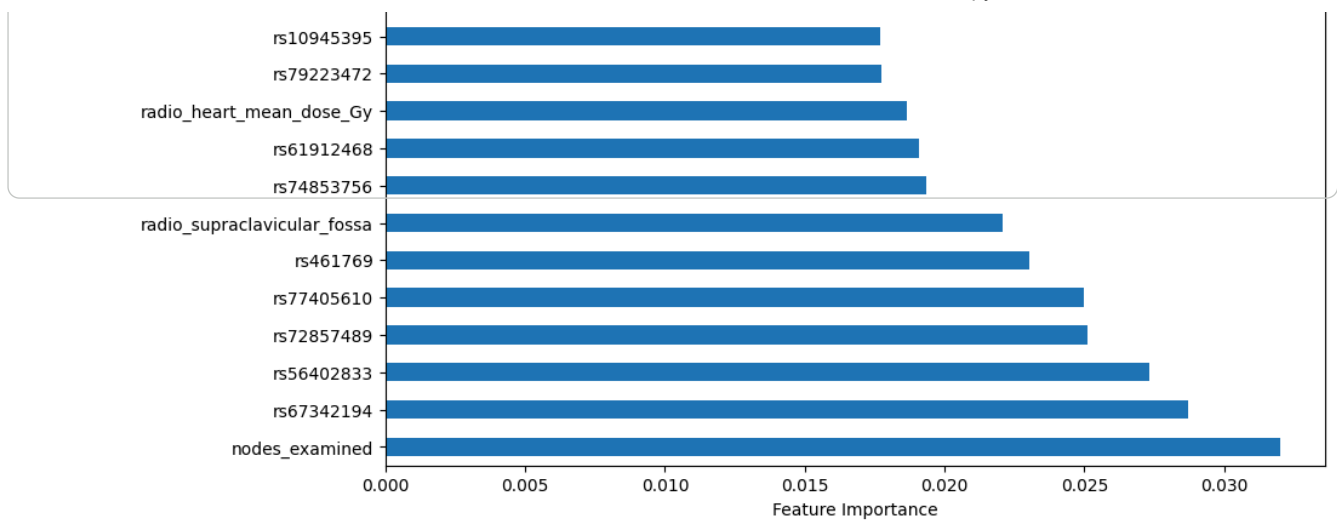
Choosing 20 Features:



Important 20 features: ['nodes_examined', 'rs67342194', 'rs56402833', 'rs72857489', 'rs77405610', 'rs461769', 'radio_suprac
Mean AUC score using top 20 features: 0.8993

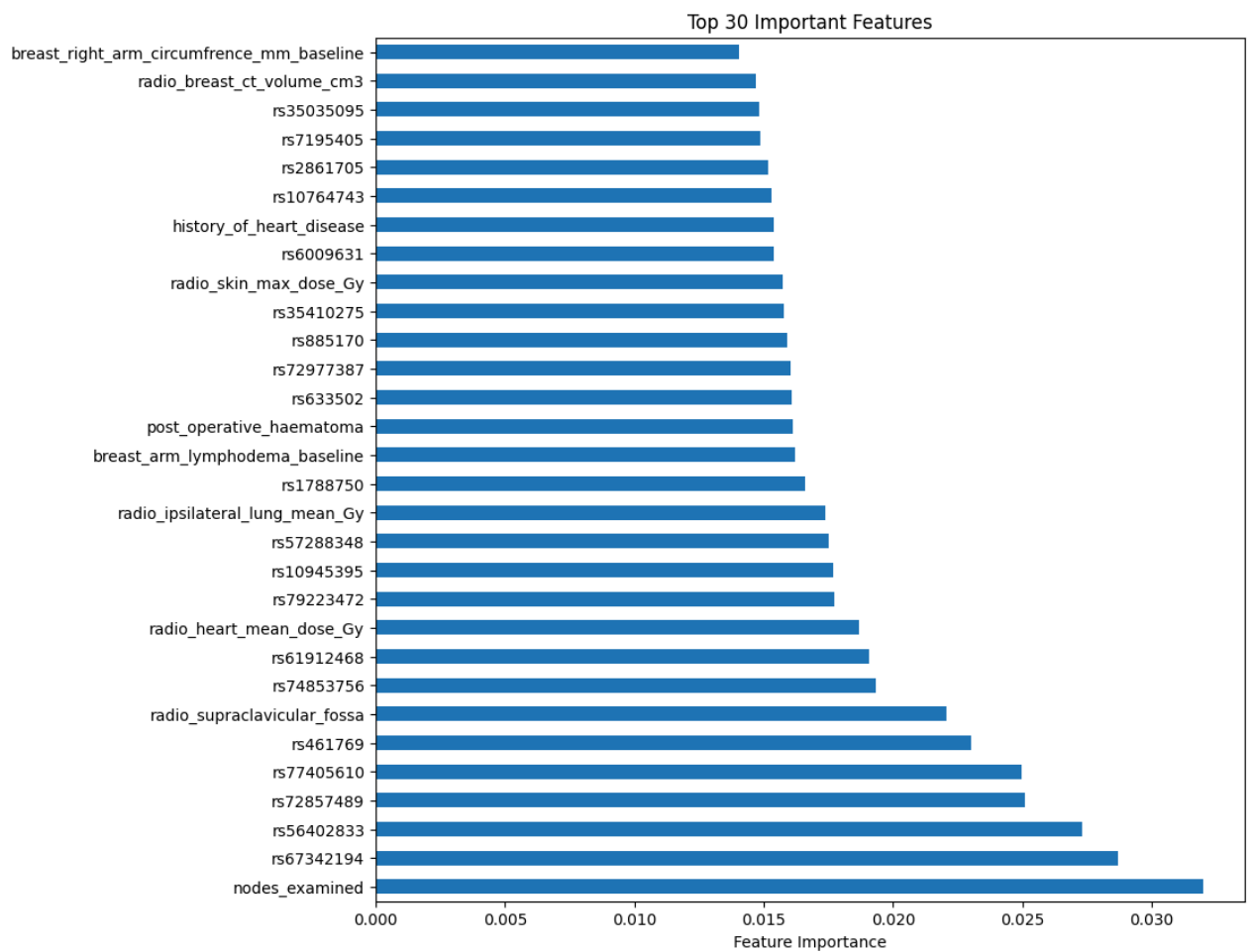
Choosing 25 Features:





Important 25 features: ['nodes_examined', 'rs67342194', 'rs56402833', 'rs72857489', 'rs77405610', 'rs461769', 'radio_suprac
Mean AUC score using top 25 features: 0.8989

Choosing 30 Features:



Important 30 features: ['nodes_examined', 'rs67342194', 'rs56402833', 'rs72857489', 'rs77405610', 'rs461769', 'radio_suprac
Mean AUC score using top 30 features: 0.8958

Takeaways4: These based on df_clean_20

- Mean AUC score using top 10 features: 0.8794
- Mean AUC score using top 15 features: 0.9031
- Mean AUC score using top 20 features: 0.8993
- Mean AUC score using top 25 features: 0.8989
- Mean AUC score using top 30 features: 0.8958

```
print(feature_sets_featimportance)
```

```
{10: (['nodes_examined', 'rs67342194', 'rs56402833', 'rs72857489', 'rs77405610', 'rs461769', 'radio_supraclavicular_fossa',
```

```
from collections import Counter
import math
```

```
feature_counter = Counter()
```

```
for features, _ in feature_sets_featimportance.values():
    feature_counter.update(features)
```

```
# 60% threshold
```

```
threshold = math.ceil(0.8 * len(feature_sets_featimportance))
```

```
stable_features_featimportance = [
    feature
    for feature, count in feature_counter.items()
    if count >= threshold
]
```

```
print(stable_features_featimportance)
print(len(stable_features_featimportance))
```

```
['nodes_examined', 'rs67342194', 'rs56402833', 'rs72857489', 'rs77405610', 'rs461769', 'radio_supraclavicular_fossa', 'rs74815
```

```
cross_validated_feature_selection(df_clean_20[stable_features.
```

```
Original Data shape is (2067, 16)
```

```
Data shape after dropping missing target: (1691, 16)
```

```
✅ Mean AUC with final selected features: 0.8831
```

✖ inside CV

```
# This is the method that I used to compare among features nui
for i in k:
```

```
    Mean_auc_scores_CV_inside_pipeline2= featureselection2_pipe.
```

```
Mean AUC score using top 10 features (Inside CV): 0.795
```

```
Mean AUC score using top 15 features (Inside CV): 0.860
```

```
Mean AUC score using top 20 features (Inside CV): 0.847
```

```
Mean AUC score using top 25 features (Inside CV): 0.863
```

```
Mean AUC score using top 30 features (Inside CV): 0.851
```

Keywords: with scale:

- Mean AUC score using top 10 features (Inside CV): 0.80
- Mean AUC score using top 15 features (Inside CV): 0.86
- Mean AUC score using top 20 features (Inside CV): 0.85
- Mean AUC score using top 25 features (Inside CV): 0.86
- Mean AUC score using top 30 features (Inside CV): 0.85

without scaled:

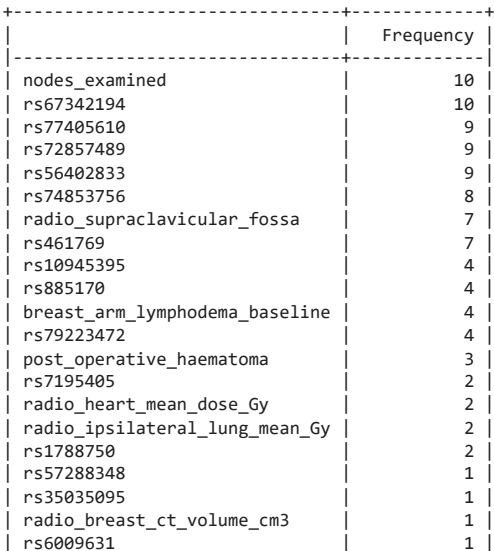
- Mean AUC score using top 10 features (Inside CV): 0.80
- Mean AUC score using top 15 features (Inside CV): 0.86
- Mean AUC score using top 20 features (Inside CV): 0.85
- Mean AUC score using top 25 features (Inside CV): 0.86
- Mean AUC score using top 30 features (Inside CV): 0.85

▼ features frequency inside pipeline

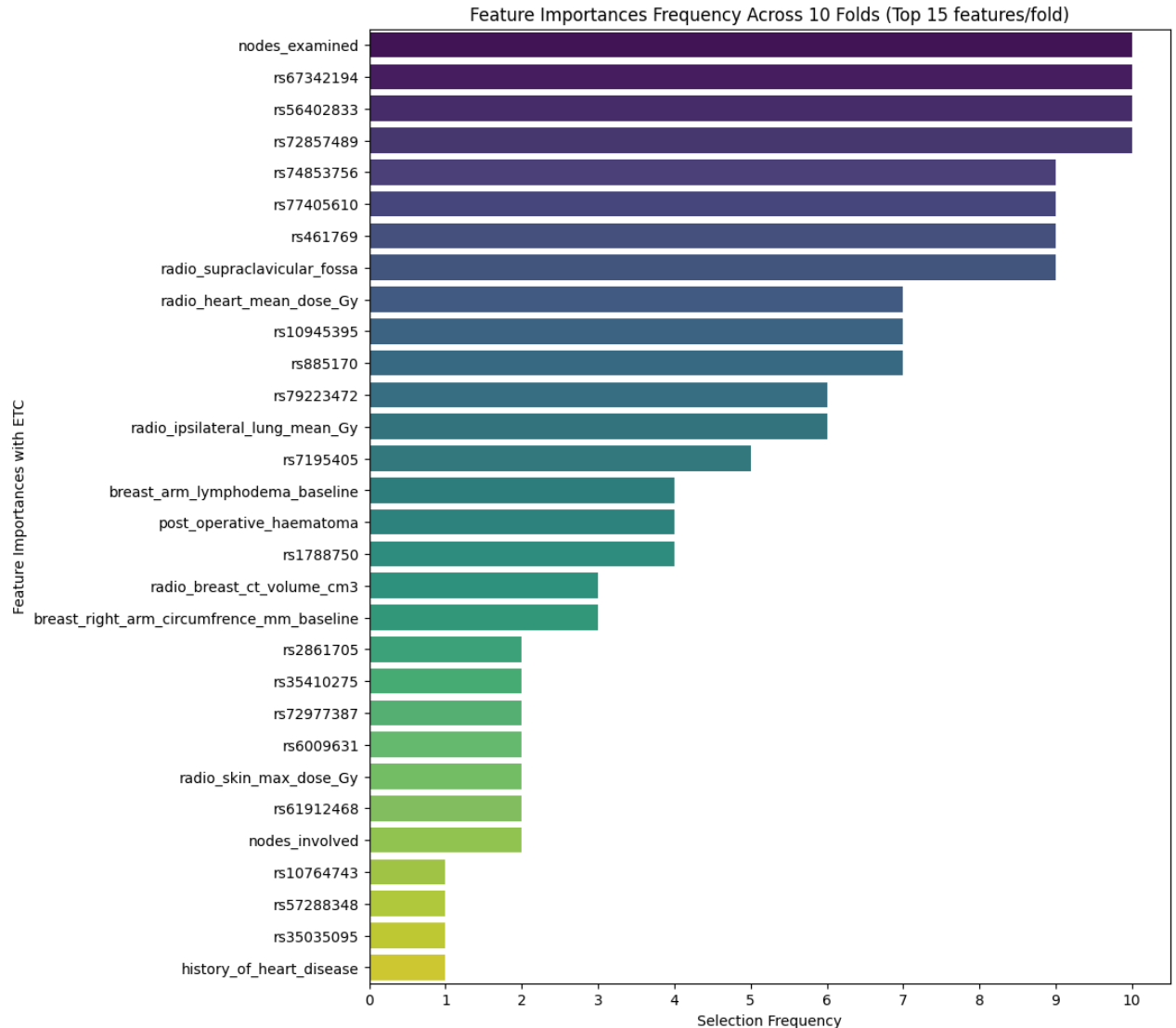
```
feature_sets_featimportance_inside_pipeline = {}

for i in k:
    freq_df2= featureselection2_freq(df_clean_20, dependent1,
    feature_sets_featimportance_inside_pipeline[i] = freq_df2
```


Feature Importances Frequency Across 10 Folds (Top 10 features/fold)



Fold 1: Selected features: ['nodes_examined', 'radio_ipsilateral_lung_mean_Gy', 'radio_supraclavicular_fossa', 'rs77405610',
Fold 2: Selected features: ['nodes_examined', 'post_operative_haematoma', 'radio_breast_ct_volume_cm3', 'radio_heart_mean_dose_Gy',
Fold 3: Selected features: ['nodes_examined', 'radio_supraclavicular_fossa', 'rs77405610', 'rs67342194', 'rs1788750', 'rs72977387',
Fold 4: Selected features: ['nodes_involved', 'nodes_examined', 'radio_heart_mean_dose_Gy', 'radio_ipsilateral_lung_mean_Gy',
Fold 5: Selected features: ['history_of_heart_disease', 'nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_ipsilateral_lung_mean_Gy',
Fold 6: Selected features: ['nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_heart_mean_dose_Gy', 'radio_breast_ct_volume_cm3',
Fold 7: Selected features: ['nodes_examined', 'post_operative_haematoma', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy',
Fold 8: Selected features: ['nodes_examined', 'post_operative_haematoma', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ipsilateral_lung_mean_Gy',
Fold 9: Selected features: ['nodes_examined', 'radio_heart_mean_dose_Gy', 'radio_supraclavicular_fossa', 'breast_right_arm_circumference_mm_baseline',
Fold 10: Selected features: ['nodes_examined', 'radio_heart_mean_dose_Gy', 'radio_supraclavicular_fossa', 'breast_arm_lymphodema_baseline',
Mean AUC using top 15 features across 10 folds: 0.8601

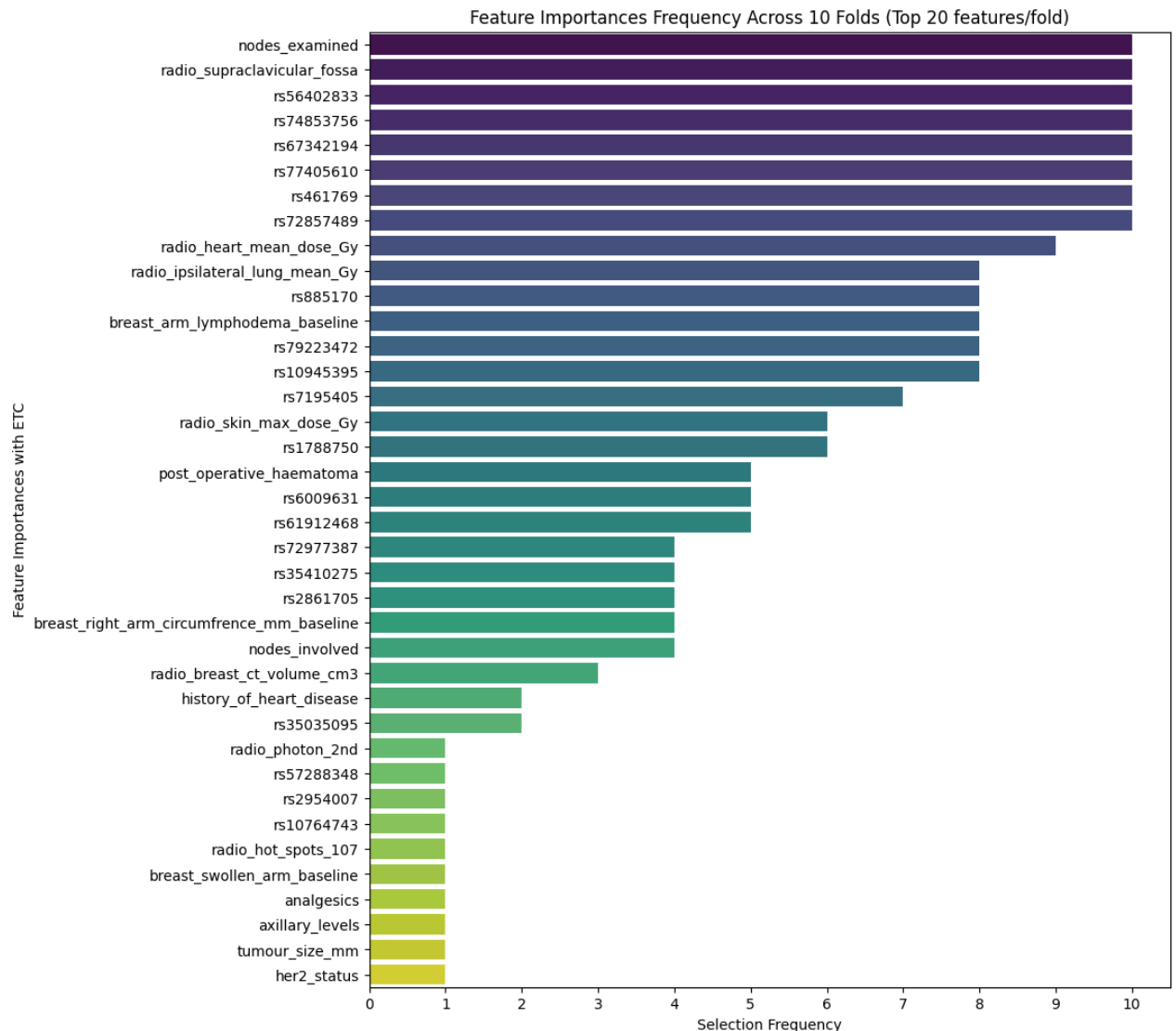


	Frequency
nodes_examined	10
rs67342194	10
rs56402833	10
rs72857489	10
rs74853756	9
rs77405610	9
rs461769	9
radio_supraclavicular_fossa	9
radio_heart_mean_dose_Gy	7
rs10945395	7
rs885170	7
rs79223472	6
radio_ipsilateral_lung_mean_Gy	6
rs7195405	5
breast_arm_lymphodema_baseline	4
post_operative_haematoma	4
rs1788750	4
radio_breast_ct_volume_cm3	3
breast_right_arm_circumference_mm_baseline	3
rs2861705	2
rs35410275	2
rs72977387	2
rs6009631	2
radio_skin_max_dose_Gy	2
rs61912468	2
nodes_involved	2
rs10764743	1
rs57288348	1
rs35035095	1
history_of_heart_disease	1

nodes_involved	2
rs10764743	1
rs57288348	1
rs35035095	1
history_of_heart_disease	1

Top 15 most frequently Features across 10 folds: ['nodes_examined', 'rs67342194', 'rs56402833', 'rs72857489', 'rs74853756',

Fold 1: Selected features: ['nodes_examined', 'radio_heart_mean_dose_Gy', 'radio_ipsilateral_lung_mean_Gy', 'radio_supraclav
Fold 2: Selected features: ['nodes_examined', 'post_operative_haematoma', 'radio_breast_ct_volume_cm3', 'radio_heart_mean_dose_Gy', 'radio_ipsilateral_lung_mean_Gy', 'radio_skin_max_dose_Gy', 'radio_supraclavicular_fossa', 'breast_right_arm_circumference_mm_baseline', 'nodes_involved', 'history_of_heart_disease', 'rs35035095', 'rs10764743', 'rs57288348', 'rs2954007', 'radio_hot_spots_107', 'breast_swollen_arm_baseline', 'analgesics', 'axillary_levels', 'tumour_size_mm', 'her2_status']
Fold 3: Selected features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_supraclavicular_fossa', 'breast_right_arm_circumference_mm_baseline', 'nodes_involved', 'history_of_heart_disease', 'rs35035095', 'rs10764743', 'rs57288348', 'rs2954007', 'radio_hot_spots_107', 'breast_swollen_arm_baseline', 'analgesics', 'axillary_levels', 'tumour_size_mm', 'her2_status']
Fold 4: Selected features: ['history_of_heart_disease', 'nodes_involved', 'nodes_examined', 'radio_heart_mean_dose_Gy', 'radio_ipsilateral_lung_mean_Gy', 'radio_skin_max_dose_Gy', 'radio_supraclavicular_fossa', 'breast_right_arm_circumference_mm_baseline', 'nodes_involved', 'history_of_heart_disease', 'rs35035095', 'rs10764743', 'rs57288348', 'rs2954007', 'radio_hot_spots_107', 'breast_swollen_arm_baseline', 'analgesics', 'axillary_levels', 'tumour_size_mm', 'her2_status']
Fold 5: Selected features: ['history_of_heart_disease', 'nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_supraclavicular_fossa', 'breast_right_arm_circumference_mm_baseline', 'nodes_involved', 'history_of_heart_disease', 'rs35035095', 'rs10764743', 'rs57288348', 'rs2954007', 'radio_hot_spots_107', 'breast_swollen_arm_baseline', 'analgesics', 'axillary_levels', 'tumour_size_mm', 'her2_status']
Fold 6: Selected features: ['nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_skin_max_dose_Gy', 'radio_supraclavicular_fossa', 'breast_right_arm_circumference_mm_baseline', 'nodes_involved', 'history_of_heart_disease', 'rs35035095', 'rs10764743', 'rs57288348', 'rs2954007', 'radio_hot_spots_107', 'breast_swollen_arm_baseline', 'analgesics', 'axillary_levels', 'tumour_size_mm', 'her2_status']
Fold 7: Selected features: ['nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_supraclavicular_fossa', 'breast_right_arm_circumference_mm_baseline', 'nodes_involved', 'history_of_heart_disease', 'rs35035095', 'rs10764743', 'rs57288348', 'rs2954007', 'radio_hot_spots_107', 'breast_swollen_arm_baseline', 'analgesics', 'axillary_levels', 'tumour_size_mm', 'her2_status']
Fold 8: Selected features: ['analgesics', 'nodes_examined', 'post_operative_haematoma', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ipsilateral_lung_mean_Gy', 'radio_breast_ct_volume_cm3', 'radio_supraclavicular_fossa', 'breast_right_arm_circumference_mm_baseline', 'nodes_involved', 'history_of_heart_disease', 'rs35035095', 'rs10764743', 'rs57288348', 'rs2954007', 'radio_hot_spots_107', 'breast_swollen_arm_baseline', 'analgesics', 'axillary_levels', 'tumour_size_mm', 'her2_status']
Fold 9: Selected features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'axillary_levels', 'radio_breast_ct_volume_cm3', 'radio_supraclavicular_fossa', 'breast_right_arm_circumference_mm_baseline', 'nodes_involved', 'history_of_heart_disease', 'rs35035095', 'rs10764743', 'rs57288348', 'rs2954007', 'radio_hot_spots_107', 'breast_swollen_arm_baseline', 'analgesics', 'tumour_size_mm', 'her2_status']
Fold 10: Selected features: ['nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'tumour_size_mm', 'her2_status']
Mean AUC using top 20 features across 10 folds: 0.8469

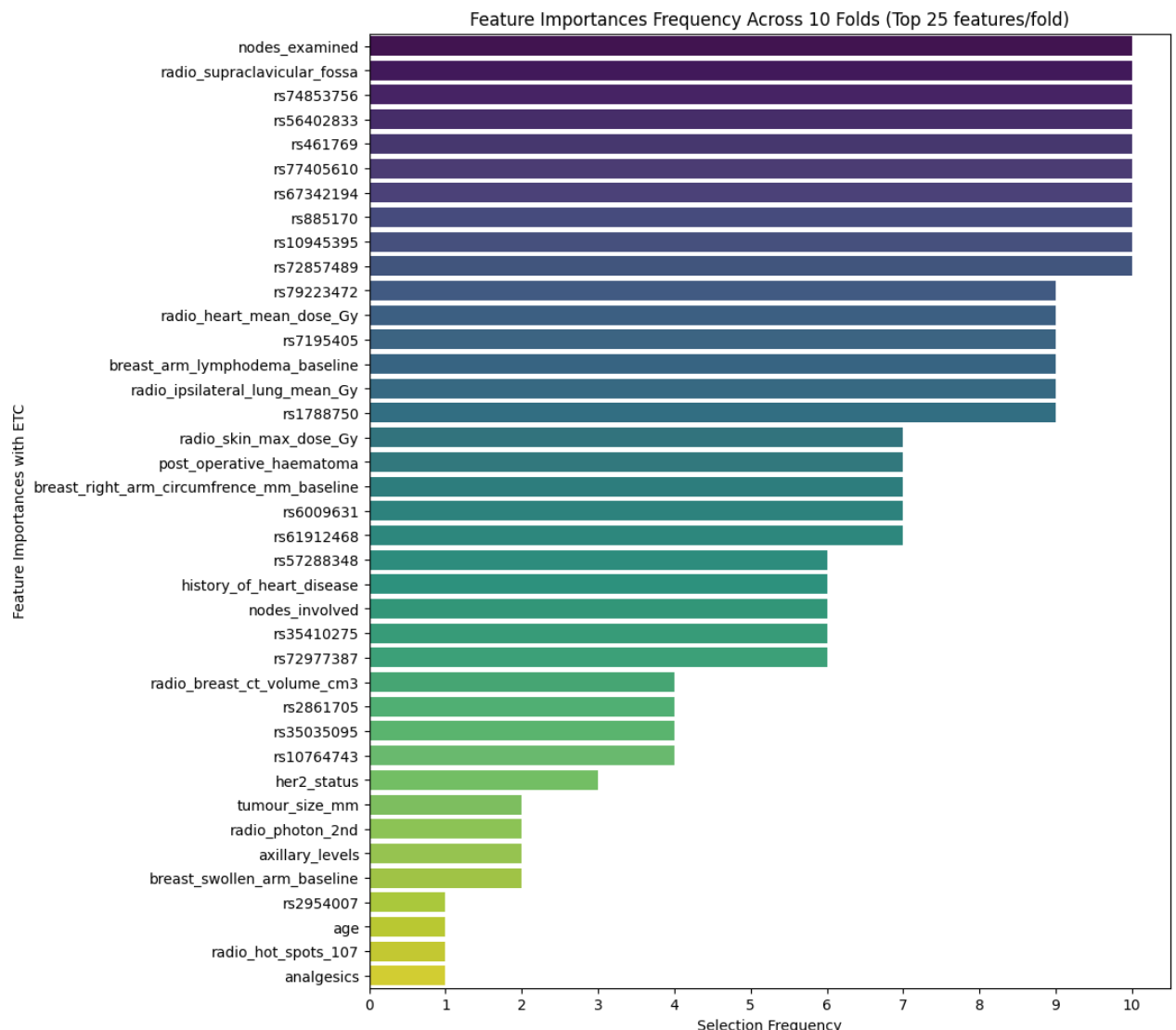


	Frequency
nodes_examined	10
radio_supraclavicular_fossa	10
rs56402833	10
rs74853756	10
rs67342194	10
rs77405610	10
rs461769	10
rs72857489	10
radio_heart_mean_dose_Gy	9
radio_ipsilateral_lung_mean_Gy	8
rs885170	8
breast_arm_lymphodema_baseline	8
rs79223472	8
rs10945395	8
rs7195405	7
radio_skin_max_dose_Gy	6
rs1788750	6

post_operative_naematoma	5
rs6009631	5
rs61912468	5
rs72977387	4
rs35410275	4
rs2861705	4
breast_right_arm_circumfrence_mm_baseline	4
nodes_involved	4
radio_breast_ct_volume_cm3	3
history_of_heart_disease	2
rs35035095	2
radio_photon_2nd	1
rs57288348	1
rs2954007	1
rs10764743	1
radio_hot_spots_107	1
breast_swollen_arm_baseline	1
analgesics	1
axillary_levels	1
tumour_size_mm	1
her2_status	1

Top 20 most frequently Features across 10 folds: ['nodes_examined', 'radio_supraclavicular_fossa', 'rs56402833', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756', 'rs74853756']

Fold 1: Selected features: ['history_of_heart_disease', 'nodes_examined', 'post_operative_haematoma', 'radio_breast_ct_volum
Fold 2: Selected features: ['age', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_breast_ct_volume_c
Fold 3: Selected features: ['history_of_heart_disease', 'nodes_examined', 'post_operative_haematoma', 'her2_status', 'radio_
Fold 4: Selected features: ['history_of_heart_disease', 'nodes_involved', 'nodes_examined', 'radio_heart_mean_dose_Gy', 'rac
Fold 5: Selected features: ['history_of_heart_disease', 'nodes_examined', 'her2_status', 'radio_breast_ct_volume_cm3', 'radi
Fold 6: Selected features: ['nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'tumour_size_mm', 'radio_skin_ma
Fold 7: Selected features: ['history_of_heart_disease', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'rac
Fold 8: Selected features: ['analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_skin_max_dc
Fold 9: Selected features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'axillary_levels', 'radi
Fold 10: Selected features: ['history_of_heart_disease', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'tu
Mean AUC using top 25 features across 10 folds: 0.8625



	Frequency
nodes_examined	10

radio_supraclavicular_fossa	10
rs74853756	10
rs56402833	10
rs461769	10
rs77405610	10
rs67342194	10
rs885170	10
rs10945395	10
rs72857489	10
rs79223472	9
radio_heart_mean_dose_Gy	9
rs7195405	9
breast_arm_lymphodema_baseline	9
radio_ipsilateral_lung_mean_Gy	9
rs1788750	9
radio_skin_max_dose_Gy	7
post_operative_haematoma	7
breast_right_arm_circumfrence_mm_baseline	7
rs6009631	7
rs61912468	7
rs57288348	6
history_of_heart_disease	6
nodes_involved	6
rs35410275	6
rs72977387	6
radio_breast_ct_volume_cm3	4
rs2861705	4
rs35035095	4
rs10764743	4
her2_status	3
tumour_size_mm	2
radio_photon_2nd	2
axillary_levels	2
breast_swollen_arm_baseline	2
rs2954007	1
age	1
radio_hot_spots_107	1
analgesics	1

Top 25 most frequently Features across 10 folds: ['nodes_examined', 'radio_supraclavicular_fossa', 'rs74853756', 'rs56402833', 'rs461769', 'rs77405610', 'rs67342194', 'rs885170', 'rs10945395', 'rs72857489', 'rs79223472', 'radio_heart_mean_dose_Gy', 'rs7195405', 'breast_arm_lymphodema_baseline', 'radio_ipsilateral_lung_mean_Gy', 'rs1788750', 'radio_skin_max_dose_Gy', 'post_operative_haematoma', 'breast_right_arm_circumfrence_mm_baseline', 'rs6009631', 'rs61912468', 'rs57288348', 'history_of_heart_disease', 'nodes_involved', 'rs35410275', 'rs72977387', 'radio_breast_ct_volume_cm3', 'rs2861705', 'rs35035095', 'rs10764743', 'her2_status', 'tumour_size_mm', 'radio_photon_2nd', 'axillary_levels', 'breast_swollen_arm_baseline', 'rs2954007', 'age', 'radio_hot_spots_107', 'analgesics']

Fold 1: Selected features: ['history_of_heart_disease', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs74853756', 'rs56402833', 'rs461769', 'rs77405610', 'rs67342194', 'rs885170', 'rs10945395', 'rs72857489', 'rs79223472', 'radio_heart_mean_dose_Gy', 'rs7195405', 'breast_arm_lymphodema_baseline', 'radio_ipsilateral_lung_mean_Gy', 'rs1788750', 'radio_skin_max_dose_Gy', 'post_operative_haematoma', 'breast_right_arm_circumfrence_mm_baseline', 'rs6009631', 'rs61912468', 'rs57288348', 'history_of_heart_disease', 'nodes_involved', 'rs35410275', 'rs72977387', 'radio_breast_ct_volume_cm3', 'rs2861705', 'rs35035095', 'rs10764743', 'her2_status', 'tumour_size_mm', 'radio_photon_2nd', 'axillary_levels', 'breast_swollen_arm_baseline', 'rs2954007', 'age', 'radio_hot_spots_107', 'analgesics']

Fold 2: Selected features: ['age', 'band_size', 'history_of_heart_disease', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs74853756', 'rs56402833', 'rs461769', 'rs77405610', 'rs67342194', 'rs885170', 'rs10945395', 'rs72857489', 'rs79223472', 'radio_heart_mean_dose_Gy', 'rs7195405', 'breast_arm_lymphodema_baseline', 'radio_ipsilateral_lung_mean_Gy', 'rs1788750', 'radio_skin_max_dose_Gy', 'post_operative_haematoma', 'breast_right_arm_circumfrence_mm_baseline', 'rs6009631', 'rs61912468', 'rs57288348', 'history_of_heart_disease', 'nodes_involved', 'rs35410275', 'rs72977387', 'radio_breast_ct_volume_cm3', 'rs2861705', 'rs35035095', 'rs10764743', 'her2_status', 'tumour_size_mm', 'radio_photon_2nd', 'axillary_levels', 'breast_swollen_arm_baseline', 'rs2954007', 'age', 'radio_hot_spots_107', 'analgesics']

Fold 3: Selected features: ['history_of_heart_disease', 'nodes_examined', 'post_operative_haematoma', 'her2_status', 'radio_breast_ct_volume_cm3', 'radio_ipsilateral_lung_mean_Gy', 'rs1788750', 'radio_skin_max_dose_Gy', 'post_operative_haematoma', 'breast_right_arm_circumfrence_mm_baseline', 'rs6009631', 'rs61912468', 'rs57288348', 'history_of_heart_disease', 'nodes_involved', 'rs35410275', 'rs72977387', 'radio_breast_ct_volume_cm3', 'rs2861705', 'rs35035095', 'rs10764743', 'her2_status', 'tumour_size_mm', 'radio_photon_2nd', 'axillary_levels', 'breast_swollen_arm_baseline', 'rs2954007', 'age', 'radio_hot_spots_107', 'analgesics']

Fold 4: Selected features: ['history_of_heart_disease', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs74853756', 'rs56402833', 'rs461769', 'rs77405610', 'rs67342194', 'rs885170', 'rs10945395', 'rs72857489', 'rs79223472', 'radio_heart_mean_dose_Gy', 'rs7195405', 'breast_arm_lymphodema_baseline', 'radio_ipsilateral_lung_mean_Gy', 'rs1788750', 'radio_skin_max_dose_Gy', 'post_operative_haematoma', 'breast_right_arm_circumfrence_mm_baseline', 'rs6009631', 'rs61912468', 'rs57288348', 'history_of_heart_disease', 'nodes_involved', 'rs35410275', 'rs72977387', 'radio_breast_ct_volume_cm3', 'rs2861705', 'rs35035095', 'rs10764743', 'her2_status', 'tumour_size_mm', 'radio_photon_2nd', 'axillary_levels', 'breast_swollen_arm_baseline', 'rs2954007', 'age', 'radio_hot_spots_107', 'analgesics']

Fold 5: Selected features: ['history_of_heart_disease', 'nodes_examined', 'her2_status', 'radio_breast_ct_volume_cm3', 'radio_ipsilateral_lung_mean_Gy', 'rs1788750', 'radio_skin_max_dose_Gy', 'post_operative_haematoma', 'breast_right_arm_circumfrence_mm_baseline', 'rs6009631', 'rs61912468', 'rs57288348', 'history_of_heart_disease', 'nodes_involved', 'rs35410275', 'rs72977387', 'radio_breast_ct_volume_cm3', 'rs2861705', 'rs35035095', 'rs10764743', 'her2_status', 'tumour_size_mm', 'radio_photon_2nd', 'axillary_levels', 'breast_swollen_arm_baseline', 'rs2954007', 'age', 'radio_hot_spots_107', 'analgesics']

Fold 6: Selected features: ['nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'tumour_size_mm', 'radio_breast_ct_volume_cm3', 'radio_ipsilateral_lung_mean_Gy', 'rs1788750', 'radio_skin_max_dose_Gy', 'post_operative_haematoma', 'breast_right_arm_circumfrence_mm_baseline', 'rs6009631', 'rs61912468', 'rs57288348', 'history_of_heart_disease', 'nodes_involved', 'rs35410275', 'rs72977387', 'radio_breast_ct_volume_cm3', 'rs2861705', 'rs35035095', 'rs10764743', 'her2_status', 'tumour_size_mm', 'radio_photon_2nd', 'axillary_levels', 'breast_swollen_arm_baseline', 'rs2954007', 'age', 'radio_hot_spots_107', 'analgesics']

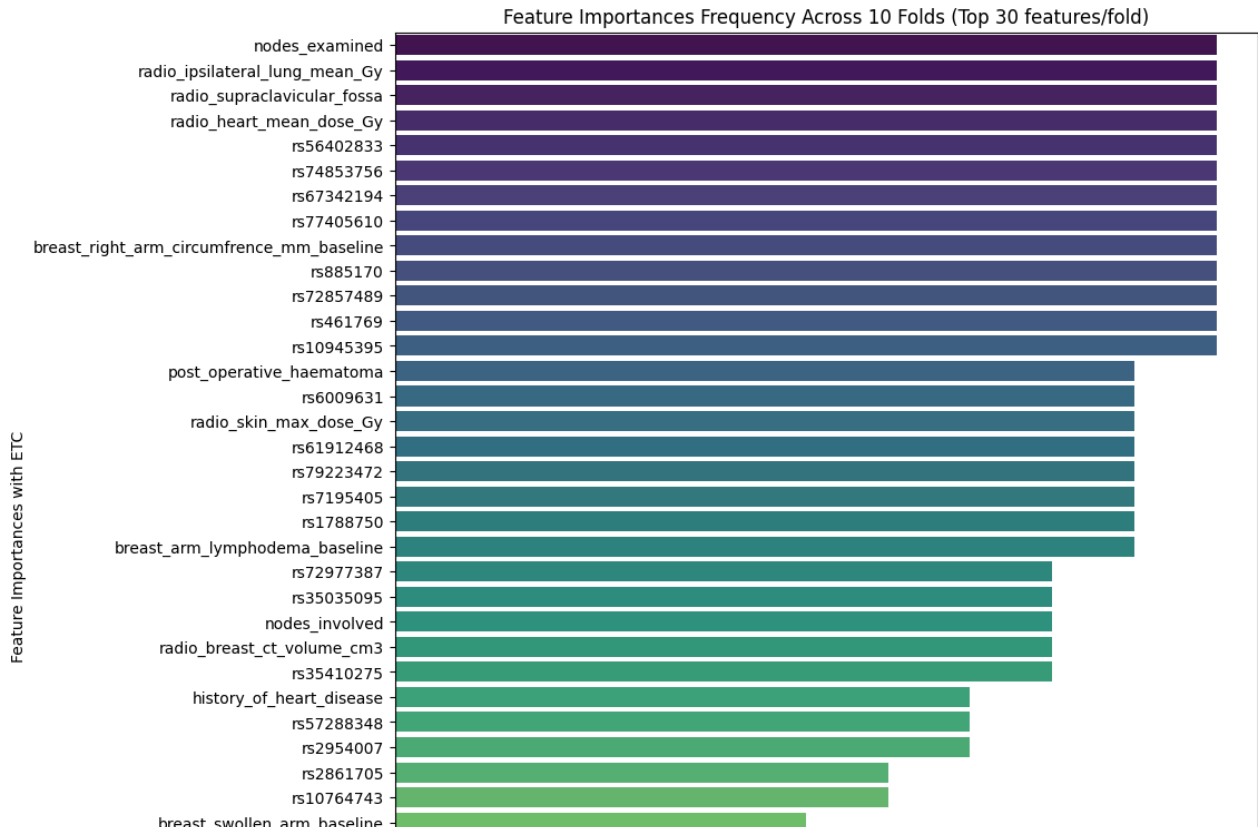
Fold 7: Selected features: ['history_of_heart_disease', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_supraclavicular_fossa', 'rs74853756', 'rs56402833', 'rs461769', 'rs77405610', 'rs67342194', 'rs885170', 'rs10945395', 'rs72857489', 'rs79223472', 'radio_heart_mean_dose_Gy', 'rs7195405', 'breast_arm_lymphodema_baseline', 'radio_ipsilateral_lung_mean_Gy', 'rs1788750', 'radio_skin_max_dose_Gy', 'post_operative_haematoma', 'breast_right_arm_circumfrence_mm_baseline', 'rs6009631', 'rs61912468', 'rs57288348', 'history_of_heart_disease', 'nodes_involved', 'rs35410275', 'rs72977387', 'radio_breast_ct_volume_cm3', 'rs2861705', 'rs35035095', 'rs10764743', 'her2_status', 'tumour_size_mm', 'radio_photon_2nd', 'axillary_levels', 'breast_swollen_arm_baseline', 'rs2954007', 'age', 'radio_hot_spots_107', 'analgesics']

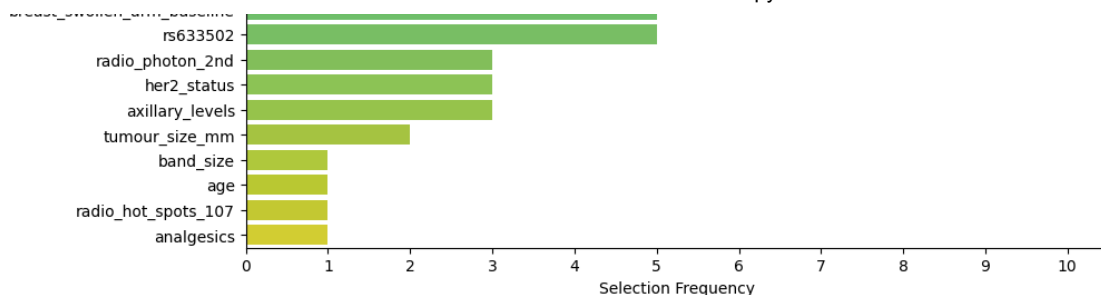
Fold 8: Selected features: ['analgesics', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_skin_max_dose_Gy', 'radio_breast_ct_volume_cm3', 'radio_ipsilateral_lung_mean_Gy', 'rs1788750', 'radio_skin_max_dose_Gy', 'post_operative_haematoma', 'breast_right_arm_circumfrence_mm_baseline', 'rs6009631', 'rs61912468', 'rs57288348', 'history_of_heart_disease', 'nodes_involved', 'rs35410275', 'rs72977387', 'radio_breast_ct_volume_cm3', 'rs2861705', 'rs35035095', 'rs10764743', 'her2_status', 'tumour_size_mm', 'radio_photon_2nd', 'axillary_levels', 'breast_swollen_arm_baseline', 'rs2954007', 'age', 'radio_hot_spots_107', 'analgesics']

Fold 9: Selected features: ['nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'radio_breast_ct_volume_cm3', 'radio_ipsilateral_lung_mean_Gy', 'rs1788750', 'radio_skin_max_dose_Gy', 'post_operative_haematoma', 'breast_right_arm_circumfrence_mm_baseline', 'rs6009631', 'rs61912468', 'rs57288348', 'history_of_heart_disease', 'nodes_involved', 'rs35410275', 'rs72977387', 'radio_breast_ct_volume_cm3', 'rs2861705', 'rs35035095', 'rs10764743', 'her2_status', 'tumour_size_mm', 'radio_photon_2nd', 'axillary_levels', 'breast_swollen_arm_baseline', 'rs2954007', 'age', 'radio_hot_spots_107', 'analgesics']

Fold 10: Selected features: ['history_of_heart_disease', 'nodes_involved', 'nodes_examined', 'post_operative_haematoma', 'tumour_size_mm', 'radio_breast_ct_volume_cm3', 'radio_ipsilateral_lung_mean_Gy', 'rs1788750', 'radio_skin_max_dose_Gy', 'post_operative_haematoma', 'breast_right_arm_circumfrence_mm_baseline', 'rs6009631', 'rs61912468', 'rs57288348', 'history_of_heart_disease', 'nodes_involved', 'rs35410275', 'rs72977387', 'radio_breast_ct_volume_cm3', 'rs2861705', 'rs35035095', 'rs10764743', 'her2_status', 'tumour_size_mm', 'radio_photon_2nd', 'axillary_levels', 'breast_swollen_arm_baseline', 'rs2954007', 'age', 'radio_hot_spots_107', 'analgesics']

Mean AUC using top 30 features across 10 folds: 0.8511





	Frequency
nodes_examined	10
radio_ipsilateral_lung_mean_Gy	10
radio_supraclavicular_fossa	10
radio_heart_mean_dose_Gy	10
rs56402833	10
rs74853756	10
rs67342194	10
rs77405610	10
breast_right_arm_circumfrence_mm_baseline	10
rs885170	10
rs72857489	10
rs461769	10
rs10945395	10
post_operative_haematoma	9
rs6009631	9
radio_skin_max_dose_Gy	9
rs61912468	9
rs79223472	9
rs7195405	9
rs1788750	9
breast_arm_lymphodema_baseline	9
rs72977387	8
rs35035095	8
nodes_involved	8
radio_breast_ct_volume_cm3	8
rs35410275	8
history_of_heart_disease	7
rs57288348	7
rs2954007	7
rs2861705	6
rs10764743	6
breast_swollen_arm_baseline	5
rs633502	5
radio_photon_2nd	3
her2_status	3
axillary_levels	3
tumour_size_mm	2
band_size	1
age	1
radio_hot_spots_107	1
analgesics	1

Top 30 most frequently Features across 10 folds: ['nodes_examined', 'radio_ipsilateral_lung_mean_Gy', 'radio_supraclavicular

feature_sets_univariate_scaled_inside_pipeline

```
{10: (
nodes_examined          Frequency
post_operative_haematoma 10
radio_supraclavicular_fossa 10
rs72857489               10
rs67342194               10
breast_swollen_arm_baseline 9
breast_arm_lymphodema_baseline 9
rs56402833               9
axillary_levels          6
diabetes                  5
analgesics                5
radio_photon_2nd          2
history_of_heart_disease  2
chemo_adjuvant            2
rs74853756               1,
np.float64(0.8316789215686274)),
15: (
nodes_examined          Frequency
radio_supraclavicular_fossa 10
post_operative_haematoma 10
rs56402833               10
rs67342194               10
rs72857489               10
breast_swollen_arm_baseline 9
breast_arm_lymphodema_baseline 9
analgesics                9
diabetes                  8
axillary_levels          7
radio_photon_2nd          6
rs74853756               6
chemo_neo_adjuvant        6
chemo_adjuvant            6
history_of_heart_disease  5
nodes_involved            3
rs77405610               3
rs10945395               3
radio_hot_spots_107       3
hypertension              2
rs7195405                2
surgery_type              1
rs461769                  1
depression                1,
np.float64(0.8333333333333334)),
20: (
diabetes                  Frequency
post_operative_haematoma 10
nodes_examined          10
radio_supraclavicular_fossa 10
rs67342194               10
rs74853756               10
rs56402833               10
rs72857489               10
breast_arm_lymphodema_baseline 9
analgesics                9
rs7195405                9
breast_swollen_arm_baseline 9
chemo_adjuvant            8
```

```
#print(feature_sets_featimportance_inside_pipeline)
```

```
stable_features_featimportance_inside_pipeline = get_frequent.
feature_sets_featimportance_inside_pipeline,
freq_threshold=8
)
```

```
print(stable_features_featimportance_inside_pipeline)
print(len(stable_features_featimportance_inside_pipeline))
```

```
['rs72857489', 'radio_skin_max_dose_Gy', 'radio_ipsilateral_lung_mean_Gy', 'rs74853756', 'rs79223472', 'post_operative_haema
26
```

```
#df_Top_selected = df_20[top_features_final + [dependent1]]
cross_validated_feature_selection(df_clean_20[stable_features.
```


Original Data shape is (2067, 27)
Data shape after dropping missing target: (1691, 27)
✅ Mean AUC with final selected features: 0.8962

▼ Nested CV

```
summary2, fold_details2, feature_frequency2 = nested_cv_featu
```

Outer CV: 10it [02:11, 13.16s/it]

summary2

	mean_auc	std_auc	
RandomForest	0.876225	0.078748	

fold_details2

	fold	model	best_k	inner_auc	outer_auc	selected_features	
0	1	RandomForest	30	0.802979	0.985294	[history_of_heart_disease, nodes_involved, nod...	
1	2	RandomForest	20	0.823063	0.880515	[nodes_examined, post_operative_haematoma, rad...	
2	3	RandomForest	30	0.906602	0.761029	[history_of_heart_disease, nodes_examined, pos...	
3	4	RandomForest	30	0.867436	0.733456	[history_of_heart_disease, nodes_involved, nod...	
4	5	RandomForest	30	0.856727	0.897059	[history_of_heart_disease, nodes_examined, her...	
5	6	RandomForest	30	0.843191	0.862745	[nodes_involved, nodes_examined, post_operativ...	
6	7	RandomForest	30	0.855372	0.901961	[history_of_heart_disease, nodes_involved, nod...	
7	8	RandomForest	30	0.850822	0.946078	[analgesics, nodes_involved, nodes_examined, p...	
8	9	RandomForest	30	0.810227	0.965686	[nodes_involved, nodes_examined, post_operativ...	
9	10	RandomForest	20	0.879610	0.828431	[nodes_involved, nodes_examined, post_operativ...	

Next steps: [Generate code with fold_details2](#) [New interactive sheet](#)

```
#print(feature_frequency.head(40))  
display(feature_frequency2.head(100))
```

	feature	count	percentage	
0	nodes_examined	10	100.0	
1	radio_heart_mean_dose_Gy	10	100.0	
2	radio_ipsilateral_lung_mean_Gy	10	100.0	
3	rs56402833	10	100.0	
4	rs67342194	10	100.0	
5	rs77405610	10	100.0	
6	radio_supraclavicular_fossa	10	100.0	
7	rs885170	10	100.0	
8	rs461769	10	100.0	
9	rs10945395	10	100.0	
10	rs74853756	10	100.0	
11	rs72857489	10	100.0	
12	breast_right_arm_circumfrence_mm_baseline	9	90.0	
13	rs79223472	9	90.0	
14	post_operative_haematoma	9	90.0	
15	rs7195405	9	90.0	
16	breast_arm_lymphodema_baseline	9	90.0	
17	rs72977387	8	80.0	
18	rs6009631	8	80.0	
19	rs61912468	8	80.0	
20	rs1788750	8	80.0	
21	nodes_involved	7	70.0	
22	radio_breast_ct_volume_cm3	7	70.0	
23	radio_skin_max_dose_Gy	7	70.0	
24	rs35035095	7	70.0	
25	rs35410275	7	70.0	
26	rs57288348	6	60.0	
27	rs2861705	6	60.0	
28	history_of_heart_disease	5	50.0	
29	rs10764743	5	50.0	
30	rs2954007	5	50.0	
31	breast_swollen_arm_baseline	5	50.0	
32	rs633502	4	40.0	
33	radio_photon_2nd	3	30.0	
34	her2_status	3	30.0	
35	axillary_levels	2	20.0	
36	tumour_size_mm	2	20.0	
37	radio_hot_spots_107	1	10.0	
38	analgesics	1	10.0	

```
# Features that appeared in 8 folds and more across 10 folds
stable_features2 = feature_frequency2[feature_frequency2["count"] >= 8]
```

```
print (stable_features2)
```

```
['nodes_examined', 'radio_heart_mean_dose_Gy', 'radio_ipsilateral_lung_mean_Gy', 'rs56402833', 'rs67342194', 'rs77405610', 'rs72857489', 'rs72977387', 'rs6009631', 'rs61912468', 'rs1788750', 'nodes_involved', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'rs35035095', 'rs35410275', 'rs57288348', 'rs2861705', 'history_of_heart_disease', 'rs10764743', 'rs2954007', 'breast_swollen_arm_baseline', 'rs633502', 'radio_photon_2nd', 'her2_status', 'axillary_levels', 'tumour_size_mm', 'radio_hot_spots_107', 'analgesics']
```

```
print(len(stable_features2))
```

21

```
jaccard_stability(fold_details2, "RandomForest")
```

```
np.float64(0.6787428956594043)
```

```
cross_validated_feature_selection(df_clean_20[stable_features:
```

```
Original Data shape is (2067, 22)
```

```
Data shape after dropping missing target: (1691, 22)
```

```
✅ Mean AUC with final selected features: 0.9064
```

Third feature selection method

3.3.4 Recursive Feature Elimination (RFE)

Recursive Feature Elimination (RFE) is a feature selection method used in machine learning to identify the most relevant features for a predictive model. It works by recursively removing less important features and building the model on the remaining features to determine their importance.

outside CV

```
feature_sets_rfe = {}
```

```
for i in k:
```

```
    freq_df_scaled_auc= rfe_feature_selection3(df_clean_20, d
```

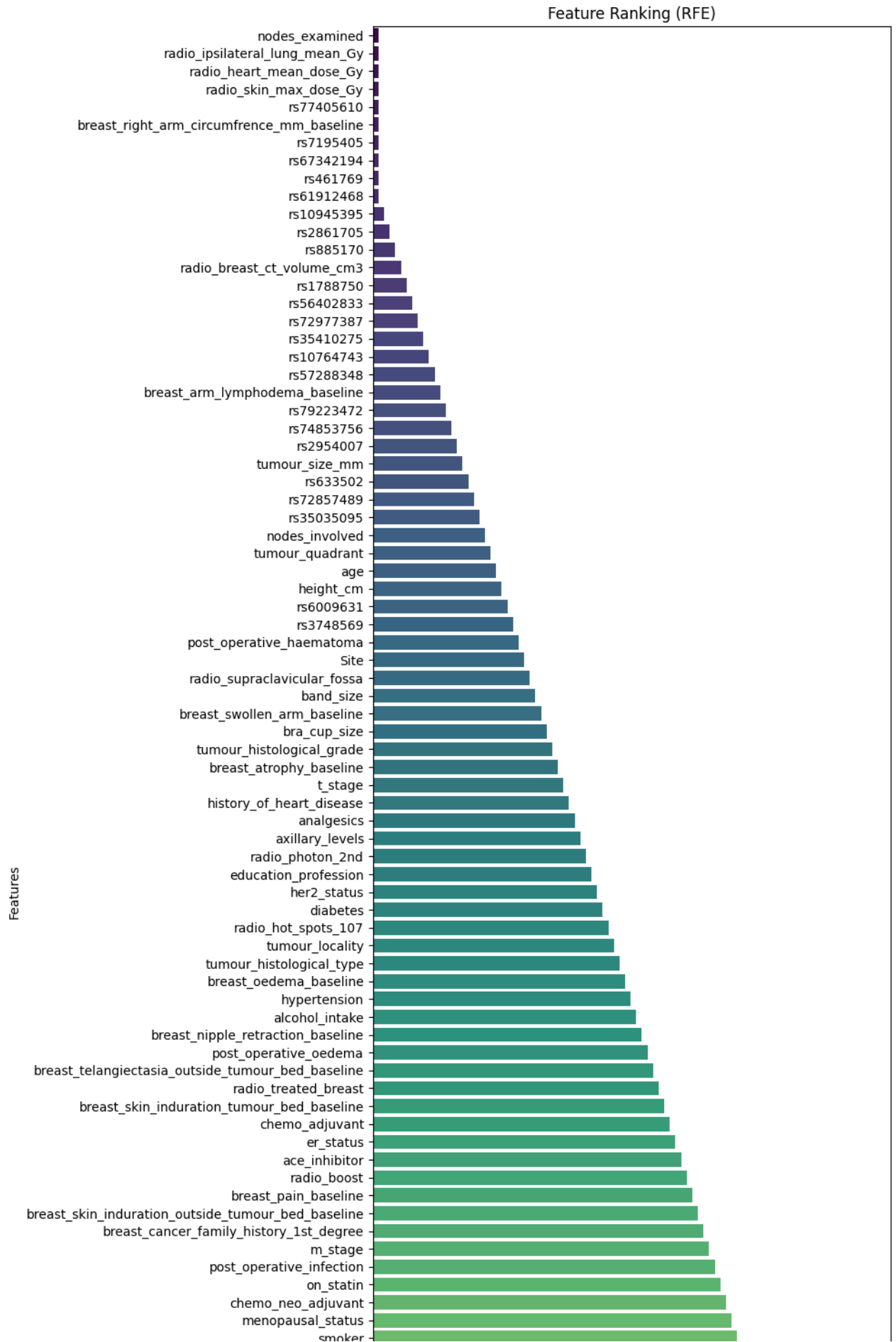
```
    feature_sets_rfe[i] = freq_df_scaled_auc
```

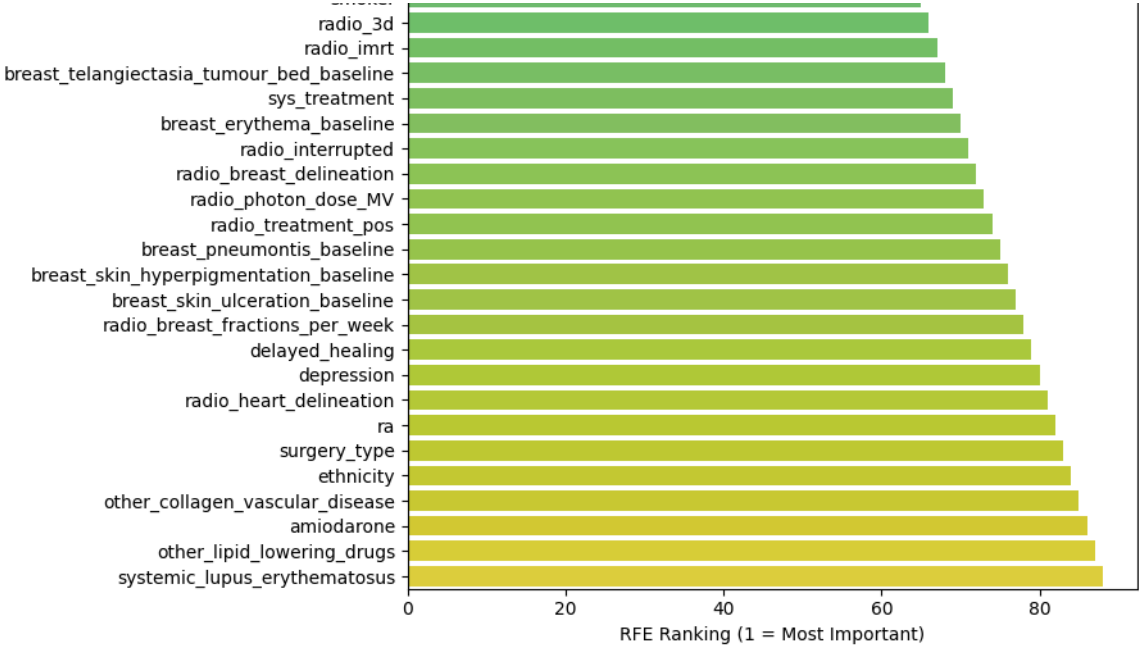

Starting with 10 Features:

List of top 10 features:

['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ipsilateral_lung_mean_Gy', 'breast_right_ar

Mean AUC: 0.8462 | Std: 0.0929

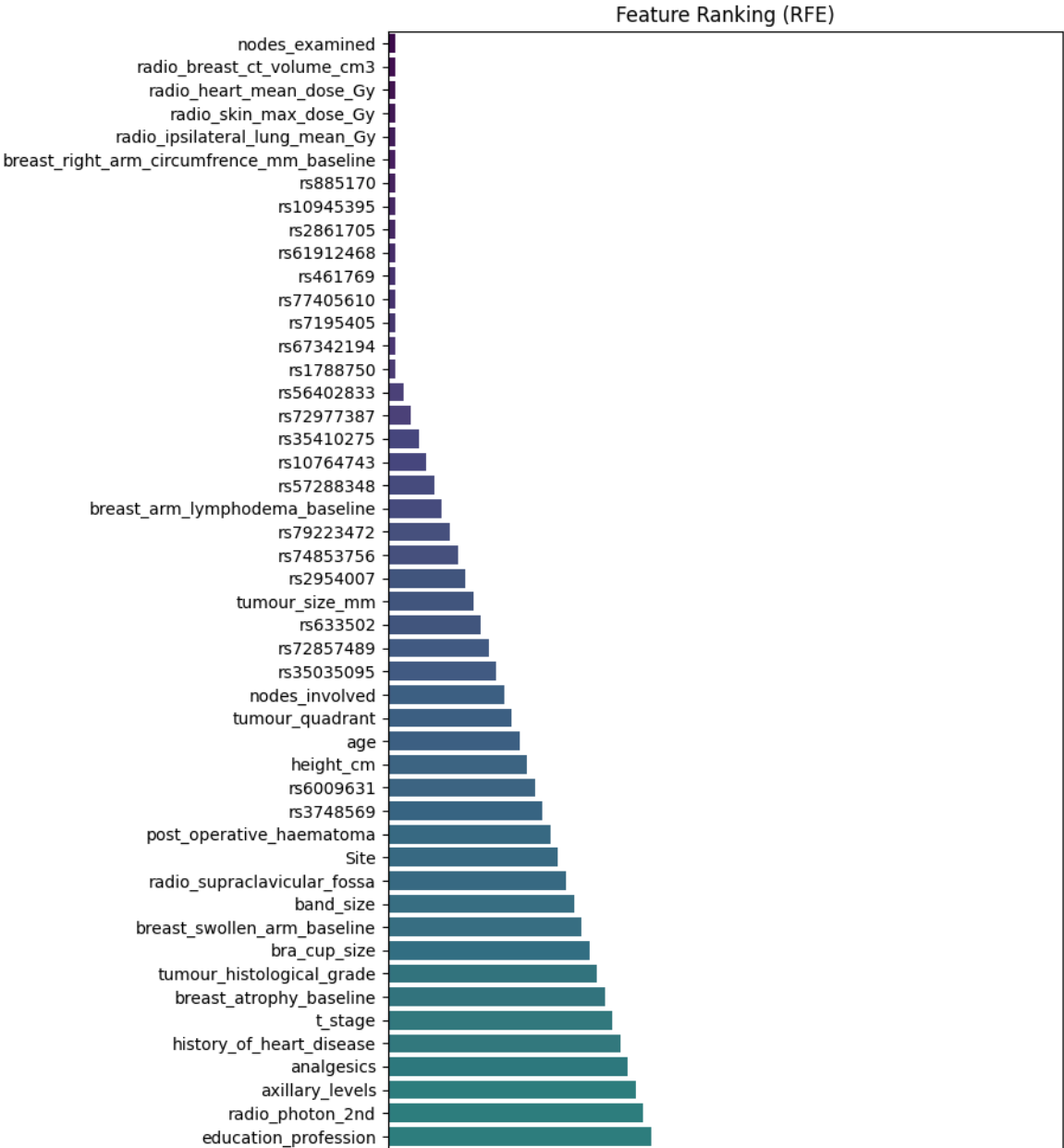


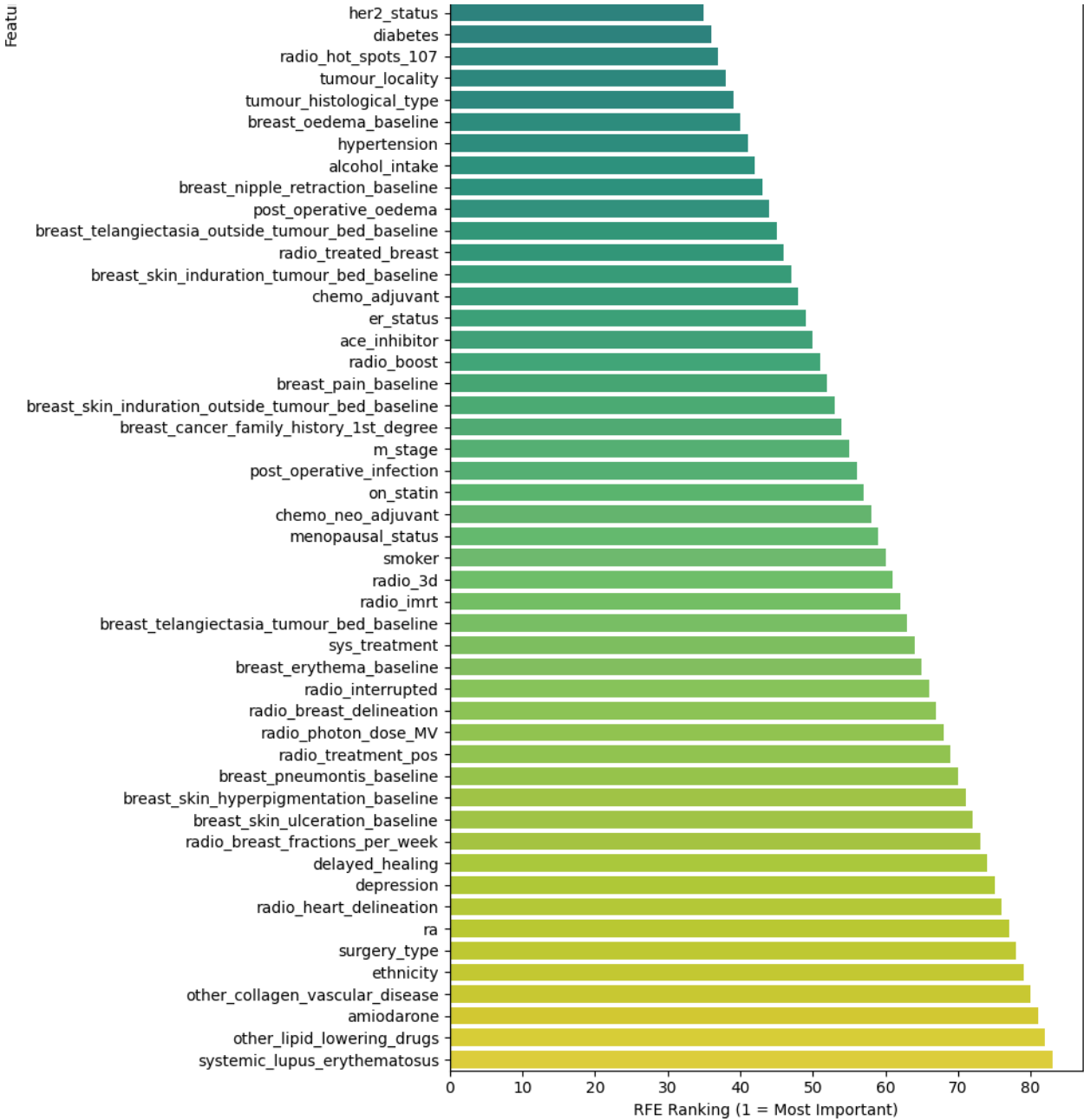


Starting with 15 Features:

List of top 15 features:
['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ipsilateral_lung_mean_Gy', 'breast_right_arm_circumfrence_mm_baseline', 'rs885170', 'rs10945395', 'rs2861705', 'rs61912468', 'rs461769', 'rs77405610', 'rs7195405', 'rs67342194', 'rs1788750', 'rs56402833', 'rs72977387', 'rs35410275', 'rs10764743', 'rs57288348', 'breast_arm_lymphodema_baseline', 'rs79223472', 'rs74853756', 'rs2954007', 'tumour_size_mm', 'rs633502', 'rs72857489', 'rs35035095', 'nodes_involved', 'tumour_quadrant', 'age', 'height_cm', 'rs6009631', 'rs3748569', 'post_operative_haematoma', 'Site', 'radio_supraclavicular_fossa', 'band_size', 'breast_swollen_arm_baseline', 'bra_cup_size', 'tumour_histological_grade', 'breast_atrophy_baseline', 't_stage', 'history_of_heart_disease', 'analgesics', 'axillary_levels', 'radio_photon_2nd', 'education_profession']

Mean AUC: 0.8620 | Std: 0.0958

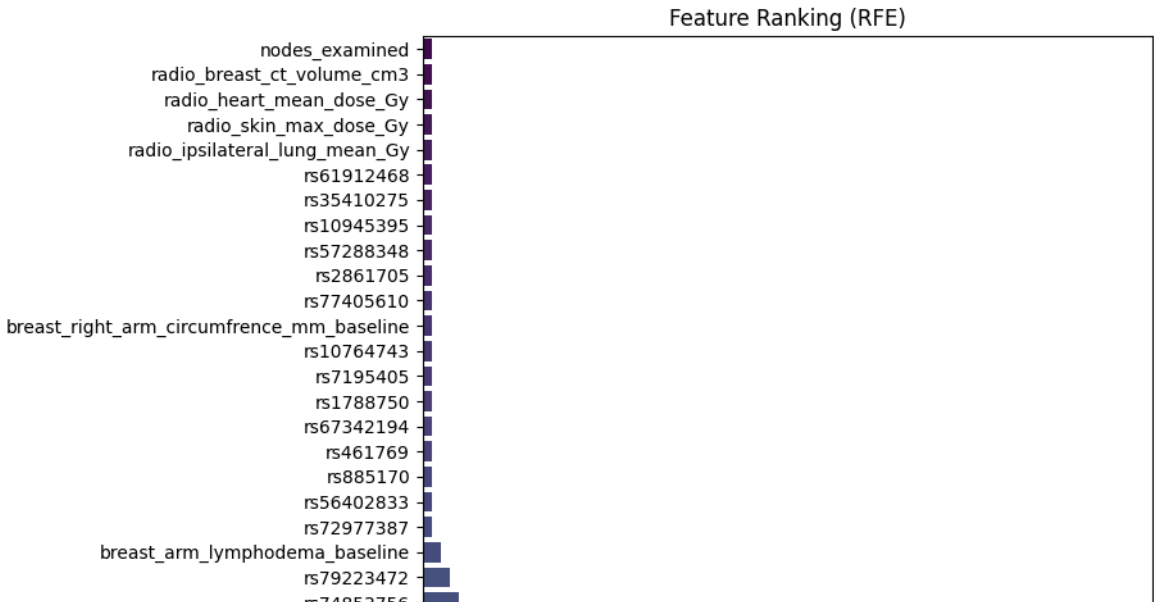




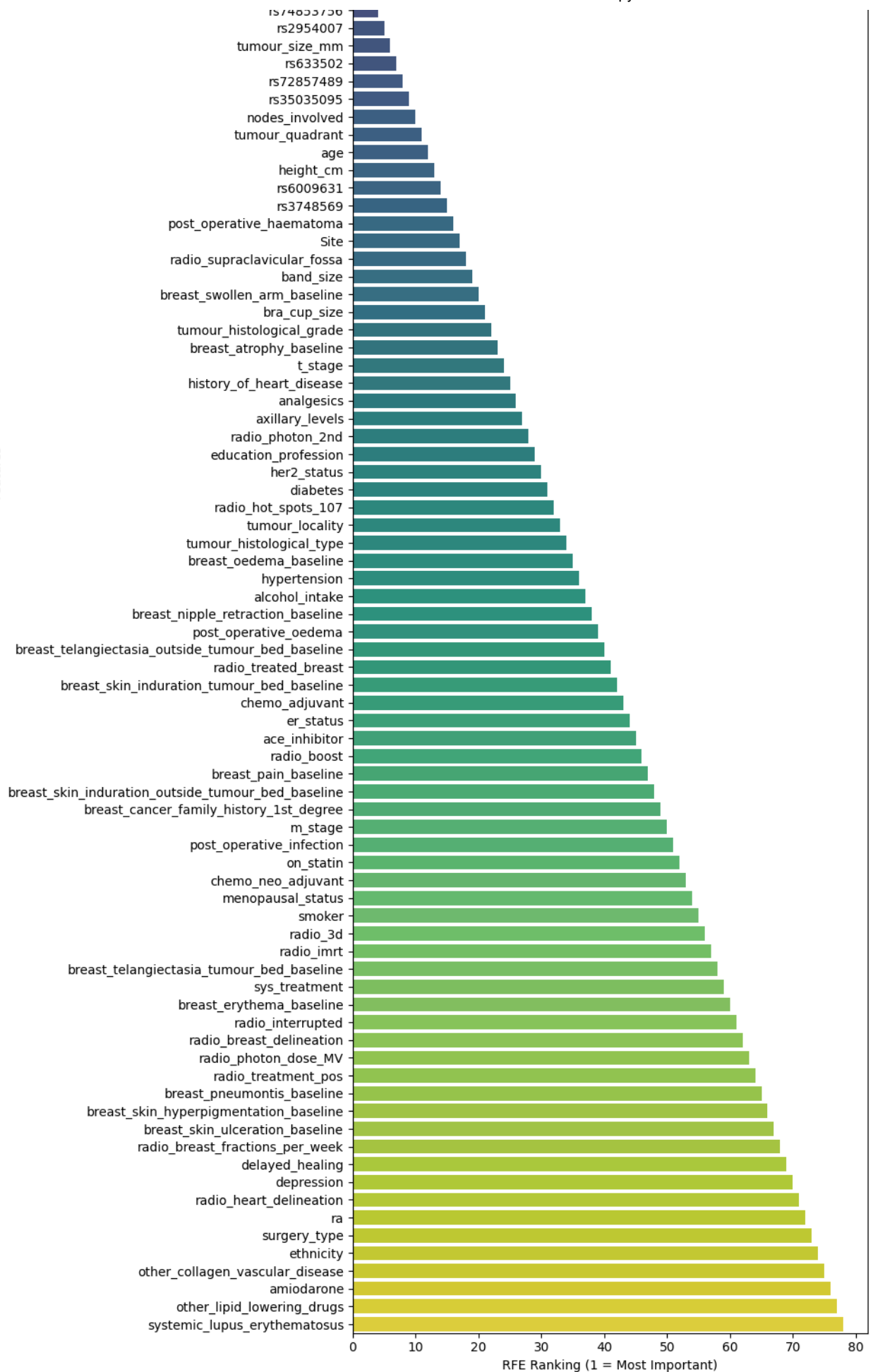
Starting with 20 Features:

List of top 20 features:
['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ipsilateral_lung_mean_dose_Gy', 'rs61912468', 'rs35410275', 'rs10945395', 'rs57288348', 'rs2861705', 'rs77405610', 'breast_right_arm_circumfrence_mm_baseline', 'rs10764743', 'rs7195405', 'rs1788750', 'rs67342194', 'rs461769', 'rs885170', 'rs56402833', 'rs72977387', 'breast_arm_lymphodema_baseline', 'rs79223472', 'rs74853756']

Mean AUC: 0.8610 | Std: 0.1072



Features

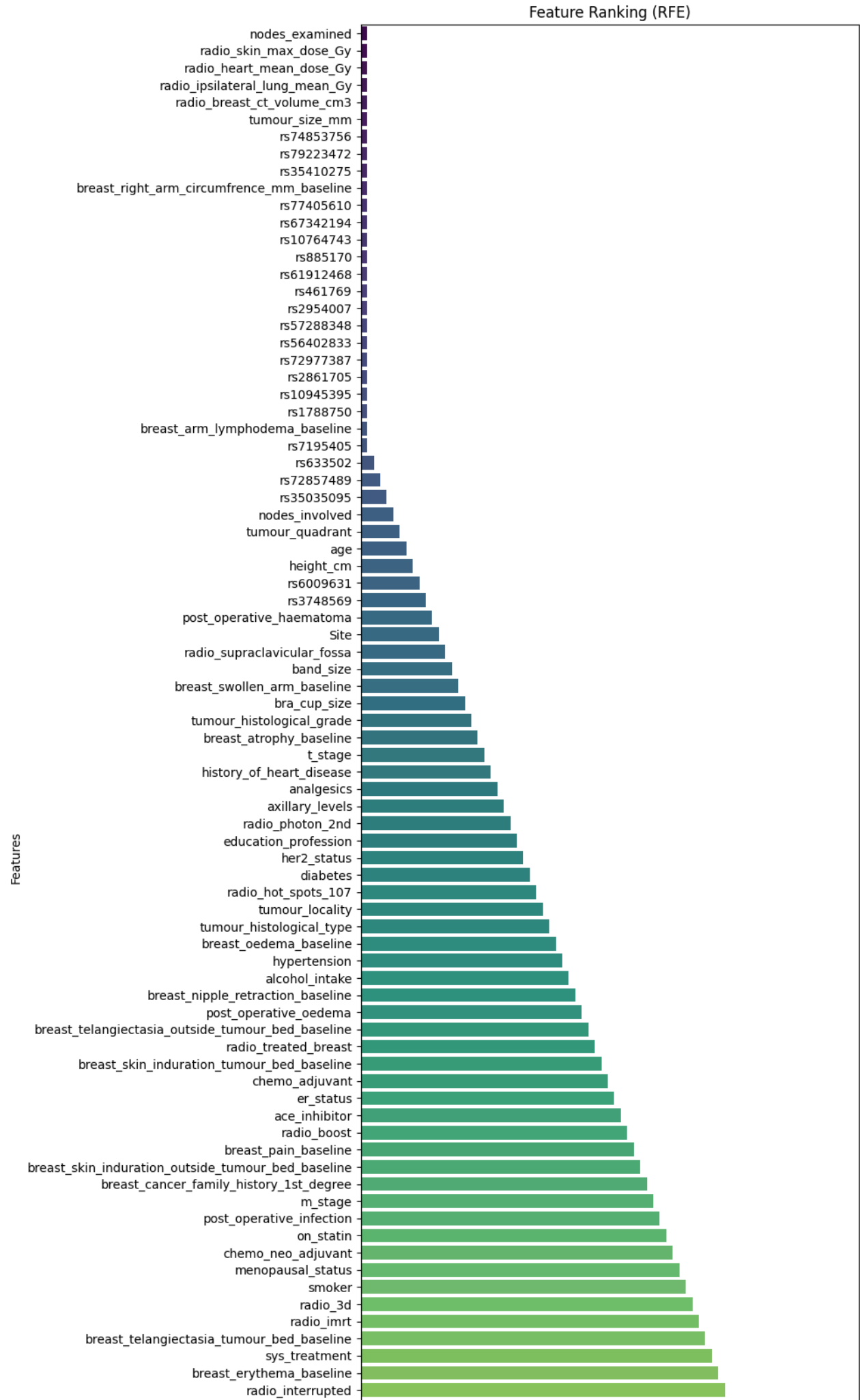


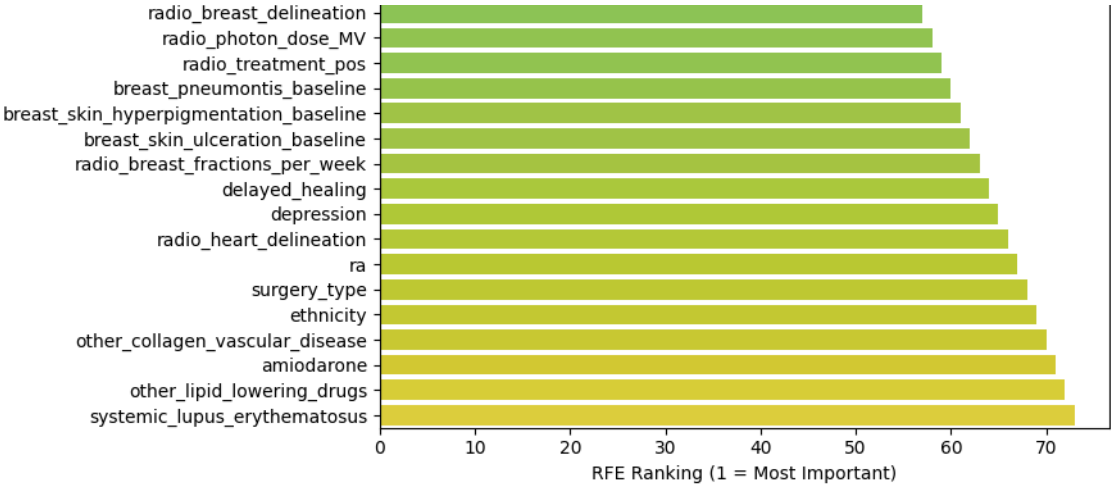
Starting with 25 Features:

List of top 25 features:

['nodes_examined', 'tumour_size_mm', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'r

Mean AUC: 0.9037 | Std: 0.0576

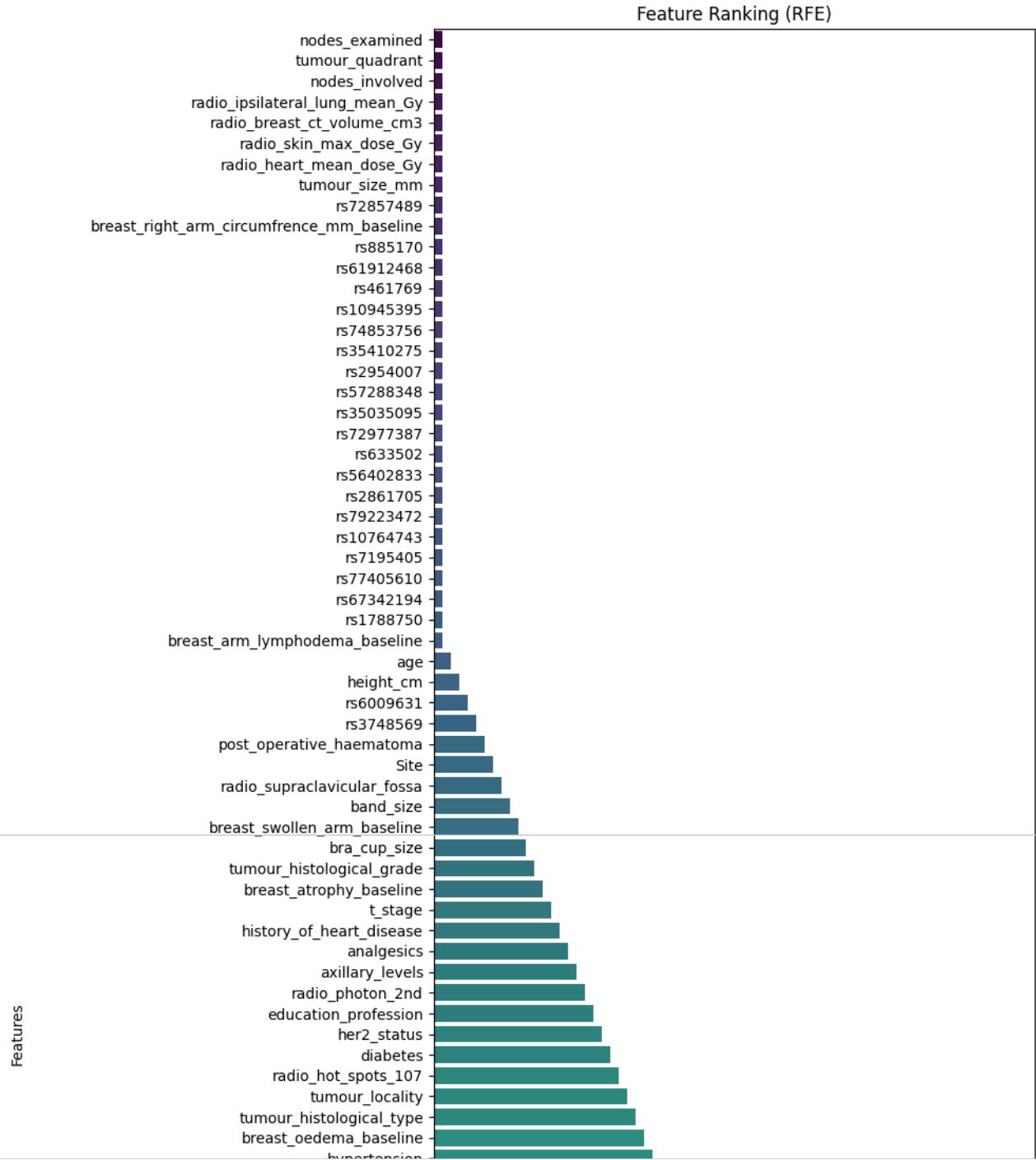




Starting with 30 Features:

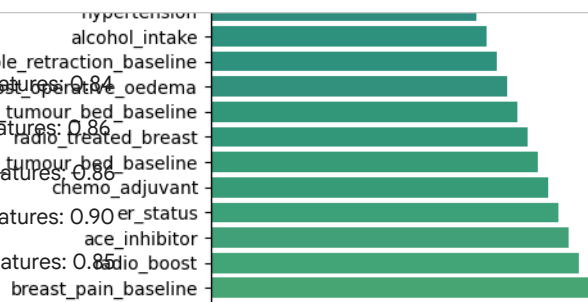
List of top 30 features:
['nodes_involved', 'nodes_examined', 'tumour_quadrant', 'tumour_size_mm', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'tumour_size_mm', 'rs72857489', 'breast_right_arm_circumfrence_mm_baseline', 'rs885170', 'rs61912468', 'rs461769', 'rs10945395', 'rs74853756', 'rs35410275', 'rs2954007', 'rs57288348', 'rs35035095', 'rs72977387', 'rs633502', 'rs56402833', 'rs2861705', 'rs79223472', 'rs10764743', 'rs7195405', 'rs77405610', 'rs67342194', 'rs1788750', 'breast_arm_lymphodema_baseline', 'age', 'height_cm', 'rs6009631', 'rs3748569', 'post_operative_haematoma', 'Site', 'radio_supraclavicular_fossa', 'band_size', 'breast_swollen_arm_baseline', 'bra_cup_size', 'tumour_histological_grade', 'breast_atrophy_baseline', 't_stage', 'history_of_heart_disease', 'analgesics', 'axillary_levels', 'radio_photon_2nd', 'education_profession', 'her2_status', 'diabetes', 'radio_hot_spots_107', 'tumour_locality', 'tumour_histological_type', 'breast_oedema_baseline', 'tumour_size_mm']

Mean AUC: 0.8549 | Std: 0.0689



Takeaway6:

- Mean AUC score using top 10 features: 0.84
- Mean AUC score using top 15 features: 0.86
- Mean AUC score using top 20 features: 0.88
- Mean AUC score using top 25 features: 0.90
- Mean AUC score using top 30 features: 0.85



```

from collections import Counter
import math

def get_stable_rfe_features(feature_sets_rfe, threshold_ratio):
    feature_counter = Counter()

    n_sets = len(feature_sets_rfe)
    threshold = math.ceil(threshold_ratio * n_sets)

    for k_value, (selected_features, auc, ranking_df) in feature_sets_rfe.items():
        # Features selected by RFE have Ranking == 1
        rfe_features = ranking_df.loc[
            ranking_df['Ranking'] == 1,
            'Feature'
        ].tolist()

        feature_counter.update(rfe_features)

    stable_features = [
        feature for feature, count in feature_counter.items()
        if count >= threshold
    ]

    return stable_features

```

```

stable_rfe_features = get_stable_rfe_features(
    feature_sets_rfe,
    threshold_ratio=0.8
)

```

```

print(stable_rfe_features)
print(len(stable_rfe_features))

```

```

['nodes_examined', 'radio_ipsilateral_lung_mean_Gy', 'radio_heart_mean_dose_Gy', 'radio_skin_max_dose_Gy', 'rs77405610', 'br
15

```

```

cross_validated_feature_selection(df_clean_20[stable_rfe_features])

```

```

Original Data shape is (2067, 16)
Data shape after dropping missing target: (1691, 16)
✅ Mean AUC with final selected features: 0.8564

```

▼ inside CV

```
#rfe_feature_selection3(df_cl20, dependent1, 20)

auc_scores_rfe_inside_pipeline = {}

for i in k:
    freq_df_scaled_auc= run_pipeline_with_feature_selection3(df_cl20, dependent1, i)
    auc_scores_rfe_inside_pipeline[i] = freq_df_scaled_auc
```

```
Mean AUC score using top 10 features Inside CV: 0.79
Mean AUC score using top 15 features Inside CV: 0.83
Mean AUC score using top 20 features Inside CV: 0.85
Mean AUC score using top 25 features Inside CV: 0.88
Mean AUC score using top 30 features Inside CV: 0.89
```

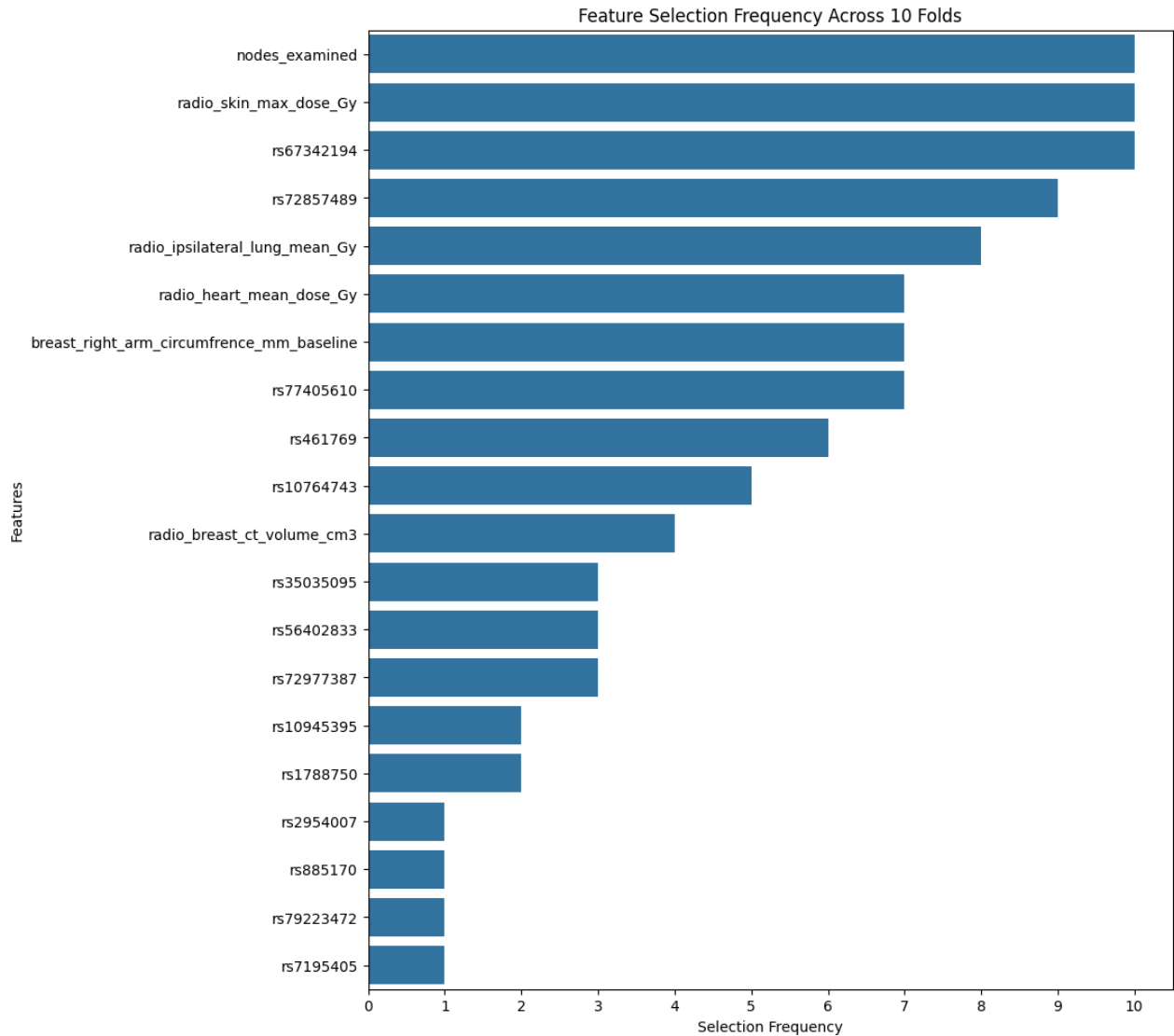
▼ Features frequency inside pipeline

```
feature_set_rfe_CV_inside_pipeline = {}
for i in k:
    feature_set_rfe_CV_inside_pipeline[i] = featureselection3_freq(df_cl20, dependent1, i)
```


Process with 10 Features:

Fold 1: AUC = 0.9522 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 2: AUC = 0.6434 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_
Fold 3: AUC = 0.7390 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_ipsilateral_lung_mean_Gy', 'br
Fold 4: AUC = 0.6949 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 5: AUC = 0.7745 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_
Fold 6: AUC = 0.8725 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_ipsilateral_lung_mean_Gy', 'br
Fold 7: AUC = 0.8725 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_
Fold 8: AUC = 0.7917 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_
Fold 9: AUC = 0.8922 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 10: AUC = 0.6789 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'breast_

Mean AUC: 0.7912 across 10 features | Std: 0.0980



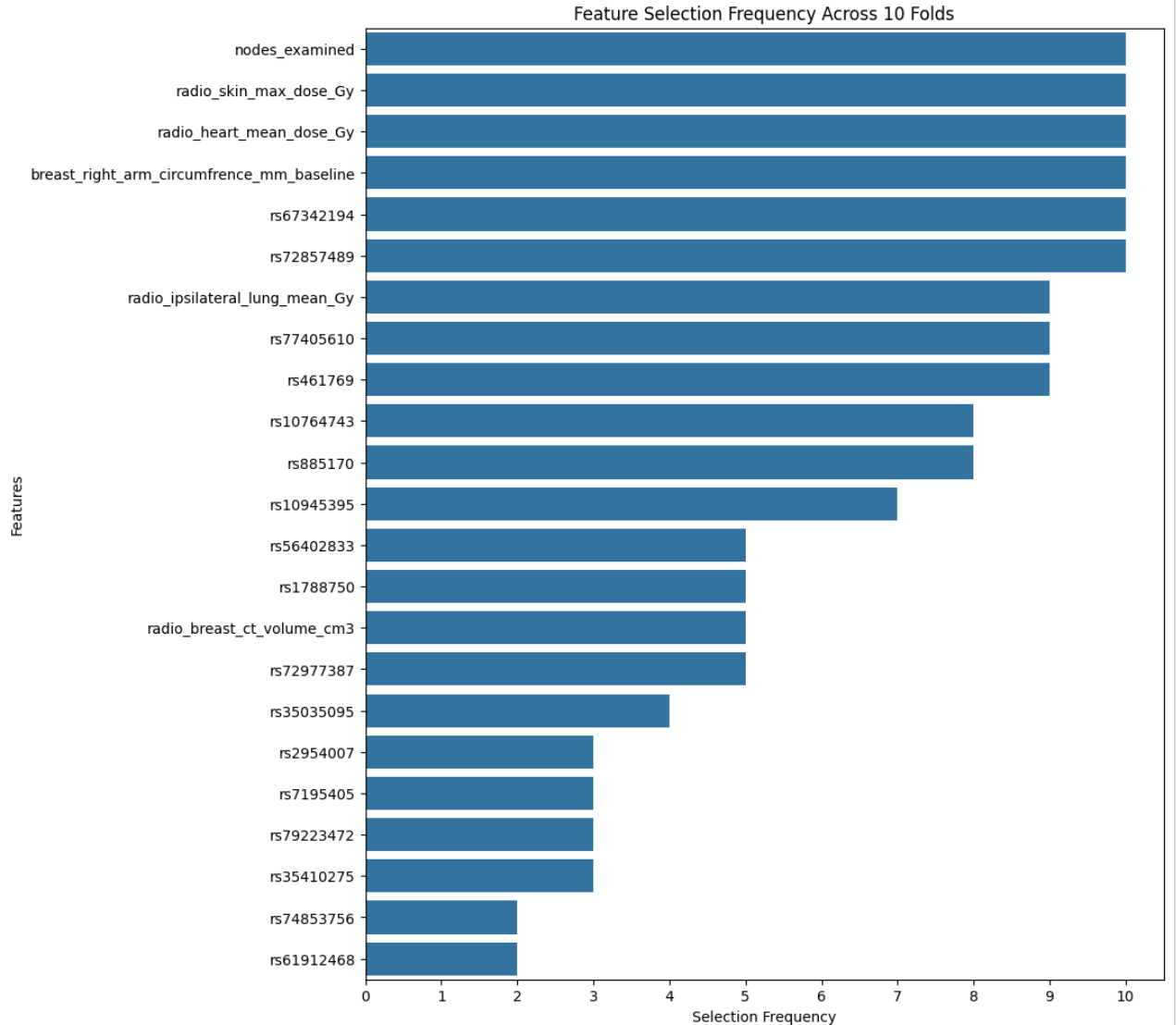
	Frequency
nodes_examined	10
radio_skin_max_dose_Gy	10
rs67342194	10
rs72857489	9
radio_ipsilateral_lung_mean_Gy	8
radio_heart_mean_dose_Gy	7
breast_right_arm_circumfrence_mm_baseline	7
rs77405610	7
rs461769	6
rs10764743	5
radio_breast_ct_volume_cm3	4
rs35035095	3
rs56402833	3
rs72977387	3
rs10945395	2
rs1788750	2
rs2954007	1
rs885170	1
rs79223472	1
rs7195405	1

Process with 15 Features:

Fold 1: AUC = 1.0000 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip

Fold 2: AUC = 0.7904 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 3: AUC = 0.6801 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 4: AUC = 0.8529 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 5: AUC = 0.8578 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 6: AUC = 0.9216 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 7: AUC = 0.8824 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 8: AUC = 0.9363 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 9: AUC = 0.8186 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 10: AUC = 0.5931 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'breast_

Mean AUC: 0.8333 across 15 features| Std: 0.1152



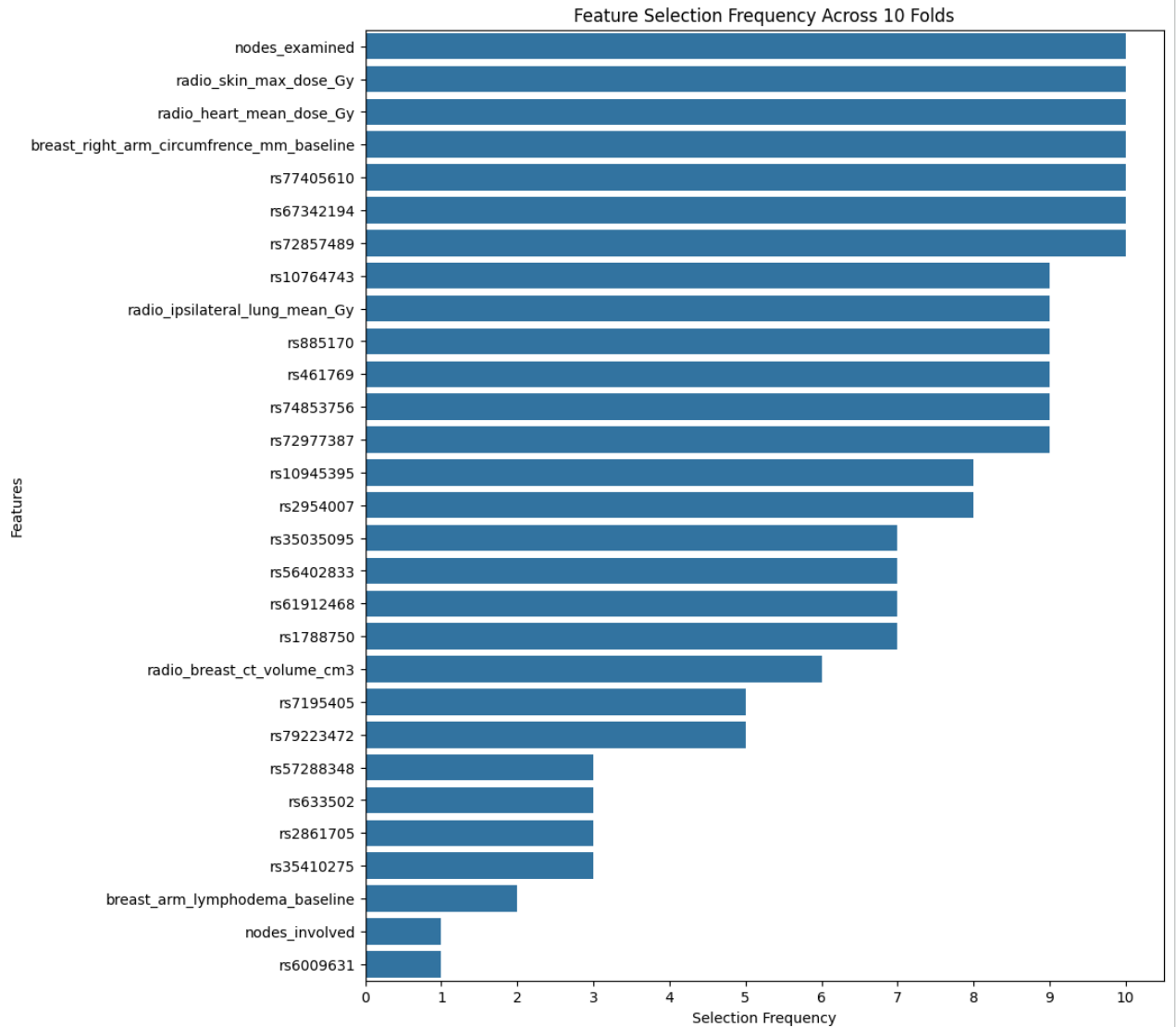
	Frequency
nodes_examined	10
radio_skin_max_dose_Gy	10
radio_heart_mean_dose_Gy	10
breast_right_arm_circumfrence_mm_baseline	10
rs67342194	10
rs72857489	10
radio_ipsilateral_lung_mean_Gy	9
rs77405610	9
rs461769	9
rs10764743	8
rs885170	8
rs10945395	7
rs56402833	5
rs1788750	5
radio_breast_ct_volume_cm3	5
rs72977387	5
rs35035095	4
rs2954007	3
rs7195405	3
rs79223472	3
rs35410275	3
rs74853756	2
rs61912468	2

Process with 20 Features:

Fold 1: AUC = 0.9890 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 2: AUC = 0.7831 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_ip

Fold 3: AUC = 0.7059 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 4: AUC = 0.7978 | Selected Features: ['nodes_involved', 'nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max
Fold 5: AUC = 0.8775 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_
Fold 6: AUC = 0.9216 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 7: AUC = 0.9118 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_
Fold 8: AUC = 0.8922 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_
Fold 9: AUC = 0.9363 | Selected Features: ['nodes_examined', 'radio_skin_max_dose_Gy', 'radio_heart_mean_dose_Gy', 'radio_ip
Fold 10: AUC = 0.6495 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_

Mean AUC: 0.8464 across 20 features | Std: 0.1031

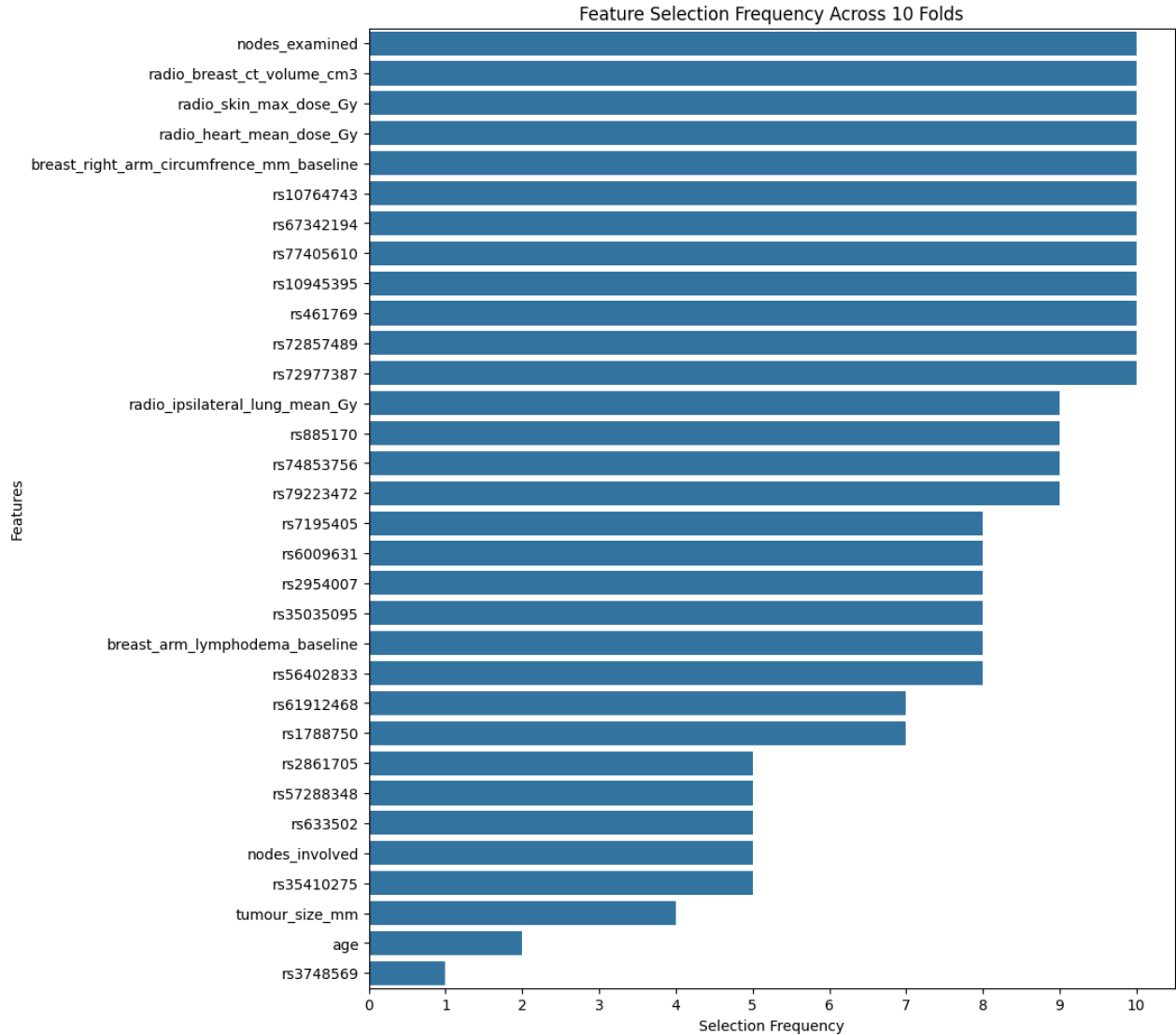


	Frequency
nodes_examined	10
radio_skin_max_dose_Gy	10
radio_heart_mean_dose_Gy	10
breast_right_arm_circumfrence_mm_baseline	10
rs77405610	10
rs67342194	10
rs72857489	10
rs10764743	9
radio_ipsilateral_lung_mean_Gy	9
rs885170	9
rs461769	9
rs74853756	9
rs72977387	9
rs10945395	8
rs2954007	8
rs35035095	7
rs56402833	7
rs61912468	7
rs1788750	7
radio_breast_ct_volume_cm3	6
rs7195405	5
rs79223472	5
rs57288348	3
rs633502	3
rs2861705	3
rs35410275	3
breast_arm_lymphodema_baseline	2
nodes_involved	1
rs6009631	1

Process with 25 Features:

Fold 1: AUC = 0.9816 | Selected Features: ['age', 'nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy',
Fold 2: AUC = 0.8713 | Selected Features: ['nodes_involved', 'nodes_examined', 'tumour_size_mm', 'radio_breast_ct_volume_cm3',
Fold 3: AUC = 0.7353 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_
Fold 4: AUC = 0.8640 | Selected Features: ['nodes_involved', 'nodes_examined', 'tumour_size_mm', 'radio_breast_ct_volume_cm3',
Fold 5: AUC = 0.8284 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_
Fold 6: AUC = 0.8922 | Selected Features: ['nodes_involved', 'nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_
Fold 7: AUC = 0.9118 | Selected Features: ['nodes_involved', 'nodes_examined', 'tumour_size_mm', 'radio_breast_ct_volume_cm3',
Fold 8: AUC = 0.8873 | Selected Features: ['age', 'nodes_examined', 'tumour_size_mm', 'radio_breast_ct_volume_cm3', 'radio_s
Fold 9: AUC = 0.9559 | Selected Features: ['nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_max_dose_Gy', 'radio_
Fold 10: AUC = 0.8284 | Selected Features: ['nodes_involved', 'nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_ma

Mean AUC: 0.8756 across 25 features | Std: 0.0661



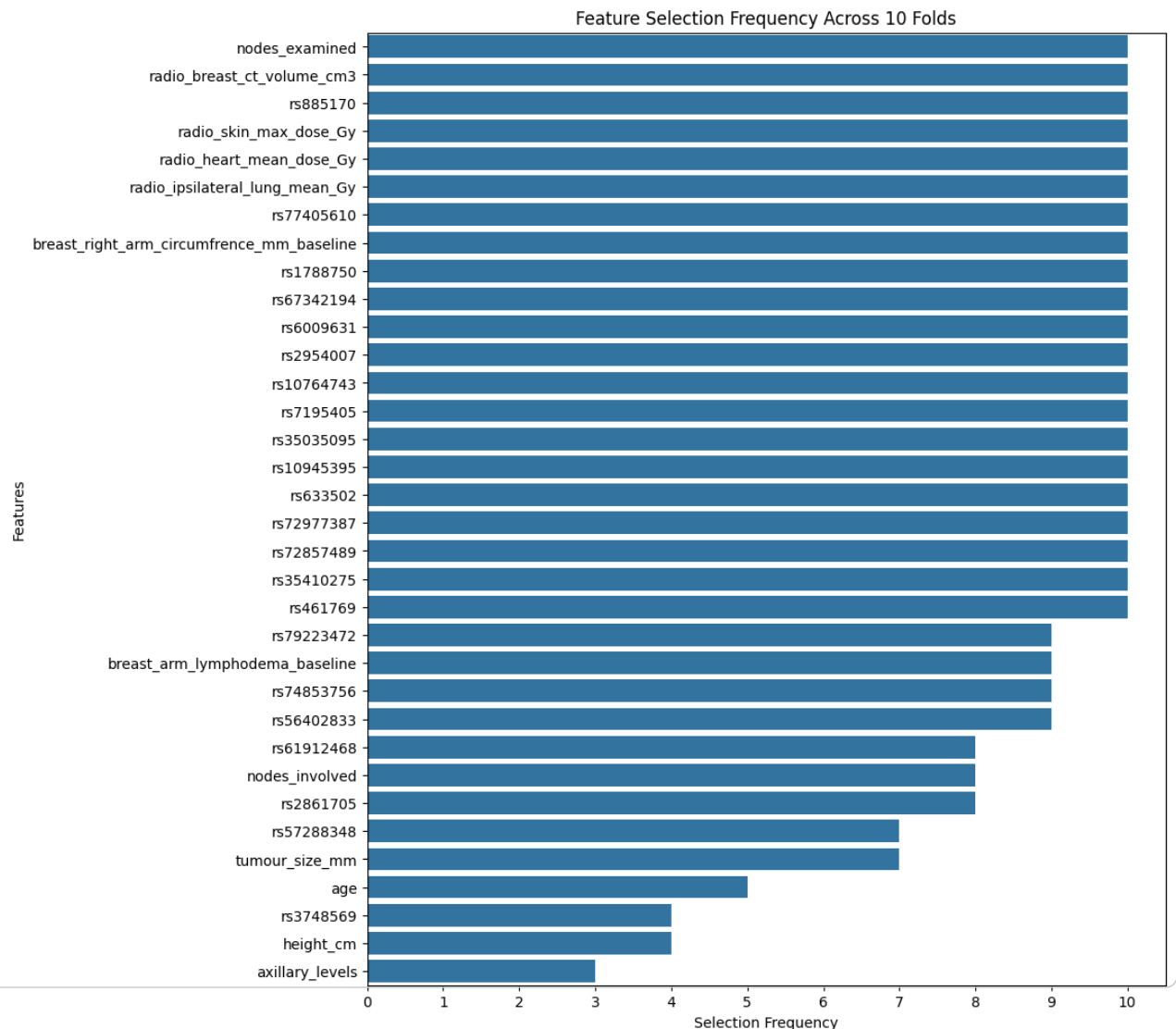
	Frequency
nodes_examined	10
radio_breast_ct_volume_cm3	10
radio_skin_max_dose_Gy	10
radio_heart_mean_dose_Gy	10
breast_right_arm_circumfrence_mm_baseline	10
rs10764743	10
rs67342194	10
rs77405610	10
rs10945395	10
rs461769	10
rs72857489	10
rs72977387	10
radio_ipsilateral_lung_mean_Gy	9
rs885170	9
rs74853756	9
rs79223472	9
rs7195405	8
rs6009631	8
rs2954007	8
rs35035095	8
breast_arm_lymphodema_baseline	8
rs56402833	8
rs61912468	7
rs1788750	7

rs2861705	5
rs57288348	5
rs633502	5
nodes_involved	5
rs35410275	5
tumour_size_mm	4
age	2
rs3748569	1

Process with 30 Features:

Fold 1: AUC = 1.0000 | Selected Features: ['age', 'nodes_examined', 'tumour_size_mm', 'radio_breast_ct_volume_cm3', 'radio_s
 Fold 2: AUC = 0.9007 | Selected Features: ['age', 'nodes_involved', 'nodes_examined', 'tumour_size_mm', 'radio_breast_ct_vol
 Fold 3: AUC = 0.7279 | Selected Features: ['height_cm', 'nodes_involved', 'nodes_examined', 'radio_breast_ct_volume_cm3', 'r
 Fold 4: AUC = 0.8713 | Selected Features: ['height_cm', 'nodes_involved', 'nodes_examined', 'tumour_size_mm', 'radio_breast
 Fold 5: AUC = 0.9363 | Selected Features: ['nodes_involved', 'nodes_examined', 'tumour_size_mm', 'radio_breast_ct_volume_cm3
 Fold 6: AUC = 0.9167 | Selected Features: ['height_cm', 'age', 'nodes_involved', 'nodes_examined', 'tumour_size_mm', 'radio_
 Fold 7: AUC = 0.8382 | Selected Features: ['nodes_involved', 'nodes_examined', 'tumour_size_mm', 'radio_breast_ct_volume_cm3
 Fold 8: AUC = 0.9265 | Selected Features: ['age', 'nodes_examined', 'tumour_size_mm', 'radio_breast_ct_volume_cm3', 'radio_s
 Fold 9: AUC = 0.9559 | Selected Features: ['height_cm', 'age', 'nodes_involved', 'nodes_examined', 'radio_breast_ct_volume_c
 Fold 10: AUC = 0.7941 | Selected Features: ['nodes_involved', 'nodes_examined', 'radio_breast_ct_volume_cm3', 'radio_skin_ma

Mean AUC: 0.8868 across 30 features | Std: 0.0769



	Frequency
nodes_examined	10
radio_breast_ct_volume_cm3	10
rs885170	10
radio_skin_max_dose_Gy	10
radio_heart_mean_dose_Gy	10
radio_ipsilateral_lung_mean_Gy	10
rs77405610	10
breast_right_arm_circumfrence_mm_baseline	10
rs1788750	10
rs67342194	10
rs6009631	10
rs2954007	10
rs10764743	10
rs7195405	10
rs35035095	10
rs10945395	10

```
#feature_set_rfe_CV_inside_pipeline

stable_features_rfe_inside_pipeline = get_frequent_features(
    feature_set_rfe_CV_inside_pipeline,
    freq_threshold=8
)

print(stable_features_rfe_inside_pipeline)
print(len(stable_features_rfe_inside_pipeline))
```

```
['rs2954007', 'radio_skin_max_dose_Gy', 'rs72857489', 'radio_ipsilateral_lung_mean_Gy', 'rs74853756', 'rs2861705', 'rs79223428']
```

```
cross_validated_feature_selection(df_clean_20[stable_features
[dependent1]], dependent1, splits=10)
```

Original Data shape is (2067, 29)

Data shape after dropping missing target: (1691, 29)

✓ Mean AUC with final selected features: 0.8866

Start coding or [generate](#) with AI.

▼ Nested CV

```
summary3, fold_details3, feature_frequency3 = nested_cv_featu
```

Outer CV: 10it [08:21, 50.12s/it]

summary3 #

	mean_auc	std_auc	
RandomForest	0.86942	0.073919	

fold_details3

	fold	model	best_k	inner_auc	outer_auc	selected_features
0	1	RandomForest	30	0.862199	0.941280	[height_cm, age, nodes_involved, nodes_examine...
1	2	RandomForest	30	0.868258	0.971370	[age, nodes_involved, nodes_examined, radio_ph...
2	3	RandomForest	30	0.892593	0.935583	[age, nodes_involved, nodes_examined, radio_br...
3	4	RandomForest	30	0.898088	0.840491	[age, nodes_involved, nodes_examined, radio_br...
4	5	RandomForest	30	0.853752	0.817996	[height_cm, age, nodes_involved, nodes_examine...
5	6	RandomForest	30	0.884813	0.738241	[height_cm, nodes_involved, nodes_examined, tu...
6	7	RandomForest	30	0.864131	0.878323	[age, nodes_involved, nodes_examined, radio_ph...
7	8	RandomForest	30	0.864312	0.797546	[age, nodes_involved, nodes_examined, tumour_s...
8	9	RandomForest	30	0.838008	0.950617	[age, nodes_examined, tumour_size_mm, radio_br...
9	10	RandomForest	30	0.854371	0.822751	[age, nodes_involved, nodes_examined, tumour_s...

Next steps: [Generate code with fold_details3](#) [New interactive sheet](#)

```
display(feature_frequency3.head(100))
```

	feature	count	percentage
0	radio_skin_max_dose_Gy	10	100.0
1	radio_breast_ct_volume_cm3	10	100.0
2	nodes_examined	10	100.0
3	rs61912468	10	100.0
4	rs35410275	10	100.0
5	rs461769	10	100.0
6	radio_heart_mean_dose_Gy	10	100.0
7	breast_right_arm_circumfrence_mm_baseline	10	100.0
8	radio_ipsilateral_lung_mean_Gy	10	100.0
9	rs77405610	10	100.0
10	rs67342194	10	100.0
11	rs2954007	10	100.0
12	rs1788750	10	100.0
13	rs10764743	10	100.0
14	rs7195405	10	100.0
15	rs35035095	10	100.0
16	rs57288348	10	100.0
17	rs2861705	10	100.0
18	rs72977387	10	100.0
19	rs885170	10	100.0
20	rs10945395	10	100.0
21	age	9	90.0
22	nodes_involved	9	90.0
23	rs79223472	9	90.0
24	rs6009631	9	90.0
25	rs56402833	9	90.0
26	rs72857489	8	80.0
27	rs3748569	8	80.0
28	breast_arm_lymphodema_baseline	7	70.0
29	radio_supraclavicular_fossa	5	50.0
30	tumour_size_mm	5	50.0
31	rs633502	4	40.0
32	height_cm	3	30.0
33	radio_photon_dose_MV	3	30.0
34	rs74853756	2	20.0

```
# Features that appeared in 8 folds and more across 10 folds
stable_features3 = feature_frequency3[feature_frequency3["count"] >= 8]
```

```
print(stable_features3)
```

```
['radio_skin_max_dose_Gy', 'radio_breast_ct_volume_cm3', 'nodes_examined', 'rs61912468', 'rs35410275', 'rs461769', 'radio_heart_mean_dose_Gy', 'breast_right_arm_circumfrence_mm_baseline', 'radio_ipsilateral_lung_mean_Gy', 'rs77405610', 'rs67342194', 'rs2954007', 'rs1788750', 'rs10764743', 'rs7195405', 'rs35035095', 'rs57288348', 'rs2861705', 'rs72977387', 'rs885170', 'rs10945395', 'age', 'nodes_involved', 'rs79223472', 'rs6009631', 'rs56402833', 'rs72857489', 'rs3748569', 'breast_arm_lymphodema_baseline', 'radio_supraclavicular_fossa', 'tumour_size_mm', 'rs633502', 'height_cm', 'radio_photon_dose_MV', 'rs74853756']
```

```
print(len(stable_features3))
```

```
28
```

```
jaccard_stability(fold_details3, "RandomForest")
```

```
np.float64(0.8443705145792434)
```

```
cross_validated_feature_selection(df_clean_20[stable_features:
```

```
Original Data shape is (2067, 29)
```

```
Data shape after dropping missing target: (1691, 29)
```

```
✔ Mean AUC with final selected features: 0.8741
```

Feature Selection 4

3.3.5 AutoEncoder feature selection (AE)

Autoencoder is a type of neural network used to compress data (encode) and then reconstruct it (decode). It's unsupervised — you don't need labels to train it. The trick is — the middle layer has fewer neurons (features), so it must learn the most important information to recreate the original. For feature selection method, it is used to extract features with Hidden Patterns. Let's say you're working with:

Medical data with 200+ features (demographics, genomics, imaging, etc.)

Training an autoencoder to reduce this to 10–20 learned features helps:

Find underlying structure

Reduce noise/redundancy

Improve model generalization.

Encoding features only

```
#
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.optimizers import Adam
import keras # Assuming keras is already imported and configured

def auto_encoder_feature_selection(df, col, feat):
    # Drop rows with any missing values as specified in the original data
    df = df.dropna(axis=0, how='any', inplace=False)

    # Separate features (X) and target (Y)
    # Drop target column and 'Subject Id'
    X = df.drop(columns=[col, 'Subject Id'])
    Y = df[col]

    # Explicitly drop all datetime columns before further processing
    # This ensures StandardScaler does not encounter them.
    datetime_cols = X.select_dtypes(include=['datetime64']).columns
    if not datetime_cols.empty:
        X = X.drop(columns=datetime_cols)

    # Convert categorical columns to numeric using one-hot encoding
```

```

X = pd.get_dummies(X, drop_first=True)

# Ensure all remaining columns are numeric before scaling
# This acts as a safeguard, though explicit datetime droppi
specific error.
X = X.select_dtypes(include=np.number)

# Normalize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# -----
# Step 2: Autoencoder Architecture
# -----
input_dim = X_scaled.shape[1]
encoding_dim = 20 # Set number of features you want to red

# Define the model
input_layer = Input(shape=(input_dim,))
#encoded = Dense(encoding_dim, activation='relu')(input_lay
#decoded = Dense(input_dim, activation='sigmoid')(encoded)

encoded = keras.layers.Dense(encoding_dim, activation="relu
decoded = keras.layers.Dense(input_dim, activation="sigmoid

autoencoder = Model(inputs=input_layer, outputs=decoded)
autoencoder.compile(optimizer=Adam(learning_rate=0.001), lo
autoencoder.summary()

# -----
# Step 3: Train the Autoencoder
# -----
# Ensure Y has the same index as X after dropping NaNs and
Y = Y.loc[X.index]
X_train, X_test = train_test_split(X_scaled, test_size=0.2,
stratify=Y)
autoencoder.fit(X_train, X_train,
                epochs=50,
                batch_size=32,
                shuffle=True,
                validation_data=(X_test, X_test),
                verbose=0)

# -----
# Step 4: Extract Encoded Features
#

```

```

encoder = Model(inputs=autoencoder.input, outputs=autoencoder.output)
encoded_features = encoder.predict(X_scaled)

return encoded_features, Y

```

```
encoded_features, Y= auto_encoder_feature_selection(df_clean_
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 97)	0
dense (Dense)	(None, 20)	1,960
dense_1 (Dense)	(None, 97)	2,037

Total params: 3,997 (15.61 KB)

Trainable params: 3,997 (15.61 KB)

Non-trainable params: 0 (0.00 B)

23/23 ————— 0s 4ms/step

```
encoded_features.shape
```

```
(714, 20)
```

▼ Inside the pipeline

```

from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler

def autoencoder_feature_pipeline(df, dependent, encoding_dim):
    df = df.dropna()
    X = df.drop(columns=[dependent]).select_dtypes(include=['float', 'int'])
    y = df[dependent]

    print(f'Preprocess with {encoding_dim} encoding dimension')

    pipeline = Pipeline([
        ('scaler', StandardScaler()),
        ('autoencoder_selector', AutoencoderFeatureSelector(encoding_dim=encoding_dim)),
        ('classifier', RandomForestClassifier(n_estimators=100))
    ])

    skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
    auc_scores = cross_val_score(pipeline, X, y, cv=skf, scoring='roc_auc')

    print(f"Mean AUC with Autoencoder-{encoding_dim} encoding dimension")
    return auc_scores.mean()

```

```
for i in k:
    autoencoder_feature_pipeline(df_clean_20, dependent1, i)
```

```
3/3 ----- 0s 11ms/step
21/21 ----- 0s 10ms/step
3/3 ----- 0s 20ms/step
21/21 ----- 0s 6ms/step
3/3 ----- 0s 24ms/step
21/21 ----- 0s 5ms/step
3/3 ----- 0s 18ms/step
21/21 ----- 0s 11ms/step
3/3 ----- 0s 26ms/step
21/21 ----- 0s 6ms/step
3/3 ----- 0s 12ms/step
21/21 ----- 0s 5ms/step
3/3 ----- 0s 12ms/step
21/21 ----- 0s 5ms/step
3/3 ----- 0s 24ms/step
Mean AUC with Autoencoder-selected features: 0.78
21/21 ----- 0s 7ms/step
3/3 ----- 0s 20ms/step
21/21 ----- 0s 4ms/step
3/3 ----- 0s 14ms/step
21/21 ----- 0s 3ms/step
3/3 ----- 0s 12ms/step
21/21 ----- 0s 4ms/step
3/3 ----- 0s 13ms/step
21/21 ----- 0s 4ms/step
3/3 ----- 0s 12ms/step
21/21 ----- 0s 3ms/step
3/3 ----- 0s 12ms/step
21/21 ----- 0s 4ms/step
3/3 ----- 0s 12ms/step
21/21 ----- 0s 4ms/step
3/3 ----- 0s 11ms/step
21/21 ----- 0s 3ms/step
3/3 ----- 0s 11ms/step
21/21 ----- 0s 4ms/step
3/3 ----- 0s 12ms/step
Mean AUC with Autoencoder-selected features: 0.83
21/21 ----- 0s 3ms/step
3/3 ----- 0s 13ms/step
21/21 ----- 0s 3ms/step
3/3 ----- 0s 11ms/step
21/21 ----- 0s 3ms/step
3/3 ----- 0s 12ms/step
21/21 ----- 0s 3ms/step
3/3 ----- 0s 12ms/step
21/21 ----- 0s 4ms/step
3/3 ----- 0s 11ms/step
21/21 ----- 0s 4ms/step
3/3 ----- 0s 12ms/step
21/21 ----- 0s 3ms/step
3/3 ----- 0s 13ms/step
21/21 ----- 0s 3ms/step
3/3 ----- 0s 12ms/step
21/21 ----- 0s 3ms/step
3/3 ----- 0s 12ms/step
21/21 ----- 0s 4ms/step
3/3 ----- 0s 13ms/step
Mean AUC with Autoencoder-selected features: 0.80
```

▼ Nested CV

```
from sklearn.base import BaseEstimator, TransformerMixin
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras import regularizers
from tensorflow.keras.optimizers import Adam
import numpy as np
```

```
class AutoencoderFeatureSelectorNew(BaseEstimator, TransformerMixin):
    def __init__(self, encoding_dim=10, epochs=30, batch_size=
```



```

self.encoding_dim = encoding_dim
self.epochs = epochs
self.batch_size = batch_size
self.verbose = verbose

def fit(self, X, y=None):
    input_dim = X.shape[1]

    input_layer = Input(shape=(input_dim,))

    encoded = Dense(
        self.encoding_dim,
        activation="relu",
        activity_regularizer=regularizers.l1(1e-5)
    )(input_layer)

    decoded = Dense(input_dim, activation="linear")(encoded)

    self.autoencoder = Model(inputs=input_layer, outputs=decoded)
    self.encoder = Model(inputs=input_layer, outputs=encoded)

    self.autoencoder.compile(
        optimizer=Adam(learning_rate=0.001),
        loss="mse"
    )

    self.autoencoder.fit(
        X,
        X,
        epochs=self.epochs,
        batch_size=self.batch_size,
        shuffle=True,
        verbose=self.verbose
    )

    return self

def transform(self, X):

```

```

from sklearn.base import BaseEstimator, TransformerMixin
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras import regularizers
from tensorflow.keras.optimizers import Adam
import numpy as np
from sklearn.impute import SimpleImputer
def nested_cv_autoencoder(df, target, encoding_dims,

```

```

        outer_splits=10, inner_splits=3):

df = df.dropna(subset=[target])

X = df.drop(columns=[target]).select_dtypes(include=["num
y = df[target]

outer_cv = StratifiedKFold(
    n_splits=outer_splits,
    shuffle=True,
    random_state=0
)

inner_cv = StratifiedKFold(
    n_splits=inner_splits,
    shuffle=True,
    random_state=1
)

results = []

for fold_id, (train_idx, test_idx) in enumerate(outer_cv.

    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    best_dim = None
    best_score = -np.inf

    for dim in encoding_dims:
        inner_scores = []
        for inner_train_idx, inner_val_idx in inner_cv.sp

            X_tr = X_train.iloc[inner_train_idx]
            X_val = X_train.iloc[inner_val_idx]
            y_tr = y_train.iloc[inner_train_idx]
            y_val = y_train.iloc[inner_val_idx]

            pipeline = Pipeline([
                ("imputer", SimpleImputer(strategy="media
                ("scaler", MinMaxScaler()),
                ("autoencoder", AutoencoderFeatureSelecto
                    encoding_dim=dim,
                    epochs=30,
                    batch_size=32,
                    verbose=0
            )),

```

```
        ("model", RandomForestClassifier(
            n_estimators=100,
            max_depth=10,
            random_state=5,
            n_jobs=-1
        ))
    ])

    pipeline.fit(X_tr, y_tr)

    y_pred = pipeline.predict_proba(X_val)[: , 1]
    inner_scores.append(roc_auc_score(y_val, y_pr

mean_score = np.mean(inner_scores)

if mean_score > best_score:
    best_score = mean_score
    best_dim = dim

final_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", MinMaxScaler()),
    ("autoencoder", AutoencoderFeatureSelectorNew(
        encoding_dim=best_dim,
        epochs=30,
        batch_size=32,
        verbose=0
    )),
    ("model", RandomForestClassifier(
        n_estimators=100,
        max_depth=10,
        random_state=5,
        n_jobs=-1
    ))
])

final_pipeline.fit(X_train, y_train)

y_pred = final_pipeline.predict_proba(X_test)[: , 1]
outer_auc = roc_auc_score(y_test, y_pred)

results.append({
    "fold": fold_id,
    "best_encoding_dim": best_dim,
    "inner_auc": best_score,
    "outer_auc": outer_auc
})
```

```
results = pd.DataFrame(results)

print(results)
print("Mean AUC:", results["outer_auc"].mean())
print("STD AUC:", results["outer_auc"].std())
```

```
ae_results = nested_cv_autoencoder(df_clean_20, dependent1,
                                   encoding_dims=[10, 15, 20,
                                   outer_splits=10, inner_splits=10])
```

	fold	best_encoding_dim	inner_auc	outer_auc
0	1	30	0.798102	0.778265
1	2	30	0.780929	0.962168
2	3	30	0.780944	0.742331
3	4	30	0.768399	0.742331
4	5	30	0.752948	0.871166
5	6	30	0.822396	0.779141
6	7	20	0.801280	0.851738
7	8	30	0.779191	0.730061
8	9	30	0.779743	0.907407
9	10	15	0.782365	0.841270

Mean AUC: 0.8205878534099392

STD AUC: 0.07852175982361587

4) Feature engineering and evaluation:

Create new features or transform existing ones to improve the model's performance.

SNPs Data

```
lymphedema_SNP= ['rs77405610', 'rs67342194', 'rs2954011', 'rs:
```

this function to print the summary for all features against

```
def summarize_by_hue(df, hue_col, feature_cols):
    summary_tables = {}
    counts_dict = {}
    percentages_dict = {}

    for col in feature_cols:
        # Cross-tabulate counts
        counts = pd.crosstab(df[col], df[hue_col])

        # Calculate percentages
        percentages = counts.div(counts.sum(axis=1), axis=0)

        # Store each feature's count and percent table
        counts_dict[col] = counts
        percentages_dict[col] = percentages.round(1)
        # Combine into one table
        combined = counts.astype(int).astype(str) + " (" + pe
        summary_tables[col] = combined

    return summary_tables, counts_dict, percentages_dict
```

```
# if I want to check the summary of features with dependent
summary, counts, percentages = summarize_by_hue(reduced_df, d
```

```
#percentages['smoker']
#counts['smoker']
#summary['smoker']
```

```
for feature, table in summary.items():
    print(f"Summary for {feature}:")
    print(table)
    print("\n")
```

```
def cleanData(df, features, dependent):
    df_feat= df[features]
    print(df_feat.shape)
    print(df[dependent].sort_values().value_counts())
    clean_data = df_feat.dropna(axis=0, how='any', inplace=False)
    print(clean_data.shape)
    df = clean_data
    print(df[dependent].sort_values().value_counts())
    ***normalization
    x_data = df.drop([dependent], axis=1)
    #X = (x_data - np.min(x_data)) / (np.max(x_data) - np.min(x_data))
    X = (x_data - np.min(x_data)) / (np.max(x_data) - np.min(x_data))
    #X = df.drop(['arm_lymphodema_24'], axis=1) # without normalization
    y = df[dependent]
    return df, X, y
```

```
reduced_df = reduced_df.replace(dict(breast_arm_lymphodema_60=0))
```

```
# call the functioning to Cleaning
#df, X, y= cleanData(reduced_df, lymphedema_M2_SNP, dependent1)
df, X, y= cleanData(reduced_df, lymphedema_M1, dependent1) #
#df, X, y= cleanData(reduced_df, lymphedema_SNP, dependent1) #
#df, X, y= cleanData(reduced_df, lit_snp1, dependent1)
```

```
# call the functioning to Cleaning
df_combined, X_combined, y= cleanData(reduced_df, lymphedema_M1, dependent1)
df_clinic, X_clinic, y= cleanData(reduced_df, lymphedema_M1, dependent1)
df_snps, X_snps, y= cleanData(reduced_df, lymphedema_SNP, dependent1)
```

Start coding or [generate](#) with AI.

```
# Step 1: find intersection of all patient IDs
common_ids = df_clinic.index.intersection(df_snps.index).intersection(df_snp2.index)

# Step 2: subset all X datasets to common IDs
X_clinic_aligned = df_clinic.loc[common_ids]
X_snps_aligned = df_snps.loc[common_ids]
```

```
X_combined_aligned = df_combined.loc[common_ids]

# Step 3: subset y to the same patients
y_aligned = y.loc[common_ids] # if y is a Series
# or
# y_aligned = y.values[common_indices] # if y is a numpy arr
```

```
print(X_clinic_aligned.shape)
print(X_snps_aligned.shape)
print(X_combined_aligned.shape)
```

```
print(X_clinic.shape)
print(X_snps.shape)
print(X_combined.shape)
```

```
# This is the extended one that covers all metrics AUC, PR, F
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.metrics import (roc_auc_score, average_precision_score)
from sklearn.base import clone
```

```
def evaluate_feature_sets_extended(X_clinic, X_snps, X_combined, y):
    # Step 1: Align datasets by common index
    common_ids = X_clinic.index.intersection(X_snps.index).intersection(X_combined.index)
    X_clinic = X_clinic.loc[common_ids]
    X_snps = X_snps.loc[common_ids]
    X_combined = X_combined.loc[common_ids]
    y = y.loc[common_ids]
```

```
# Step 2: Setup CV
cv = RepeatedStratifiedKFold(n_splits=n_splits, n_repeats=n_repeats)
```

```
# Step 3: Prepare storage for metrics
metrics = ["auc", "auprc", "f1", "balanced_accuracy"]
fold_metrics = {
    "clinic": {m: [] for m in metrics},
    "snps": {m: [] for m in metrics},
    "combined": {m: [] for m in metrics}
}
```

```
# Step 4: Run CV
for train_idx, test_idx in cv.split(X_clinic, y):
    Xtrain_c, Xtest_c = X_clinic.iloc[train_idx], X_clinic.iloc[test_idx]
```



```

Xtrain_s, Xtest_s = X_snps.iloc[train_idx], X_snps.i
Xtrain_m, Xtest_m = X_combined.iloc[train_idx], X_con
ytrain, ytest = y.iloc[train_idx], y.iloc[test_idx]

for Xtrain, Xtest, name in zip(
    [Xtrain_c, Xtrain_s, Xtrain_m],
    [Xtest_c, Xtest_s, Xtest_m],
    ["clinic", "snps", "combined"]
):
    clf = clone(model)
    clf.fit(Xtrain, ytrain)

    if hasattr(clf, "predict_proba"):
        y_score = clf.predict_proba(Xtest)[:,-1]
    else:
        y_score = clf.decision_function(Xtest)

    y_pred = (y_score >= 0.5).astype(int)

    fold_metrics[name]["auc"].append(roc_auc_score(yt
    fold_metrics[name]["auprc"].append(average_precis
    fold_metrics[name]["f1"].append(f1_score(ytest, y
    fold_metrics[name]["balanced_accuracy"].append(ba

# Step 5: Summarize metrics
def summarize(arr):
    arr = np.array(arr)
    mean = np.mean(arr)
    sd = np.std(arr, ddof=1)
    ci_low = mean - 1.96 * sd / np.sqrt(len(arr))
    ci_up = mean + 1.96 * sd / np.sqrt(len(arr))
    return mean, ci_low, ci_up

def cohens_d(x, y):
    diff = np.array(x) - np.array(y)
    return np.mean(diff) / np.std(diff, ddof=1)

# Build results table
results = {}
for metric in metrics:
    for feature in ["clinic", "snps", "combined"]:
        mean, ci_low, ci_up = summarize(fold_metrics[feat
        results[f"{feature}_{metric}_mean"] = float(round(
        # Convert CI to plain Python floats
        results[f"{feature}_{metric}_CI"] = [float(round(

# Step 6: Paired statistical tests vs combined

```

```

for metric in metrics:
    _, p_c = stats.ttest_rel(fold_metrics["combined"][metric], fold_metrics["combined"][metric])
    _, p_s = stats.ttest_rel(fold_metrics["combined"][metric], fold_metrics["combined"][metric])
    d_c = cohens_d(fold_metrics["combined"][metric], fold_metrics["combined"][metric])
    d_s = cohens_d(fold_metrics["combined"][metric], fold_metrics["combined"][metric])
    results[f"p_clinic_vs_combined_{metric}"] = float(round(p_c, 4))
    results[f"cohen_d_clinic_vs_combined_{metric}"] = float(round(d_c, 4))
    results[f"p_snps_vs_combined_{metric}"] = float(round(p_s, 4))
    results[f"cohen_d_snps_vs_combined_{metric}"] = float(round(d_s, 4))

return pd.DataFrame(results)

```

recall the extended function that cover all metrics

```

models = {
    "Random Forest": RandomForestClassifier(
        n_estimators=100, max_depth=10, min_samples_split=5,
        min_samples_leaf=1, bootstrap=True, random_state=5
    ),
    "Extra Trees": ExtraTreesClassifier(
        n_estimators=100, max_depth=None, min_samples_split=4,
        bootstrap=False, random_state=5
    ),
    "Gradient Boosting": GradientBoostingClassifier(
        n_estimators=50, learning_rate=0.1, max_depth=3, random_state=5
    ),
    "AdaBoost": AdaBoostClassifier(n_estimators=50, random_state=5),
    "XGBoost": XGBClassifier(
        random_state=5, use_label_encoder=False, eval_metric='logloss'
    ),
    "Decision Tree": DecisionTreeClassifier(
        max_depth=5, min_samples_split=10, min_samples_leaf=5,
        criterion="entropy", random_state=5, class_weight="balanced"
    ),
    "QDA": make_pipeline(StandardScaler(), QuadraticDiscriminantAnalysis()),
    "Naive Bayes": make_pipeline(StandardScaler(), GaussianNB()),
    "SVM (RBF)": make_pipeline(StandardScaler(), SVC(kernel='rbf'))
}

```

```

"KNN": make_pipeline(StandardScaler(), KNeighborsClassifi
"Logistic Regression": make_pipeline(StandardScaler(), Lc
"MLP": make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(100,), activation='
),
}

# ♦ Run all models and collect result
rows = []
for name, model in models.items():
    results, fold_aucs = evaluate_feature_sets_extended(
        X_clinic, X_snps, X_combined, y, model
    )
    row = {"Model": name}
    # Convert the first row of the results DataFrame to a dic
    row.update(results.iloc[0].to_dict())
    rows.append(row)

# ♦ Store in DataFrame
results_df = pd.DataFrame(rows).set_index("Model")

# Show results
print(results_df)

# Save for later use
results_df.to_csv("armlymphedema_all_metrics_results.csv", ir

```

```

import matplotlib.pyplot as plt

def plot_metrics_and_cohens_d_separate(fold_metrics, results_
    metrics = ["auc", "auprc", "f1", "balanced_accuracy"]

    # --- Figure 1: Boxplots ---
    plt.figure(figsize=(10,6))
    data = []
    labels = []
    for metric in metrics:
        for feature in ["clinic", "snps", "combined"]:
            data.append(fold_metrics[feature][metric])
            labels.append(f"{feature}_{metric}")

    plt.boxplot(data, labels=labels, patch_artist=True)

```

```

plt.title(f"Fold-level distributions ({model_name})")
plt.xticks(rotation=45, ha="right")
plt.ylabel("Score")
plt.tight_layout()
plt.show()

# --- Figure 2: Cohen's d barplot ---
plt.figure(figsize=(8,6))
cohen_d_cols = [c for c in results_df.columns if "cohen_c"
cohen_d_vals = results_df.iloc[0][cohen_d_cols].values
clean_labels = [c.replace("cohen_d_", "").replace("_vs_",
               for c in cohen_d_cols]

plt.barh(clean_labels, cohen_d_vals, color="skyblue")
plt.axvline(0, color="black", linewidth=1)
plt.title("Cohen's d Effect Sizes")
plt.xlabel("Cohen's d")
plt.tight_layout()
plt.show()

```

```

results, folds = evaluate_feature_sets_extended(X_clinic, X_si
plot_metrics_and_cohens_d_separate(folds, results, model_name:

```

```

def plot_auc_cohens_d(fold_metrics, group1="clinic", group2='
    from scipy.stats import norm

    # Extract fold-level AUCs
    auc1 = np.array(fold_metrics[group1]["auc"])
    auc2 = np.array(fold_metrics[group2]["auc"])

    # Means and pooled std
    mean1, mean2 = auc1.mean(), auc2.mean()
    pooled_std = np.sqrt(((auc1.std(ddof=1) ** 2) + (auc2.stc
    d = (mean2 - mean1) / pooled_std

    # x-axis range
    x = np.linspace(min(mean1, mean2) - 3*pooled_std,
                    max(mean1, mean2) + 3*pooled_std, 500)

    # PDFs
    y1 = norm.pdf(x, mean1, pooled_std)
    y2 = norm.pdf(x, mean2, pooled_std)

```

```

# Plot
plt.figure(figsize=(8,5))
plt.plot(x, y1, label=f"{group1} mean = {mean1:.3f}", col
plt.plot(x, y2, label=f"{group2} mean = {mean2:.3f}", col

plt.title(f"Cohen's d = {d:.2f} (AUC)\nPooled Std = {pool
plt.xlabel("Mean AUC")
plt.ylabel("Density")
plt.legend(loc= 'upper right')
plt.show()

return d

```

```

results, folds = evaluate_feature_sets_extended(X_clinic, X_s

# Example: compare combined vs clinic AUC
d_value = plot_auc_cohens_d(folds, group1="clinic", group2="c
print("Cohen's d:", d_value)

```

```

from tqdm import tqdm
def repeated_auc_pr_roc_models(X, y, kfolds):
    plt.figure(figsize=(8, 8), dpi=300)

    # Define models (with pipeline for those needing scaling)
    MLmodels = [
        RandomForestClassifier(n_estimators=100, max_depth=10,
                               min_samples_leaf=1, bootstrap=
        ExtraTreesClassifier(n_estimators=100, max_depth=None,
                              bootstrap=False, random_state=5)
        GradientBoostingClassifier(n_estimators=50, learning_
                                   random_state=5),
        AdaBoostClassifier(n_estimators=50, random_state=5),

        XGBClassifier(random_state=5, use_label_encoder=False

        DecisionTreeClassifier(max_depth=5, min_samples_split
                                criterion="entropy", random_st
        make_pipeline(StandardScaler(), QuadraticDiscriminant

        make_pipeline(StandardScaler(), GaussianNB()),

        make_pipeline(StandardScaler(), SVC(kernel='rbf', prc

```

```

make_pipeline(StandardScaler(), KNeighborsClassifier(

make_pipeline(StandardScaler(), LogisticRegression(r

make_pipeline(StandardScaler(), MLPClassifier(hidden_
solver=

]

colors = cm.get_cmap('tab20', len(MLmodels))
cv = RepeatedStratifiedKFold(n_splits=kfolds, n_repeats=5
#cv = StratifiedKFold(n_splits=kfolds, shuffle=True, rand

results = []
results2 = []
for i, classifier in enumerate(tqdm(MLmodels, desc="Evalu
    y_real = []
    y_proba = []
    aucs = []
    pr_aucs = []

    for train_index, test_index in cv.split(X, y):
        Xtrain, Xtest = X.iloc[train_index], X.iloc[test_
        ytrain, ytest = y.iloc[train_index], y.iloc[test_

        classifier.fit(Xtrain, ytrain)
        y_score = (classifier.predict_proba(Xtest)[: , 1]
                    if hasattr(classifier, 'predict_proba'
                    else classifier.decision_function(Xtes

        # ROC
        fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
        aucs.append(auc(fpr_fold, tpr_fold))

        # PR
        pr_aucs.append(average_precision_score(ytest, y_s

        y_real.append(ytest)
        y_proba.append(y_score)

    # concatenate predictions for plotting only
    y_real = np.concatenate(y_real)
    y_proba = np.concatenate(y_proba)

    # Mean metrics across folds
    auc_mean = np.mean(aucs)
    std_auc = np.std(aucs, ddof=1)

```

```

auc_pr = np.mean(pr_aucs)

# Build curves from pooled predictions (for plotting
fpr, tpr, _ = roc_curve(y_real, y_proba)
precision, recall, _ = precision_recall_curve(y_real,

# Model name (handle pipelines)
model_name = (type(classifier.named_steps[list(classi
                if hasattr(classifier, 'named_steps') )

results.append({
    'classifier': model_name,
    'fpr': fpr,
    'tpr': tpr,
    'auc_mean': auc_mean,
    'std_auc': std_auc,
    'precision': precision,
    'recall': recall,
    'auc_pr': auc_pr
})

# Sort by ROC-AUC
results = sorted(results, key=lambda x: x['auc_mean'], re
# Sort by PR-AUC
results2 = sorted(results, key=lambda x: x['auc_pr'], rev

# ROC Plot
for i, result in enumerate(results):
    plt.plot(result['fpr'], result['tpr'], lw=2,
             label=f'{result["classifier"]} (ROC AUC = {r
             color=colors(i))
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', lw
plt.xlabel('False Positive Rate', fontsize=14)
plt.ylabel('True Positive Rate', fontsize=14)
plt.title(f'(c) ROC Curves of repeated {kfolds}-Fold CV (
        fontsize=16, weight='bold', pad=15)
plt.grid(True, linestyle='--', linewidth=0.5, color='ligh
plt.legend(loc='lower right', fontsize=12, frameon=True,
plt.tight_layout()
plt.show()

# prevalence = positive class proportion
prevalence = np.mean(y)

# PR Plot
plt.figure(figsize=(8, 8), dpi=300)
for i, result in enumerate(results2):

```

```

plt.plot(result['recall'], result['precision'], lw=2,
         label=f'{result["classifier"]} (AUC-PR = {re
         color=colors(i))
# plot PR baseline
plt.plot([0, 1], [prevalence, prevalence], linestyle='--'
         lw=1.5, label=f'Chance Level = {prevalence:.2f}')
plt.xlabel('Recall', fontsize=14)
plt.ylabel('Precision', fontsize=14)
plt.title(f'(c) Precision-Recall Curves of repeated {kfol
         fontsize=16, weight='bold', pad=15)
plt.grid(True, linestyle='--', linewidth=0.5, color='ligh
plt.legend(loc='upper right', fontsize=12, frameon=True,
plt.tight_layout()
plt.show()

```

```
repeated_auc_pr_roc_models(X, y, 10)
```

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
```

```

plt.figure(figsize=(6, 5))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='bwr', edgeco
plt.title("PCA Projection (2D)")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.grid(True)
plt.tight_layout()
plt.show()

```

```
# Try t-SNE or UMAP: These methods can better capture complex
```

```
from sklearn.manifold import TSNE
```

```

tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)

```



```
plt.figure(figsize=(6, 5))
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=y, cmap='bwr', edge
plt.title("t-SNE Projection (2D)")
plt.xlabel("Dim 1")
plt.ylabel("Dim 2")
plt.grid(True)
plt.tight_layout()
plt.show()
```

5) Model Building and algorithms selection:

Choose the appropriate algorithm(s) to use for your problem. Consider factors such as the nature of the data, the size of the dataset, and the interpretability of the model

Normal statistic Method

3.5.1.1) Building x and y with Ordinary Least Square method (statsmodels)

3.5.1.2) With chi square contingency

```
# to check the data variables in case you are seek for specif
#df.head(5)
```

The chi-square test statistic is 10.5. The p-value is 0.0012. The degrees of freedom are 3. This information is useful for determining whether there is a statistically significant association between the two categorical variables represented by dependent and x.

```
def chi_contingency(df, dependent, categorical_col):
    for x in categorical_col:
        # For Imbalanced Data to see how many value of y feature
        chisqt = pd.crosstab(df[dependent], df[x], margins=True
        value = np.array([chisqt.iloc[0].values,
                          chisqt.iloc[1].values])
        print(dependent, 'and', x ,chi2_contingency(value)[0:3])
```

```
#chi_contingency(df, dependent1, features)
```

From above for surgery_axilla_type , 1.3579100832798462e-27 is the p-value, 132.13916260784183 is the statistical value and 4 is the degree of freedom. As the p-value is less than 0.05, we reject the NULL hypothesis and assume that the variables arm_lymphodema_24 and surgery_axilla_type are dependent of each other.

3.6.1.3) T test Method

```
# T test result
def t_test(df, dependent, features):
    for x in features:
        print(dependent, 'and', x , stats.ttest_rel(df[dependent]
```

```
#t_test(df, dependent1, features)
```

3.6.1.4) find correlation using ANOVA Test

```
# firstly we need to change the column name
#df = df.rename(columns={'arm_lymphodema_24_1.0': 'arm_lympho
```

```
# anova test
def anova_test(df, dependent, features):
    for x in features:
        # we can do ANOVA test between two var y and X or as
        model = ols(dependent + '~' + x, data = df).fit() #
        result_anova = sm.stats.anova_lm(model, type=2) # ANO
        print(result_anova)
```

```
#anova_test(df, dependent1, features)
```

Start coding or [generate](#) with AI.

```
def logitstat_model(X, y):
    # Without Standardize the features
    train_x, test_x, train_y, test_y = train_test_split(X, y,
    # Logistic Regression using Statsmodels
    train_x['intercept'] = 1.0
    logit = sm.Logit(train_y, train_x).fit()
    summaryStat= logit.summary()
    print(summaryStat)
    #####
    # Caculate Test Error
    test_x['intercept'] = 1.0
    pred = logit.predict(test_x)
    pred[pred > 0.5] = 1
    pred[pred <= 0.5] = 0
    test_err = 1 - (test_y == pred).mean()
    print("test error rate: " + str(test_err))
    #####
    # performing predictions on the test dataset
    yhat = logit.predict(test_x)
    prediction = list(map(round, yhat))

    # comparing original and predicted values of y
    print('Actual values', list(test_y.values))
    print('Predictions :', prediction)

    # confusion matrix
    cm = confusion_matrix(test_y, prediction)
```

```
print ("Confusion Matrix : \n", cm)
```

```
# accuracy score of the model
```

```
logitstat_model(X, y)
```

Start coding or [generate](#) with AI.

```
# Plotting correlation with each features against arm_lymph
def logreg(x1, dependent):
    X= df[x1].values
    y= df[dependent]
    # Splitting the dataset into the Training set and Test set
    X_train, X_test, y_train, y_test = train_test_split(X, y,
    X_train= X_train.reshape(-1,1)
    X_test= X_test.reshape(-1,1)
    # fitting linear regression to the training set
    regressor = LogisticRegression()
    #regressor = LinearRegression()
    regressor.fit(X_train, y_train)
    # predicting the test set result
    y_pred = regressor.predict(X_test)
    # show the accuracy values of the predictive model
    train_score=regressor.score(X_train, y_train)
    test_score=regressor.score(X_test, y_test)
    print("Training score: {}".format(train_score))
    print("Test score: {}".format(test_score))

#coef and intercept are coefficients and intercepts resp
print("Coefficient:\n", regressor.coef_)
print("Intercept:\n", regressor.intercept_)
# Mean squared error
print("Mean squared error: %.2f" % mean_squared_error(y_t
# Variance score (1 for the perfect prediction)
print("Variance score: %2f" % r2_score(y_test, y_pred))
# visualizing the training set result
plt.scatter(X_train, y_train, color = 'red')
plt.plot(X_train, regressor.predict(X_train), color = 'b')
plt.title('(Training set)')
plt.xlabel(x1)
plt.ylabel('arm_lymphodema_24 grade1')
plt.show()
# visualizing the test set result
plt.scatter(X_test, y_test, color = 'purple')
plt.plot(X_train, regressor.predict(X_train), color = 'b')
plt.title('(Test set)')
```

```
plt.xlabel(x1 )
plt.ylabel('arm_lymphodema_24 grade1')
plt.show()
```

```
for f in lit_snp:
    logreg(f, dependent1)
```

✓ Building Machine Learning Models:

✓ Model training:

Logistic regression one to one model one to one relationship

```
# Plotting correlation with each feature against the depender
def logreg(x1, dependent):
    X = df[x1].values
    y = df[dependent]

    # Splitting the dataset into the Training set and Test set
    X_train, X_test, y_train, y_test = train_test_split(X, y,

    # Reshaping the training and test data to be 2D arrays
    X_train = X_train.reshape(-1, 1)
    X_test = X_test.reshape(-1, 1)

    # Fitting logistic regression to the training set
    regressor = LogisticRegression()
    regressor.fit(X_train, y_train)

    # Predicting the test set results
    y_pred = regressor.predict(X_test)

    # Showing the accuracy values of the predictive model
    train_score = regressor.score(X_train, y_train)
    test_score = regressor.score(X_test, y_test)
    print("Training score: {}".format(train_score))
    print("Test score: {}".format(test_score))

    # Coefficients and intercept for the model
    print("Coefficient:\n", regressor.coef_)
```

```

print("Intercept:\n", regressor.intercept_)

# Confusion matrix and classification report
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

# Visualizing the training set results
plt.scatter(X_train, y_train, color='red')
plt.plot(X_train, regressor.predict(X_train), color='blue')
plt.title('Training set')
plt.xlabel(x1)
plt.ylabel(dependent)
plt.show()

# Visualizing the test set results
plt.scatter(X_test, y_test, color='purple')
plt.plot(X_train, regressor.predict(X_train), color='blue')
plt.title('Test set')
plt.xlabel(x1)
plt.ylabel(dependent)
plt.show()

```

```

for f in features:
    logreg(f, dependent1)

```

```

def calculate_rates(confusion_matrix):
    TN, FP, FN, TP = confusion_matrix.ravel()
    tpr = TP / (TP + FN)
    fpr = FP / (FP + TN)
    tnr = TN / (TN + FP)
    return tpr, fpr, tnr, FN / (FN + TP)

```

```

def calculate_tp_fp_tn_fn(confusion_matrix):
    TP = confusion_matrix[1][1]
    FP = confusion_matrix[0][1]
    TN = confusion_matrix[0][0]
    FN = confusion_matrix[1][0]
    return TP, FP, TN, FN

```

BUILDING WITH NORMAL TRAIN AND TEST

```

# this is for the features to plot coefficients in logistic

def models(clf, X, y, title= 'AUC'):
    # split the data
    X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

```

# Fit the model on traing set
y_pred = clf.fit(X_train, Y_train).predict(X_test) #
x_pred= clf.fit(X_train, Y_train).predict(X_train) #
# print model name
print(bold("% s model" % (title)))
# show the accuracy values of the predictive model
print(bold("Training accuracy score:"), (metrics.accu
print(bold("Test accuracy score:"), (metrics.accuracy
#print(bold("Test specificity:"), (metrics.recall_scc
# calculate cross validation score
scores = cross_val_score(clf, X, y, scoring='accuracy
print(bold("Cross validation mean score:"), (scores.n

# Plotting the coefficient
print(bold("\nCoefficients:\n")) # Cc
for feat, coef in zip(features, clf.coef_[0]): # Tr
    print(" {:>20}: {}".format(feat, coef))

#Generate the confusion matrix
cf_matrix = confusion_matrix(Y_test, y_pred)
print(bold("\nconfusion matrix:\n"), cf_matrix)

# To Claculate the rates
tpr, fpr, tnr, fnr = calculate_rates(cf_matrix)
print("True Positive Rate (TPR):", tpr)
print("False Positive Rate (FPR):", fpr)
print("True Negative Rate (TNR):", tnr)
print("False Negative Rate (FNR):", fnr)

TP, FP, TN, FN = calculate_tp_fp_tn_fn(cf_matrix)
print("True Positives (TP):", TP)
print("False Positives (FP):", FP)
print("True Negatives (TN):", TN)
print("False Negatives (FN):", FN)
# Calculate sensitivity and specificity
sensitivity = TP / (TP + FN) * 100
specificity = TN / (TN + FP) * 100

print("Sensitivity: %.2f%%" % sensitivity)
print("Specificity: %.2f%%" % specificity)
ax = sns.heatmap(cf_matrix/np.sum(cf_matrix), annot=1
                fmt='.2%', cmap='Blues')
ax.set_xlabel('\nPredicted Values')
ax.set_ylabel('Actual Values ');
## Ticket labels - List must be in alphabetical order
ax.xaxis.set_ticklabels(['0','1'])
ax.yaxis.set_ticklabels(['0','1'])

```

```
plt.show() # Display the visualization of the Confusion matrix
# plotting AUC
prob = np.array(clf.predict_proba(X_test)[:, 1])
Y_test+= 1
fpr, sensitivity, _ = metrics.roc_curve(Y_test, prob,
print(bold("\nAUC = {}".format(metrics.auc(fpr, sensi
plt.scatter(fpr, fpr, c='b', marker='s')
plt.scatter(fpr, sensitivity, c='r', marker='o')
plt.show() # Display the visualization of the AUC.
```

```
models(LogisticRegression(random_state=10),X, y, title= 'Logi
```

```
models(LogisticRegression(random_state=10),X, y, title= 'Logi
'''models(RandomForestClassifier(n_estimators=50, random_state=10),X, y, title= 'RF')
models(DecisionTreeClassifier(random_state=10),X, y, title= 'DT')
models(KNN(n_neighbors = best_k),X, y, title= 'KNN')
models(SVC(probability= True),X, y, title= 'SVC')
models(XGBClassifier(random_state=10),X, y, title= 'XGB')
models(ExtraTreesClassifier(random_state=10),X, y, title= 'ETC')
models(MLPClassifier(), X, y, title= 'MLPC')
models(QuadraticDiscriminantAnalysis(),X, y, title= 'QDA')
models(GradientBoostingClassifier(n_estimators=50, learning_rate=0.1),X, y, title= 'GBM')
models(GaussianNB(),X, y, title= 'NB') # highest tpr
'''
```

Implement the ROC curve

```
def ROC_Curve(X_train, X_test, y_train, y_test, s):
    plt.figure(figsize=(10, 8), dpi=150)

    # Train multiple models
    MLmodels = [
        SVC(kernel='rbf', probability=True, random_state=10),
        MLPClassifier(hidden_layer_sizes=(100,), activation='tanh'),
        LogisticRegression(random_state=10),
        GaussianNB(),
        ExtraTreesClassifier(random_state=10),
        RandomForestClassifier(random_state=10),
        AdaBoostClassifier(n_estimators=50, random_state=10),
        XGBClassifier(random_state=10),
        GradientBoostingClassifier(n_estimators=50, learning_rate=0.1),
        DecisionTreeClassifier(random_state=10),
        KNeighborsClassifier(n_neighbors=3),
```

```
        QuadraticDiscriminantAnalysis(),
    ]

    # Store results for sorting
    results = []

    # Train and evaluate each model
    for model in MLmodels:
        model.fit(X_train, y_train)

        # Predict probabilities or decision function
        if hasattr(model, 'predict_proba'):
            y_pred_proba = model.predict_proba(X_test)[: , 1]
        else:
            y_pred_proba = model.decision_function(X_test)

        # Calculate ROC curve and AUC
        fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
        auc = roc_auc_score(y_test, y_pred_proba)

        # Calculate g-mean for each threshold
        gmeans = sqrt(tpr * (1 - fpr))
        ix = argmax(gmeans)

        # Store the results for later plotting
        results.append({
            'model_name': type(model).__name__,
            'fpr': fpr,
            'tpr': tpr,
            'auc': auc,
            'best_threshold': thresholds[ix],
            'best_fpr': fpr[ix],
            'best_tpr': tpr[ix],
        })

    # Sort results by AUC (ascending)
    results = sorted(results, key=lambda x: x['auc'])

    # Plot ROC curves in sorted order
    for res in results:
        plt.plot(res['fpr'], res['tpr'], label=f"{res['model_

        # Mark the best threshold
        plt.scatter(res['best_fpr'], res['best_tpr'], marker=
        plt.text(res['best_fpr'], res['best_tpr'], f'{res["be

    # Plot the diagonal line (random chance level)
```



```
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')

# Set plot labels and title
plt.xlabel('False Positive Rate -->')
plt.ylabel('True Positive Rate -->')
plt.title(f'Receiver Operating Characteristic (ROC) Curve')

# Set grid and background color
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, linestyle='-', linewidth=1, color='white')

# Display legend with optimized placement
plt.legend(fontsize='small', loc='lower right', frameon=True)

# Show the plot
plt.show()
```

```
ROC_Curve(X_train, X_test, y_train, y_test, 'normal training data')
```

```
# with auto encoded
ROC_Curve(encoded_Xtrain, encoded_Xtest, ytrain, ytest, 'Encoded')
```

```
# Example classifier
clf = RandomForestClassifier(random_state=42)

# Apply 5-fold cross-validation
scores = cross_val_score(clf, encoded_Xtrain, ytrain, cv=5, scoring='roc_auc')

print("Cross-validated AUC scores:", scores)
print("Mean AUC:", np.mean(scores))
```

```
# Calculate class weights
n_samples = len(y_train)
n_classes = len(np.unique(y_train))
class_counts = Counter(y_train)

class_weights = {
    cls: n_samples / (n_classes * count)
```

```

        for cls, count in class_counts.items()
    }

print("Computed Class Weights:", class_weights)

# Step 2: Apply to models

# Random Forest
rf_model = RandomForestClassifier(class_weight=class_weights)
rf_model.fit(X_train, y_train)

# Logistic Regression
lr_model = LogisticRegression(class_weight=class_weights)
lr_model.fit(X_train, y_train)

# Support Vector Classifier (SVC)
svc_model = SVC(class_weight=class_weights)
svc_model.fit(X_train, y_train)

# with auto encoda
#PrecisionRecallCurve(encoded_Xtrain, encoded_Xtest, ytrain, ytest)

```

▼ ROC curve with Stratified KFold Cross Validation

Cross-validation. Cross-validation is the most commonly used tuning method. It entails splitting a training dataset into ten equal parts (folds). A given model is trained on only nine folds and then tested on the tenth one

```

# Plotting ROC Curve for cross validation models
roc_skfold_validationAny(RandomForestClassifier(random_state=
#roc_skfold_validationAny(LogisticRegression(random_state= 10
#roc_skfold_validationAny(KNeighborsClassifier(n_neighbors = 1
#roc_skfold_validationAny(XGBClassifier(random_state= 10), X,
#roc_skfold_validationAny(GaussianNB(), X, y, 5)
#roc_skfold_validationAny(DecisionTreeClassifier(random_state=
#roc_skfold_validationAny(ExtraTreesClassifier(random_state=
#roc_skfold_validationAny(SVC(kernel='linear', probability=True,
#roc_skfold_validationAny(SVC(kernel='rbf', probability=True,

```

▼ Repeated CV

```

roc_models_new(X, y, 10) # FEATURES CAME FROM FEATURES SELEC

```

Start coding or [generate](#) with AI.

```

from scipy.stats import norm
import numpy as np
import pandas as pd
from tqdm import tqdm
from sklearn.metrics import roc_curve, auc
from sklearn.model_selection import RepeatedStratifiedKFold

def roc_models_side_effect_repeated(X, y, kfold):
    # -----
    # Define classifiers
    # -----
    MLmodels = [
        RandomForestClassifier(n_estimators=100, max_depth=10,
                               bootstrap=True, random_state=5),
        ExtraTreesClassifier(n_estimators=100, max_depth=None,
                              random_state=5),
        GradientBoostingClassifier(n_estimators=50, learning_rate=0.1,
                                    random_state=5),
        AdaBoostClassifier(n_estimators=50, random_state=5),
        XGBClassifier(random_state=5, use_label_encoder=False,
                       booster='gbtree',
                       criterion='entropy', random_state=5),
        DecisionTreeClassifier(max_depth=5, min_samples_split=10,
                               criterion='entropy', random_state=5),
        make_pipeline(StandardScaler(), QuadraticDiscriminantAnalysis()),
        make_pipeline(StandardScaler(), GaussianNB()),
        make_pipeline(StandardScaler(), SVC(kernel='rbf', gamma=0.01,
                                             probability=True,
                                             random_state=5)),
        make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5,
                                                              random_state=5)),
        make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000,
                                                             random_state=5)),
        make_pipeline(StandardScaler(), MLPClassifier(hidden_layer_sizes=(100, 100),
                                                         max_iter=1000,
                                                         solver='lbfgs',
                                                         random_state=5))
    ]

    cv = RepeatedStratifiedKFold(n_splits=kfold, n_repeats=5, random_state=5)
    results = []

    for clf in tqdm(MLmodels, desc="Evaluating models"):
        fold_aucs = []
        y_real_all, y_proba_all = [], []

        for train_idx, test_idx in cv.split(X, y):
            Xtrain = X.iloc[train_idx] if isinstance(X, pd.DataFrame) else X[train_idx]
            Xtest = X.iloc[test_idx] if isinstance(X, pd.DataFrame) else X[test_idx]
            ytrain = y.iloc[train_idx] if isinstance(y, pd.Series) else y[train_idx]
            ytest = y.iloc[test_idx] if isinstance(y, pd.Series) else y[test_idx]

            clf.fit(Xtrain, ytrain)
            y_score = clf.predict_proba(Xtest)[:, 1] if hasattr(clf, 'predict_proba') else clf.predict(Xtest)
            fpr, tpr, _ = roc_curve(ytest, y_score)
            auc_val = auc(fpr, tpr)
            fold_aucs.append(auc_val)
            y_real_all.append(ytest)
            y_proba_all.append(y_score)

    return fold_aucs, y_real_all, y_proba_all

```

```

# Fold AUC
fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
fold_aucs.append(auc(fpr_fold, tpr_fold))

y_real_all.append(ytest)
y_proba_all.append(y_score)

# Overall AUC from concatenated predictions
y_real_concat = np.concatenate(y_real_all)
y_proba_concat = np.concatenate(y_proba_all)
fpr, tpr, _ = roc_curve(y_real_concat, y_proba_concat)
overall_auc = auc(fpr, tpr)

# Fold-wise mean AUC  $\pm$  95% CI
fold_aucs = np.array(fold_aucs)
mean_auc = fold_aucs.mean()
std_auc = fold_aucs.std(ddof=1)
se_auc = std_auc / np.sqrt(kfolds)
z = norm.ppf(0.975)
ci_lower = mean_auc - z * se_auc
ci_upper = mean_auc + z * se_auc

results.append({
    'classifier': type(clf.named_steps[list(clf.named_steps) if hasattr(clf, 'named_steps') else
    'overall_auc': overall_auc,
    'mean_auc': mean_auc,
    'std_auc': std_auc,
    'se_auc': se_auc,
    'ci_lower': ci_lower,
    'ci_upper': ci_upper
})

# Sort results
df_results = pd.DataFrame(results).sort_values(by='mean_auc')
df_results = df_results.round(3)

# Save results to Excel (clean table, no fpr/tpr)
df_results.to_excel("5repeated_rmLymphedema_snp.xlsx", index=False)

return df_results

```

```
roc_models_side_effect_repeated(X, y, 10)
```

```
from scipy.stats import norm

def roc_models_side_effectfull(X, y, kfold):
    plt.figure(figsize=(10, 8), dpi=150)

    # -----
    # Define classifiers
    # -----
    MLmodels = [
        RandomForestClassifier(n_estimators=100, max_depth=10,
                               bootstrap=True, random_state=5),
        ExtraTreesClassifier(n_estimators=100, max_depth=None,
                              random_state=5),
        GradientBoostingClassifier(n_estimators=50, learning_rate=0.1,
                                    random_state=5),
        AdaBoostClassifier(n_estimators=50, random_state=5),
        XGBClassifier(random_state=5, use_label_encoder=False,
                       booster='gbtree', n_estimators=100),
        DecisionTreeClassifier(max_depth=5, min_samples_split=10,
                               criterion="entropy", random_state=5),
        make_pipeline(StandardScaler(), QuadraticDiscriminantAnalysis()),
        make_pipeline(StandardScaler(), GaussianNB()),
        make_pipeline(StandardScaler(), SVC(kernel='rbf', probability=True)),
        make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5)),
        make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
        make_pipeline(StandardScaler(), MLPClassifier(hidden_layer_sizes=(100, 50),
                                                         solver='adam',
                                                         max_iter=1000))
    ]

    colors = cm.get_cmap('tab20', len(MLmodels))
    #cv = StratifiedKFold(n_splits=kfold, shuffle=True, random_state=5)
    cv = RepeatedStratifiedKFold(n_splits=kfold, n_repeats=5, random_state=5)

    results = []

    for i, clf in enumerate(tqdm(MLmodels, desc="Evaluating model %d" % i)):
        fold_aucs = []
        y_real_all, y_proba_all = [], []

        for train_idx, test_idx in cv.split(X, y):
            Xtrain = X.iloc[train_idx] if isinstance(X, pd.DataFrame) else X[train_idx]
            Xtest = X.iloc[test_idx] if isinstance(X, pd.DataFrame) else X[test_idx]
            ytrain = y.iloc[train_idx] if isinstance(y, pd.Series) else y[train_idx]
            ytest = y.iloc[test_idx] if isinstance(y, pd.Series) else y[test_idx]

            clf.fit(Xtrain, ytrain)

            y_proba = clf.predict_proba(Xtest)

            y_proba_all.append(y_proba)

            fold_auc = roc_auc_score(ytest, y_proba[:, 1])
            fold_aucs.append(fold_auc)

        results.append((i, clf, fold_aucs))
```

```

# Prediction
y_score = clf.predict_proba(Xtest)[:, 1] if hasattr

# Fold-wise AUC
fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
fold_aucs.append(auc(fpr_fold, tpr_fold))

y_real_all.append(ytest)
y_proba_all.append(y_score)

# Concatenate predictions for overall ROC
y_real_concat = np.concatenate(y_real_all)
y_proba_concat = np.concatenate(y_proba_all)
fpr, tpr, _ = roc_curve(y_real_concat, y_proba_concat)
overall_auc = auc(fpr, tpr)

# Fold-wise mean AUC  $\pm$  95% CI
fold_aucs = np.array(fold_aucs)
mean_auc = fold_aucs.mean()
std_auc = fold_aucs.std(ddof=1) # Standard Error
se_auc = std_auc / np.sqrt(kfolds) # Standard Error
z = norm.ppf(0.975)
ci_lower = mean_auc - z * se_auc
ci_upper = mean_auc + z * se_auc

results.append({
    'classifier': type(clf.named_steps[list(clf.named_steps) if hasattr(clf, 'named_steps') else
    'fpr': fpr,
    'tpr': tpr,
    'overall_auc': overall_auc, # concatenated predictions
    'mean_auc': mean_auc, # fold-wise mean
    'std_auc': std_auc, # NEW: standard deviation
    'se_auc': se_auc, # NEW: standard error
    'ci_lower': ci_lower,
    'ci_upper': ci_upper
})

# Sort by fold-wise mean AUC
results = sorted(results, key=lambda x: x['mean_auc'], reverse=True)

# Plot ROC curves (overall AUC from concatenated predictions)
for i, r in enumerate(results):
    plt.plot(r['fpr'], r['tpr'],
             label=f"{r['classifier']} (overall AUC={r['overall_auc']:.2f}, mean AUC={r['mean_auc']:.2f}, 95% CI

```

```

        color=colors(i))

plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'ROC Curves for Side Effect Prediction ({kfold})')
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, color='white', linestyle='-', linewidth=1)
plt.legend(loc='lower right', fontsize='small', frameon=True)
plt.show()
# Convert to DataFrame (exclude fpr/tptr arrays to keep table compact)
df_results = pd.DataFrame(results)
df_results = df_results.round(3) # nicer formatting

# Save results to Excel
df_results.to_excel("rmLymphedema_clinic.xlsx", index=False)
# Return DataFrame with SD and SE included
return pd.DataFrame(results).sort_values(by='mean auc', ascending=False)

```

```
roc_models_side_effectfull(X, y, 10)
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

```

# WITH STANDARD ERROR
def roc_models_newSE(X, y, kfold):
    plt.figure(figsize=(10, 8), dpi=150)

    # List of models to evaluate
    MLmodels = [
        RandomForestClassifier(n_estimators=100, max_depth=10),
        ExtraTreesClassifier(n_estimators=100, max_depth=None),

```

```

GradientBoostingClassifier(n_estimators=50, learning_
AdaBoostClassifier(n_estimators=50, random_state=5),
XGBClassifier(random_state=10, use_label_encoder=False)
DecisionTreeClassifier(max_depth=10, min_samples_split=10)

make_pipeline(StandardScaler(), QuadraticDiscriminant
make_pipeline(StandardScaler(), GaussianNB()),
make_pipeline(StandardScaler(), SVC(kernel='rbf', probability=True)),
make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5)),
make_pipeline(StandardScaler(), LogisticRegression(solver='lbfgs')),
make_pipeline(StandardScaler(), MLPClassifier(hidden_layer_sizes=(100, 100, 100)))
]

colors = cm.get_cmap('tab20', len(MLmodels))
cv = StratifiedKFold(n_splits=kfolds, shuffle=True, random_state=42)

results = []

for i, classifier in enumerate(tqdm(MLmodels, desc="Evaluating Model %d" % i)):
    y_real = []
    y_proba = []
    aucs = []

    for train_index, test_index in cv.split(X, y):
        if isinstance(X, pd.DataFrame):
            Xtrain, Xtest = X.iloc[train_index], X.iloc[test_index]
            ytrain, ytest = y.iloc[train_index], y.iloc[test_index]
        else:
            Xtrain, Xtest = X[train_index], X[test_index]
            ytrain, ytest = y.iloc[train_index], y.iloc[test_index]

        classifier.fit(Xtrain, ytrain)
        y_score = (
            classifier.predict_proba(Xtest)[:, 1]
            if hasattr(classifier, 'predict_proba')
            else classifier.decision_function(Xtest)
        )

        fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
        auc_fold = auc(fpr_fold, tpr_fold)
        aucs.append(auc_fold)

    y_real.append(ytest)
    y_proba.append(y_score)

y_real = np.concatenate(y_real)

```



```

y_proba = np.concatenate(y_proba)

fpr, tpr, _ = roc_curve(y_real, y_proba)
auc_mean = auc(fpr, tpr)

# Compute Standard Error of the Mean (SEM) for AUC
sem_auc = np.std(aucs, ddof=1) / np.sqrt(len(aucs))

results.append({
    'classifier': type(classifier.named_steps[list(c)
        if hasattr(classifier, 'named_steps')
    'fpr': fpr,
    'tpr': tpr,
    'auc_mean': auc_mean,
    'sem_auc': sem_auc
}))

# Sort results by AUC mean
results = sorted(results, key=lambda x: x['auc_mean'])

# Plot ROC curves
for i, result in enumerate(results):
    fpr, tpr, auc_mean, sem_auc = result['fpr'], result['tpr'], result['auc_mean'], result['sem_auc']
    plt.plot(fpr, tpr, label=f'{result["classifier"]} (Mean AUC: {auc_mean:.2f})')

plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random Guess')
plt.xlabel('False Positive Rate -->')
plt.ylabel('True Positive Rate -->')
plt.title(f'ROC Curve Comparison ({kfolds}-Fold Cross-Validation)')
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, color='white', linestyle='-', linewidth=1)
plt.legend(loc='lower right', fontsize='small', frameon=True)
plt.show()

# Optionally return results
# return pd.DataFrame(results).sort_values(by='auc_mean',

```

```
roc_models_newSE(X, y, 10) # FEATURES CAME FROM FEATURES SELI
```

```
encoded_features, Y= auto_encoder_feature_selection(reduced_d
```

```
roc_models_new(encoded_features, Y, 10) # FEATURES SELECTED B'
```

```
# auc and precision
```

```
def auc_pr_roc_models(X, y, kfold):
```

```
    plt.figure(figsize=(10, 8), dpi=150)
```

```
    # Define models (with pipeline for those needing scaling)
```

```
    MLmodels = [
```

```
        RandomForestClassifier(n_estimators=100, max_depth=10
```

```
        ExtraTreesClassifier(n_estimators=100, max_depth=Nor
```

```
        GradientBoostingClassifier(n_estimators=50, learning_
```

```
        AdaBoostClassifier(n_estimators=50, random_state=5),
```

```
        XGBClassifier(random_state=10, use_label_encoder=False
```

```
        DecisionTreeClassifier(max_depth=10, min_samples_spli
```

```
        make_pipeline(StandardScaler(), QuadraticDiscriminant
```

```
        make_pipeline(StandardScaler(), GaussianNB()),
```

```
        make_pipeline(StandardScaler(), SVC(kernel='rbf', pro
```

```
        make_pipeline(StandardScaler(), KNeighborsClassifier(
```

```
        make_pipeline(StandardScaler(), LogisticRegression(ra
```

```
        make_pipeline(StandardScaler(), MLPClassifier(hidden_
```

```
    ]
```

```
    colors = cm.get_cmap('tab20', len(MLmodels))
```

```
    cv = StratifiedKFold(n_splits=kfold, shuffle=True, rand
```

```
    results = []
```

```
    results2=[]
```

```
    for i, classifier in enumerate(MLmodels):
```

```
        y_real = []
```

```
        y_proba = []
```

```
        aucs = []
```

```
        for train_index, test_index in cv.split(X, y):
```

```
            Xtrain, Xtest = X.iloc[train_index], X.iloc[test_
```

```
            ytrain, ytest = y.iloc[train_index], y.iloc[test_
```

```
            classifier.fit(Xtrain, ytrain)
```

```
            y_score = (classifier.predict_proba(Xtest)[: , 1]
```

```
                        if hasattr(classifier, 'predict_proba'
```

```
                        else classifier.decision_function(Xtes
```

```

fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
auc_fold = auc(fpr_fold, tpr_fold)
aucs.append(auc_fold)

y_real.append(ytest)
y_proba.append(y_score)

y_real = np.concatenate(y_real)
y_proba = np.concatenate(y_proba)

# ROC-AUC
fpr, tpr, _ = roc_curve(y_real, y_proba)
auc_mean = auc(fpr, tpr)
std_auc = np.std(aucs)

# PR-AUC
precision, recall, _ = precision_recall_curve(y_real,
auc_pr = average_precision_score(y_real, y_proba)

# Model name (handle pipelines)
model_name = (type(classifier.named_steps[list(classi
            if hasattr(classifier, 'named_steps') €

results.append({
    'classifier': model_name,
    'fpr': fpr,
    'tpr': tpr,
    'auc_mean': auc_mean,
    'std_auc': std_auc,
    'precision': precision,
    'recall': recall,
    'auc_pr': auc_pr
})
# Sort by AUC
results = sorted(results, key=lambda x: x['auc_mean'])
results2 = sorted(results, key=lambda x: x['auc_pr'])
# ROC Plot
for i, result in enumerate(results):
    plt.plot(result['fpr'], result['tpr'],
            label=f'{result["classifier"]} (ROC AUC = {r
            color=colors(i))
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', la
plt.xlabel('False Positive Rate -->')
plt.ylabel('True Positive Rate -->')
plt.title(f'ROC Curve Comparison ({kfolds}-Fold Cross-Val
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, color='white', linestyle='-', linewidth=1)

```

```
plt.legend(loc='lower right', fontsize='small', frameon=1)
plt.show()

# PR Plot
plt.figure(figsize=(10, 8), dpi=150)
for i, result in enumerate(results2):
    plt.plot(result['recall'], result['precision'],
             label=f'{result["classifier"]} (AUC-PR = {result["auc_pr"]})',
             color=colors(i))
plt.xlabel('Recall -->')
plt.ylabel('Precision -->')
plt.title(f'Precision-Recall Curve Comparison ({kfolds}-Fold)')
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, color='white', linestyle='--', linewidth=1)
plt.legend(loc='upper right', fontsize='small', frameon=1)
plt.show()
```

```
auc_pr_roc_models(X, y, 10)
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import matthews_corrcoef
from sklearn.ensemble import RandomForestClassifier # replace
from sklearn.datasets import make_classification
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt
plt.rcParamsdefaults() # Reset ALL matplotlib settings
plt.style.use('default') # Use clean default style

def MCC_curve(X, y, kfold):

    # Model
    #model = RandomForestClassifier(n_estimators=100, max_depth=5)
    #model = AdaBoostClassifier(n_estimators=50, random_state=5)
    # 10-Fold Stratified CV
    model = make_pipeline(StandardScaler(), LogisticRegression())
```

```

#model = make_pipeline(StandardScaler(), MLPClassifier(hidden_layer_sizes=(100, 100),
cv = StratifiedKFold(n_splits=kfold, shuffle=True, random_state=42))

# Store all predictions and labels
all_probs = []
all_labels = []

for train_idx, test_idx in cv.split(X, y):
    model.fit(X.iloc[train_idx], y.iloc[train_idx])
    probs = model.predict_proba(X.iloc[test_idx])[:, 1]
    all_probs.extend(probs)
    all_labels.extend(y.iloc[test_idx])

# Convert to numpy arrays
all_probs = np.array(all_probs)
all_labels = np.array(all_labels)

# Compute MCC at different thresholds
thresholds = np.linspace(0, 1, 200)
mcc_scores = []

for t in thresholds:
    preds = (all_probs >= t).astype(int)
    mcc = matthews_corrcoef(all_labels, preds)
    mcc_scores.append(mcc)

# Step 6: Find the best threshold
best_idx = np.argmax(mcc_scores)
best_thresh = thresholds[best_idx]
best_mcc = mcc_scores[best_idx]

# Step 7: Plot MCC curve
plt.figure(figsize=(10, 8))
plt.plot(thresholds, mcc_scores, label='MCC Curve')
plt.axvline(best_thresh, color='red', linestyle='--', label='Best Threshold')
plt.scatter(best_thresh, best_mcc, color='red')
plt.xlabel('Threshold')
plt.ylabel('Matthews Correlation Coefficient')
plt.title('MCC Curve with 10-Fold Cross-Validation')
#plt.gca().set_facecolor('#F9F9F9')
plt.grid(True, color='gray')
plt.legend()
plt.show()

# Step 8: Print best threshold and MCC value
print(f'Best MCC: {best_mcc:.2f} at threshold = {best_thresh:.2f}')

```

MCC_curve(X, y, 10)

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import matthews_corrcoef, confusion_matrix, f1_score, precision_score, recall_score

def evaluate_model_cv_with_threshold_verbose(X, y, threshold,
                                             skf = StratifiedKFold(n_splits=kfolds, shuffle=True, random_state=42)):
    all_true = []
    all_probs = []
    all_preds = []
    all_indices = []

    for fold, (train_idx, test_idx) in enumerate(skf.split(X, y)):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

        #model = RandomForestClassifier(n_estimators=100, max_depth=5, min_samples_split=5, min_samples_leaf=1, bootstrap=True)
        #model = make_pipeline(StandardScaler(), MLPClassifier(100), activation='relu', solver='adam', random_state=42)
        model = make_pipeline(StandardScaler(), LogisticRegression())

        model.fit(X_train, y_train)

        probs = model.predict_proba(X_test)[:, 1]
        preds = (probs >= threshold).astype(int)

        all_probs.extend(probs)
        all_preds.extend(preds)
        all_true.extend(y_test)
        all_indices.extend(X_test.index)

    # Store predictions
    results_df = X.loc[all_indices].copy()
    results_df['true_label'] = np.array(all_true)
    results_df['predicted_proba'] = np.array(all_probs)
    results_df['predicted_label'] = np.array(all_preds)

    # Metrics
    y_true = results_df['true_label']
    y_pred = results_df['predicted_label']
```

```

y_pred = results_df[ 'predicted_label' ]
mcc = matthews_corrcoef(y_true, y_pred)
conf_matrix = confusion_matrix(y_true, y_pred)
report = classification_report(y_true, y_pred)

print("\n 📊 Confusion Matrix:\n", conf_matrix)
print("\n 📋 Classification Report:\n", report)
print(f"✅ MCC Score at threshold {threshold}: {mcc:.4f}")

# Friendly Summary
tn, fp, fn, tp = conf_matrix.ravel()
precision_0 = tn / (tn + fn) if (tn + fn) else 0
recall_0 = tn / (tn + fp) if (tn + fp) else 0
precision_1 = tp / (tp + fp) if (tp + fp) else 0
recall_1 = tp / (tp + fn) if (tp + fn) else 0
f1_1 = f1_score(y_true, y_pred)

print("\nSummary Interpretation:")
print("\nClass 0 - Healthy Patients")
print(f"Precision: {precision_0:.2f} → Model is {int(prec) % 100} % correct when predicting 'healthy'")
print(f"Recall: {recall_0:.2f} → Correctly detects {int(recall_0 * 100)} % of healthy patients")

print("\nClass 1 - Sick Patients")
print(f"Precision: {precision_1:.2f} → Model is right {int(precision_1 * 100)} % when predicting 'sick'")
print(f"Recall: {recall_1:.2f} → Identifies {int(recall_1 * 100)} % of sick patients")
print(f"F1 Score: {f1_1:.2f} → Balance between precision and recall")

print("\nOverall Performance")
print(f"MCC Score: {mcc:.3f} → {' Strong' if mcc > 0.6 else ' Weak'} correlation between true and predicted")

print("\n Sample predictions:")
print(results_df[['true_label', 'predicted_proba', 'predicted_label']])

return results_df

```

```
evaluate_model_cv_with_threshold_verbose(X, y, 0.47, 10)
```

```
evaluate_model_cv_with_threshold(X, y, 0.29, 10)
```

```

# Finding the best parameter for all models
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import make_scorer, average_precision_score

```

```

def grid_search_model(X, y):
    # Create a pipeline (optional, but good if you need scaling)
    pipeline = Pipeline([
        ('scaler', StandardScaler()),          # Optional for
        ('clf', RandomForestClassifier(random_state=42))
    ])

    # Define the hyperparameter grid
    param_grid = {
        'clf__n_estimators': [100, 200, 300],
        'clf__max_depth': [None, 10, 20],
        'clf__min_samples_split': [2, 5, 10],
        'clf__min_samples_leaf': [1, 2, 4],
        'clf__bootstrap': [True, False]
    }

    # Use AUC-PR as the scoring metric
    scorer = make_scorer(average_precision_score, needs_proba=True)

    # Grid search
    grid_search = GridSearchCV(pipeline, param_grid, cv=10, scoring=scorer)

    # Fit to your data
    grid_search.fit(X, y)

    # Best parameters and score
    print("Best Parameters:", grid_search.best_params_)
    print("Best AUC-PR Score:", grid_search.best_score_)

grid_search_model(X, y)

```

```

# Finding the best parameter for all models
from sklearn.model_selection import GridSearchCV

def grid_search_models(X, y):
    models_and_grids = {
        'RandomForest': (RandomForestClassifier(random_state=42),
                          {
                              'n_estimators': [100, 200],
                              'max_depth': [None, 10, 20],
                              'min_samples_split': [2, 5],
                              'min_samples_leaf': [1, 3],
                          }),
        'ExtraTrees': (ExtraTreesClassifier(random_state=10),
                        {

```



```

{
    'n_estimators': [100, 200],
    'max_depth': [None, 10],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 3],
}),
'GradientBoosting': (GradientBoostingClassifier(random_state=10),
{
    'n_estimators': [100, 200],
    'learning_rate': [0.05, 0.1],
    'max_depth': [3, 5],
}),
'AdaBoost': (AdaBoostClassifier(random_state=10),
{
    'n_estimators': [50, 100],
    'learning_rate': [0.5, 1.0],
}),
'XGBoost': (XGBClassifier(use_label_encoder=False,
                           eval_metric='logloss', random_state=10),
{
    'n_estimators': [100, 200],
    'max_depth': [3, 5],
    'learning_rate': [0.05, 0.1],
}),
'LogisticRegression': (Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(max_iter=1000, random_state=10)),
{
    'clf__C': [0.1, 1, 10],
    'clf__penalty': ['l2'],
    'clf__solver': ['lbfgs']
}),
'SVC': (Pipeline([
    ('scaler', StandardScaler()),
    ('clf', SVC(probability=True, random_state=10)),
{
    'clf__C': [0.1, 1, 10],
    'clf__gamma': ['scale', 'auto'],
    'clf__kernel': ['rbf']
}),
'KNN': (Pipeline([
    ('scaler', StandardScaler()),
    ('clf', KNeighborsClassifier())]),
{
    'clf__n_neighbors': [3, 5, 6, 7]
}),
'MLP': (Pipeline([

```

```

        ('scaler', StandardScaler()),
        ('clf', MLPClassifier(max_iter=500, random_state=
{
    'clf__hidden_layer_sizes': [(50,), (100,), (1
    'clf__activation': ['relu', 'tanh'],
    'clf__solver': ['adam', 'lbfgs'],
    'clf__alpha': [0.0001, 0.001],
})),
'DecisionTree': (DecisionTreeClassifier(random_state=
{
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 3],
})),
'GaussianNB': (GaussianNB(),
{}), # No parameters to tune
'QDA': (QuadraticDiscriminantAnalysis(),
{}), # No parameters to tune
}

print("\n Starting Grid Search...\n")

for name, (model, param_grid) in models_and_grids.items():
    print(f" Tuning {name}...")
    grid = GridSearchCV(model, param_grid, cv=10, scoring='roc_auc')
    grid.fit(X, y)

    print(f" Best {name} Parameters: {grid.best_params_}")
    print(f" Best {name} AUC Score: {grid.best_score_:.4f}")

```

```

def roc_models(X, y, kfolds, best_k):
    plt.figure(figsize=(10, 8), dpi=150)

    # List of models to evaluate
    MLmodels = [
        LogisticRegression(
            penalty='l2',                # Regularization type
            C=1.0,                        # Inverse of regularization strength
            solver='lbfgs',              # Efficient for small/medium datasets
            max_iter=200,                # Increased iteration count
            random_state=12
        ),
        GaussianNB(
            var_smoothing=1e-9           # Stability parameter for numerical
        ),
        RandomForestClassifier(
            n_estimators=100,            # Number of trees

```

```

        max_depth=10,                # Limit depth to prev
        min_samples_split=5,         # Minimum samples to
        min_samples_leaf=3,          # Minimum samples per
        bootstrap=True,              # Bootstrap sampling
        random_state=12
    ),
    MLPClassifier(
        hidden_layer_sizes=(128, 64), # Two hidden layers (1
        activation='relu',            # Activation function
        solver='adam',                 # Optimizer
        alpha=0.001,                   # L2 regularization t
        learning_rate_init=0.001,      # Initial learning ra
        max_iter=300,                  # Max iterations to e
        random_state=12
    ),
    GradientBoostingClassifier(
        n_estimators=100,              # Number of boosting
        learning_rate=0.05,            # Smaller learning ra
        max_depth=4,                   # Depth of each tree
        subsample=0.8,                 # Subsample fraction
        min_samples_split=5,           # Minimum samples to
        random_state=12
    ),
    QuadraticDiscriminantAnalysis(
        reg_param=0.01                # Regularization para
    ),
    ExtraTreesClassifier(
        n_estimators=100,              # Number of trees
        max_depth=None,                # Fully grown trees
        min_samples_split=4,           # Minimum samples to
        bootstrap=False,               # No bootstrap sampli
        random_state=12
    ),
    XGBClassifier(
        n_estimators=150,              # Number of boosting
        learning_rate=0.05,            # Learning rate
        max_depth=6,                   # Tree depth
        subsample=0.8,                 # Fraction of samples
        colsample_bytree=0.8,          # Subsample ratio of
        eval_metric='logloss',         # Evaluation metric
        random_state=12,
        use_label_encoder=False       # Avoids unnecessary
    ),
    AdaBoostClassifier(
        n_estimators=100,              # Number of weak clas
        learning_rate=0.5,             # Learning rate (high

```

```

        random_state=12
    ),
    SVC(
        kernel='rbf',                # Non-linear kernel
        C=1.0,                        # Regularization parameter
        gamma='scale',               # Kernel coefficient
        probability=True,            # Enable probability
        random_state=12
    ),
    KNeighborsClassifier(
        n_neighbors=best_k,
        weights='distance',          # Weight neighbors by distance
        metric='euclidean'           # Distance metric
    ),
    DecisionTreeClassifier(
        max_depth=10,                # Limit tree depth
        min_samples_split=4,         # Minimum samples to split
        min_samples_leaf=2,          # Minimum samples per leaf
        criterion='gini',            # Gini impurity or 'entropy'
        random_state=12
    )
]

# Create a color map with enough distinct colors for all
colors = cm.get_cmap('tab20', len(MLmodels))

# Define cross-validator
cv = StratifiedKFold(n_splits=kfolds, shuffle=True, random_state=12)

# Store results for sorting
results = []

# Iterate over each classifier
for i, classifier in enumerate(MLmodels):
    y_real = []
    y_proba = []
    aucs = []

    # Perform cross-validation
    for train_index, test_index in cv.split(X, y):
        Xtrain, Xtest = X.iloc[train_index], X.iloc[test_index]
        ytrain, ytest = y.iloc[train_index], y.iloc[test_index]

        # Fit classifier and predict probabilities
        classifier.fit(Xtrain, ytrain)
        y_score = (classifier.predict_proba(Xtest)[:, 1])
        if hasattr(classifier, 'predict_proba')

```

```

        else classifier.decision_function(Xtest)

    # Calculate ROC for the current fold
    fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
    auc_fold = auc(fpr_fold, tpr_fold)
    aucs.append(auc_fold)

    # Collect true labels and probabilities for final
    y_real.append(ytest)
    y_proba.append(y_score)

# Concatenate results from all folds
y_real = np.concatenate(y_real)
y_proba = np.concatenate(y_proba)

# Calculate overall ROC and AUC
fpr, tpr, _ = roc_curve(y_real, y_proba)
auc_mean = auc(fpr, tpr)
std_auc = np.std(aucs)

# Store results for later sorting
results.append({'classifier': type(classifier).__name__})

# Sort results by AUC mean
results = sorted(results, key=lambda x: x['auc_mean'])

# Plot ROC curves in sorted order
for i, result in enumerate(results):
    fpr, tpr, auc_mean, std_auc = result['fpr'], result['tpr'], result['auc_mean'], result['std_auc']
    plt.plot(fpr, tpr, label=f'{result["classifier"]} (Mean AUC: {auc_mean:.2f})')

# Plot formatting and labeling
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
plt.xlabel('False Positive Rate -->')
plt.ylabel('True Positive Rate -->')
plt.title(f'ROC Curve Comparison ({kfold}-Fold Cross-Validation)')

# Background and grid settings
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, color='white', linestyle='-', linewidth=1)

# Display the legend
plt.legend(loc='lower right', fontsize='small', frameon=True)

# Show the plot
plt.show()

```

```
roc_models(X, y, 10, best_k)
```

✓ Bootstrap with class weight and balance

roc_bootstrap: This method estimates the variability of a single model's ROC curve using bootstrapping. It creates resampled datasets to calculate mean and standard deviation of the ROC curve.

- **bootstrap resampling**, repeatedly sampling the test set with replacement. This approach simulates variability in the data and evaluates the model's performance across resampled test sets
- **Errors in roc_bootstrap** The roc_bootstrap method may raise issues such as: Insufficient Test Set Size: If test_size in train_test_split is too small, bootstrap resampling may create overlapping or identical samples, reducing variability. Predict Probability Assumption: Assumes that the model supports predict_proba and the output is for a binary classifier with at least two classes.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

def roc_bootstrap(model, X, y, num_bootstraps, title='ROC Cur
    classifier = model

    # Split the data into train and test sets
    X_train, X_test, y_train, y_test = train_test_split(X, y,

    # Fit the model on the training data
    classifier.fit(X_train, y_train)

    # Predict probabilities for the test set
    y_score = classifier.predict_proba(X_test)

    # Check if the model provides probabilities for binary cl
    if y_score.shape[1] >= 2:
        y_score = y_score[:, 1] # Use the second column for
    else:
        raise ValueError("The predicted probabilities should

    predicted_probabilities = y_score
    true_labels = y_test.values # Convert y_test to a NumPy

    # Initialize arrays for bootstrapped TPR and AUC values
    tpr_values = []
```

```

auc_values = []
base_fpr = np.linspace(0, 1, 101)

for _ in range(num_bootstraps):
    # Resample with replacement
    indices = np.random.choice(len(predicted_probabilities))
    bootstrap_predicted_probabilities = predicted_probabilities[indices]
    bootstrap_true_labels = true_labels[indices]

    # Calculate ROC curve
    fpr, tpr, _ = roc_curve(bootstrap_true_labels, bootstrap_predicted_probabilities)

    # Interpolate TPR values to the base FPR
    tpr = np.interp(base_fpr, fpr, tpr)
    tpr[0] = 0.0 # Ensure the curve starts at 0

    tpr_values.append(tpr)
    auc_values.append(auc(base_fpr, tpr))

# Calculate the mean and standard deviation of TPR and AUC
mean_tpr = np.mean(tpr_values, axis=0)
std_tpr = np.std(tpr_values, axis=0)
mean_auc = np.mean(auc_values)
std_auc = np.std(auc_values)

# Plot the mean ROC curve
plt.figure(figsize=(10, 8), dpi=150)
plt.plot(base_fpr, mean_tpr, color='darkorange', lw=2,
         label=f'Mean ROC Curve (AUC = {mean_auc:.2f} ± {std_auc:.2f})')

# Add the confidence band
plt.fill_between(base_fpr,
                 mean_tpr - std_tpr,
                 mean_tpr + std_tpr,
                 color='grey', alpha=0.2,
                 label='Confidence Band')
#label='Confidence Band (± 1 std)')

# Plot the chance line
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')

# Set plot limits and labels
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title(f'{title}\nModel: {type(model).__name__}, Boots

```

```

# Add grid and legend
plt.grid(color='lightgrey', linestyle='--', linewidth=0.5)
plt.legend(loc="lower right", fontsize=10, frameon=True,

# Annotate the shaded area explanation
#plt.text(0.6, 0.2, f'Shaded area =  $\pm 1$  std\nTPR & AUC Va

# Show the plot
plt.show()

# Example usage with GradientBoostingClassifier
# Assuming X and y are your features and target variables, re
roc_bootstrap(AdaBoostClassifier(n_estimators=100,
                                learning_rate=0.2,           # Learning rate (high
                                random_state=12

```

```

from sklearn.utils.class_weight import compute_class_weight
import numpy as np

# Compute class weights
classes = np.unique(y)
class_weights_array = compute_class_weight(class_weight='balanced',
class_weights = {cls: weight for cls, weight in zip(classes, class_weights_array)}

print(class_weights)

```

```

# trying to plot k fold cross validation for class wight but
def roc_models_B(X, y, kfolds, best_k=5, class_weights):
    """
    Plots the mean AUC ROC curves for multiple machine learning models using
    K-Fold cross-validation.

    Parameters:
    - X: pd.DataFrame or np.ndarray
      Feature matrix.
    - y: pd.Series or np.ndarray

```



```

y = pd.Series(y_train, index=x_train.index)
    Target variable.
- kfolds: int
    Number of folds for StratifiedKFold cross-validation.
- best_k: int, optional
    The best value of k for KNeighborsClassifier (default
- class_weights: dict or str, optional
    Weights associated with classes. Use 'balanced' for a
    provide a dictionary like {0: w1, 1: w2}.

Returns:
- None. Displays the ROC plot.
"""

plt.figure(figsize=(10, 8), dpi=150)

# List of models to evaluate
MLmodels = [
    SVC(kernel='rbf', probability=True, random_state=10),
    GaussianNB(),
    ExtraTreesClassifier(random_state=10),
    XGBClassifier(random_state=10),
    RandomForestClassifier(random_state=10),
    AdaBoostClassifier(n_estimators=50, random_state=10),
    GradientBoostingClassifier(n_estimators=50, learning_
    random_state=3),
    KNeighborsClassifier(n_neighbors=best_k),
    LogisticRegression(random_state=10),
    DecisionTreeClassifier(random_state=10),
    QuadraticDiscriminantAnalysis(),
    MLPClassifier(hidden_layer_sizes=(100,), activation='
    random_state=10),
]

# Create a color map with enough distinct colors for all
colors = cm.get_cmap('tab20', len(MLmodels))

# Define cross-validator
cv = StratifiedKFold(n_splits=kfolds, shuffle=True, random_

# Store results for sorting
results = []

# Iterate over each classifier
for i, classifier in enumerate(MLmodels):
    y_real = []
    y_proba = []
    aucs = []

```

```

# Perform cross-validation
for train_index, test_index in cv.split(X, y):
    Xtrain, Xtest = X.iloc[train_index], X.iloc[test_index]
    ytrain, ytest = y.iloc[train_index], y.iloc[test_index]

    # Fit classifier and predict probabilities
    classifier.fit(Xtrain, ytrain, class_weight=class_weight)

    # Predict probabilities or decision function
    y_score = (classifier.predict_proba(Xtest)[:, 1]
               if hasattr(classifier, 'predict_proba')
               else classifier.decision_function(Xtest))

    # Calculate ROC for the current fold
    fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
    auc_fold = auc(fpr_fold, tpr_fold)
    aucs.append(auc_fold)

    # Collect true labels and probabilities for final
    y_real.append(ytest)
    y_proba.append(y_score)

# Concatenate results from all folds
y_real = np.concatenate(y_real)
y_proba = np.concatenate(y_proba)

# Calculate overall ROC and AUC
fpr, tpr, _ = roc_curve(y_real, y_proba)
auc_mean = auc(fpr, tpr)
std_auc = np.std(aucs)

# Store results for later sorting
results.append({'classifier': type(classifier).__name__,
               'auc_mean': auc_mean, 'std_auc': std_auc})

# Sort results by AUC mean
results = sorted(results, key=lambda x: x['auc_mean'])

# Plot ROC curves in sorted order
for i, result in enumerate(results):
    fpr, tpr, auc_mean, std_auc = result['fpr'], result['tpr'],
    result['auc_mean'], result['std_auc']
    plt.plot(fpr, tpr, label=f'{result["classifier"]} (Mean AUC: {auc_mean:.2f})', color=colors(i))

# Plot formatting and labeling
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')

```

```
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title(f'ROC Curve Comparison ({kfolds})-Fold Cross-Val
fontweight='bold')

# Background and grid settings
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, color='white', linestyle='--', linewidth=1)

# Display the legend
plt.legend(loc='lower right', fontsize='small', frameon=T

# Show the plot
plt.show()
```

```
#roc_models_B(X, y, 10, best_k, class_weights=class_weights)
```

```
# This function very important for plotting cross validation
# This function to plot the mean average AUC StratifiedKFold
several ML model
```

```
from matplotlib import cm
```

```
def Balanced_roc_models(X, y, kfolds):
    plt.figure(figsize=(10, 8), dpi=150)
```

```
# List of models to evaluate
MLmodels = [
```

```
    ExtraTreesClassifier(class_weight='balanced', ran
    HistGradientBoostingClassifier(class_weight='bala
    XGBClassifier(class_weight='balanced', random_sta
    RandomForestClassifier(class_weight='balanced', r
    SVC(class_weight='balanced', kernel='rbf', probab
    10),
    LogisticRegression(class_weight='balanced', rando
    DecisionTreeClassifier(class_weight='balanced', r
    ]
```

```
# Create a color map with enough distinct colors for all
colors = cm.get_cmap('tab20', len(MLmodels)) # Use 'tab2
colormap
```

```
# Define cross-validator
```

```
cv = StratifiedKFold(n_splits=kfolds, shuffle=True, rando
```

```

# Iterate over each classifier
for i, classifier in enumerate(MLmodels):
    y_real = []
    y_proba = []
    aucs = []

    # Perform cross-validation
    for train_index, test_index in cv.split(X, y):
        Xtrain, Xtest = X.iloc[train_index], X.iloc[test_index]
        ytrain, ytest = y.iloc[train_index], y.iloc[test_index]

        # Fit classifier and predict probabilities
        classifier.fit(Xtrain, ytrain)
        y_score = classifier.predict_proba(Xtest)[:, 1] if classifier.__name__ == 'predict_proba' else classifier.decision_function(Xtest)

        # Calculate ROC for the current fold
        fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
        auc_fold = auc(fpr_fold, tpr_fold)
        aucs.append(auc_fold) # Store AUC for averaging

    # plotting the folds
    #plt.plot(fpr, tpr, lw=1, linestyle='--', label=f'Fold {i+1}, roc_auc={auc_mean:.2f}') # without dots

    # Collect true labels and probabilities for final
    y_real.append(ytest)
    y_proba.append(y_score)

# Concatenate results from all folds
y_real = np.concatenate(y_real)
y_proba = np.concatenate(y_proba)

# Calculate overall ROC and AUC
fpr, tpr, _ = roc_curve(y_real, y_proba)
auc_mean = auc(fpr, tpr)
std_auc = np.std(aucs)

# Plot ROC curve for the model
plt.plot(fpr, tpr, label=f'{type(classifier).__name__} ± {std_auc:.2f}', color=colors(i)) #In the code,
standard deviation of the AUC values across the different
cross-validation. It measures the variability of the
Curve) from fold to folds.

# Plot formatting and labeling

```

```
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', la
plt.xlabel('False Positive Rate -->')
plt.ylabel('True Positive Rate -->')
plt.title(f'ROC Curve Comparison ({kfolds}-Fold Cross-Val
classes)', fontsize=14, fontweight='bold')

# Background and grid settings
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, color='white', linestyle='--', linewidth=1)

# Display the legend
plt.legend(loc='lower right', fontsize='small', frameon=T

# Show the plot
plt.show()
```

Balanced_roc_models(X, y, 10)

```
# This function very important for plotting cross validation
# This function to plot the mean average AUC repeated KFold C
several ML model
```

```
def n_repeated_roc_models_balance(X, y, kfolds, n):
    plt.figure(figsize=(10, 8), dpi=150)

    # Train multiple models
    MLmodels = [
        ExtraTreesClassifier(class_weight='balanced',
        XGBClassifier(class_weight='balanced', random
        HistGradientBoostingClassifier(class_weight='
        random_state=10),
        RandomForestClassifier(class_weight='balanced
        SVC(class_weight='balanced', kernel='rbf', pr
        random_state= 10),
        LogisticRegression(class_weight='balanced', r
        DecisionTreeClassifier(class_weight='balanced
    ]

    cv = RepeatedStratifiedKFold(n_splits=kfolds, n_repeats=
    for classifier in MLmodels:
        y_real=[]
        y_proba= []
        for i, (train_index, test_index) in enumerate(cv.split(
            Xtrain, ytrain = X.iloc[train_index], y.iloc[train_
            Xtest, ytest = X.iloc[test_index], y.iloc[test_inde
            classifier.fit(Xtrain, ytrain)
            v score = classifier.predict_proba(Xtest)[:.. 1] if
```

```

        'predict_proba') else classifier.decision_function(
fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
roc_auc= auc(fpr_fold, tpr_fold)
# plotting the folds
#plt.plot(fpr, tpr, lw=1, linestyle='--', label='Fold %d' % (k+1))
roc_auc)) # without dots
y_real.append(ytest)
y_proba.append(y_score)

y_real = np.concatenate(y_real)
y_proba= np.concatenate(y_proba)
fpr, tpr, _ = roc_curve(y_real, y_proba)
auc_mean= auc(fpr, tpr)
plt.plot(fpr, tpr, label= f'{type(classifier).__name__}')

plt.legend(loc = 'lower right')
plt.ylabel('True Positive Rate')
plt.xlabel('False Positive Rate')
#plt.title('Repeated % d-Folds ROC Curve for several models')
#(kfold)
#plt.title(f'ROC Curve Comparison ({kfold}-Fold Cross-Validation)')
classes)', fontsize=14, fontweight='bold')
plt.title(f'ROC Curve Comparison ({n}-repeats {kfold}-Fold Cross-Validation)')
Balanced classes)', fontsize=14, fontweight='bold')

# Background and grid settings
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, color='white', linestyle='--', linewidth=1)

# Display the legend
plt.legend(loc='lower right', fontsize='small', frameon=True)

# Show the plot
plt.show()

```

```
n_repeated_roc_models_balance(X, y, kfold= 10, n=3)
```

Start coding or [generate](#) with AI.

```

# This is for 1 split (training set /test set) with smote Resampling

def SMOTE_ROC_Curve(X_train, X_test, y_train, y_test):
    plt.figure(figsize=(10, 8), dpi=150)

    # Define and Train multiple models
    MLmodels = [
        SVC(kernel='rbf', probability=True, random_state=10),

```

```

MLPClassifier(hidden_layer_sizes=(100,)), activation='
LogisticRegression(random_state=10),
GaussianNB(),
ExtraTreesClassifier(random_state=10),
RandomForestClassifier(random_state=10),
AdaBoostClassifier(n_estimators=50, random_state=10),
XGBClassifier(random_state=10),
GradientBoostingClassifier(n_estimators=50, learning
DecisionTreeClassifier(random_state=10),
KNeighborsClassifier(n_neighbors=3),
QuadraticDiscriminantAnalysis()
]

# SMOTE Method
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_

# Loop over models
for model in MLmodels:
    model.fit(X_train_res, y_train_res)
    # Predict probabilities or decision function
    if hasattr(model, 'predict_proba'):
        y_pred_proba = model.predict_proba(X_test)[:, 1]
    else:
        y_pred_proba = model.decision_function(X_test)

# Calculate ROC curve and AUC
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
auc = roc_auc_score(y_test, y_pred_proba)

# Plot ROC curve
plt.plot(fpr, tpr, label=f'{type(model).__name__} (AUC: {auc:.2f})')

# Calculate g-mean for each threshold
gmeans = sqrt(tpr * (1 - fpr))
ix = argmax(gmeans)

# Mark the best threshold
plt.scatter(fpr[ix], tpr[ix], marker='o', color='red')
plt.text(fpr[ix], tpr[ix], f'{thresholds[ix]:.2f}', f'g-mean: {gmeans[ix]:.2f}')

# Plot the diagonal line (random chance level)
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')

# Set plot labels and title
plt.xlabel('False Positive Rate -->')
plt.ylabel('True Positive Rate -->')

```

```
plt.title('Receiver Operating Characteristic (ROC) Curve

# Set grid and background color
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, linestyle='-', linewidth=1, color='white')

# Display legend with optimized placement
plt.legend(fontsize='small', loc='lower right', frameon=1

# Show the plot
plt.show()
```

```
SMOTE_ROC_Curve(X_train, X_test, y_train, y_test)
```

```
# This function very important for plotting cross validation
# This function to plot the mean average AUC StratifiedKFold

def SMOTE_roc_models(X, y, kfold):
    plt.figure(figsize=(10, 8), dpi=150)

    # List of models to evaluate
    MLmodels = [
        RandomForestClassifier(random_state=10),
        ExtraTreesClassifier(random_state=10),
        GradientBoostingClassifier(n_estimators=50, learning_
        AdaBoostClassifier(n_estimators=50, random_state=10),
        XGBClassifier(random_state=10),
        QuadraticDiscriminantAnalysis(),
        GaussianNB(),
        SVC(kernel='rbf', probability=True, random_state=10),
        KNeighborsClassifier(n_neighbors=best_k),
        LogisticRegression(random_state=10),
        DecisionTreeClassifier(random_state=10),

        MLPClassifier(hidden_layer_sizes=(100,), activation='

    ]
    # Create a color map with enough distinct colors for all
    colors = cm.get_cmap('tab20', len(MLmodels)) # Use 'tab2

    # Define cross-validator
    cv = StratifiedKFold(n_splits=kfold, shuffle=True, randc
```



```

# Iterate over each classifier
for i, classifier in enumerate(MLmodels):
    y_real = []
    y_proba = []
    aucs = []

    # Perform cross-validation
    for train_index, test_index in cv.split(X, y):
        Xtrain, Xtest = X.iloc[train_index], X.iloc[test_index]
        ytrain, ytest = y.iloc[train_index], y.iloc[test_index]

        # # Apply SMOTE to the training data
        smote = SMOTE(random_state=42)
        X_res, y_res = smote.fit_resample(Xtrain, ytrain)

        # Fit classifier and predict probabilities
        classifier.fit(X_res, y_res)
        y_score = classifier.predict_proba(Xtest)[:, 1]

        # Calculate ROC for the current fold
        fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
        auc_fold = auc(fpr_fold, tpr_fold)
        aucs.append(auc_fold) # Store AUC for averaging

        # plotting the folds
        #plt.plot(fpr, tpr, lw=1, linestyle='--', label=f'Fold {i+1}')

        # Collect true labels and probabilities for final
        y_real.append(ytest)
        y_proba.append(y_score)

    # Concatenate results from all folds
    y_real = np.concatenate(y_real)
    y_proba = np.concatenate(y_proba)

    # Calculate overall ROC and AUC
    fpr, tpr, _ = roc_curve(y_real, y_proba)
    auc_mean = auc(fpr, tpr)
    std_auc = np.std(aucs)

    # Plot ROC curve for the model
    plt.plot(fpr, tpr, label=f'{type(classifier).__name__}')

# Plot formatting and labeling
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
plt.xlabel('False Positive Rate -->')

```

```

plt.ylabel('True Positive Rate -->')
plt.title(f'ROC Curve Comparison ({kfolds}-Fold Cross-Val

# Background and grid settings
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, color='white', linestyle='-', linewidth=1)

# Display the legend
plt.legend(loc='lower right', fontsize='small', frameon=1

# Show the plot
plt.show()

```

```
SMOTE_roc_models(X, y, 10)
```

```

# This function very important for plotting cross validation
# This function to plot the mean average AUC StratifiedKFold

def repeated_SMOTE_roc_models(X, y, kfolds, n):
    plt.figure(figsize=(10, 8), dpi=150)

    # List of models to evaluate
    MLmodels = [
        ExtraTreesClassifier(random_state=10),
        GaussianNB(),
        MLPClassifier(hidden_layer_sizes=(100,), activation='
        RandomForestClassifier(random_state=10),
        XGBClassifier(random_state=10),
        GradientBoostingClassifier(n_estimators=50, learning_
        AdaBoostClassifier(n_estimators=50, random_state=10)

    ]
    # Create a color map with enough distinct colors for all
    colors = cm.get_cmap('tab20', len(MLmodels)) # Use 'tab2

```

```
# Define cross-validator
#cv = StratifiedKFold(n_splits=kfolds, shuffle=True, random_state=42)
cv = RepeatedStratifiedKFold(n_splits=kfolds, n_repeats=10, random_state=42)

# Iterate over each classifier
for i, classifier in enumerate(MLmodels):
    y_real = []
    y_proba = []
    aucs = []

    # Perform cross-validation
    for train_index, test_index in cv.split(X, y):
        Xtrain, Xtest = X.iloc[train_index], X.iloc[test_index]
        ytrain, ytest = y.iloc[train_index], y.iloc[test_index]

        # # Apply SMOTE to the training data
        smote = SMOTE(random_state=42)
        X_res, y_res = smote.fit_resample(Xtrain, ytrain)

        # Fit classifier and predict probabilities
        classifier.fit(X_res, y_res)
        y_score = classifier.predict_proba(Xtest)[:, 1]

        # Calculate ROC for the current fold
        fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
        auc_fold = auc(fpr_fold, tpr_fold)
        aucs.append(auc_fold) # Store AUC for averaging

    # plotting the folds
    #plt.plot(fpr, tpr, lw=1, linestyle='--', label='F')

    # Collect true labels and probabilities for final
    y_real.append(ytest)
    y_proba.append(y_score)

# Concatenate results from all folds
y_real = np.concatenate(y_real)
y_proba = np.concatenate(y_proba)

# Calculate overall ROC and AUC
fpr, tpr, _ = roc_curve(y_real, y_proba)
auc_mean = auc(fpr, tpr)
std_auc = np.std(aucs)

# Plot ROC curve for the model
```

```

plt.plot(fpr, tpr, label=f'{type(classifier).__name__}

# Plot formatting and labeling
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', la
plt.xlabel('False Positive Rate -->')
plt.ylabel('True Positive Rate -->')
#plt.title(f'ROC Curve Comparison ({kfolds}-Fold Cross-Va
plt.title(f'ROC Curve Comparison ({n}-repeats {kfolds}-Fc

# Background and grid settings
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, color='white', linestyle='-', linewidth=1)

# Display the legend
plt.legend(loc='lower right', fontsize='small', frameon=1

# Show the plot
plt.show()

```

```
repeated_SMOTE_roc_models(X, y, kfolds= 10, n= 10)
```

Start coding or [generate](#) with AI.

```

# This function very important for plotting cross validation
# This function to plot the mean average AUC StratifiedKFold (

from matplotlib import cm

def repeated_SMOTE_balance_roc_models(X, y, kfolds, n):
    plt.figure(figsize=(10, 8), dpi=150)

    # List of models to evaluate
    # Train multiple models
    MLmodels = [
        ExtraTreesClassifier(class_weight='balanced',

```

```

XGBClassifier(class_weight='balanced', random
HistGradientBoostingClassifier(class_weight='
RandomForestClassifier(class_weight='balanced
SVC(class_weight='balanced', kernel='rbf', pr
LogisticRegression(class_weight='balanced', r
DecisionTreeClassifier(class_weight='balanced
]
# Create a color map with enough distinct colors for all
colors = cm.get_cmap('tab20', len(MLmodels)) # Use 'tab20

# Define cross-validator
#cv = StratifiedKFold(n_splits=kfolds, shuffle=True, random
cv = RepeatedStratifiedKFold(n_splits=kfolds, n_repeats=10)

# Iterate over each classifier
for i, classifier in enumerate(MLmodels):
    y_real = []
    y_proba = []
    aucs = []

    # Perform cross-validation
    for train_index, test_index in cv.split(X, y):
        Xtrain, Xtest = X.iloc[train_index], X.iloc[test_index]
        ytrain, ytest = y.iloc[train_index], y.iloc[test_index]

        # # Apply SMOTE to the training data
        smote = SMOTE(random_state=42)
        X_res, y_res = smote.fit_resample(Xtrain, ytrain)

        # Fit classifier and predict probabilities
        classifier.fit(X_res, y_res)
        y_score = classifier.predict_proba(Xtest)[:, 1] if

        # Calculate ROC for the current fold
        fpr_fold, tpr_fold, _ = roc_curve(ytest, y_score)
        auc_fold = auc(fpr_fold, tpr_fold)
        aucs.append(auc_fold) # Store AUC for averaging

    # plotting the folds
    #plt.plot(fpr, tpr, lw=1, linestyle='--', label='Fold %d' % i)

    # Collect true labels and probabilities for final
    y_real.append(ytest)
    y_proba.append(y_score)

# Concatenate results from all folds

```

```

y_real = np.concatenate(y_real)
y_proba = np.concatenate(y_proba)

# Calculate overall ROC and AUC
fpr, tpr, _ = roc_curve(y_real, y_proba)
auc_mean = auc(fpr, tpr)
std_auc = np.std(aucs)

# Plot ROC curve for the model
plt.plot(fpr, tpr, label=f'{type(classifier).__name__}')

# Plot formatting and labeling
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
plt.xlabel('False Positive Rate -->')
plt.ylabel('True Positive Rate -->')
#plt.title(f'ROC Curve Comparison ({kfolds}-Fold Cross-Validation)')
plt.title(f'ROC Curve Comparison ({n}-repeats {kfolds}-Fold Cross-Validation)')

# Background and grid settings
plt.gca().set_facecolor('#E8E8F0')
plt.grid(True, color='white', linestyle='-', linewidth=1)

# Display the legend
plt.legend(loc='lower right', fontsize='small', frameon=True)

# Show the plot
plt.show()

```

```
repeated_SMOTE_balance_roc_models(X, y, kfolds= 10, n= 3)
```

```
=====
=====
```

3.9.4) This Function to plot precision_recall curve for each model with cross validation

```

# This function to plot precision recall for any ML model
def precision_recall_kf(model, X, y, kfolds, title= 'precision_recall')

    plt.figure(figsize=(7, 7), dpi=150)
    #k_fold = KFold(n_splits=FOLDS, shuffle=True, random_state=0)
    k_fold = StratifiedKFold(n_splits=kfolds, shuffle=True, random_state=0)
    predictor = model
    y_real = []
    y_proba = []
    precision_array = []
    recall_array = np.linspace(0, 1, 100)

```

```

for i, (train_index, test_index) in enumerate(k_fold.split(Xtrain, Xtest = X.iloc[train_index], X.iloc[test_index], ytrain, ytest = y.iloc[train_index], y.iloc[test_index]):
    predictor.fit(Xtrain, ytrain)
    pred_proba = predictor.predict_proba(Xtest)[: , 1] if len(Xtest) > 0 else []
    precision_fold, recall_fold, _ = precision_recall_curve(y_real, pred_proba)
    precision_array = interp(recall_array, recall_fold, precision_fold)
    pr_auc = auc(recall_fold, precision_fold)
    lab_fold = 'Fold %d AUC=%.4f' % (i+1, pr_auc)
    plt.plot(recall_fold, precision_fold, lw=1, label=lab_fold)
    y_real.append(ytest)
    y_proba.append(pred_proba)

y_real = np.concatenate(y_real)
y_proba = np.concatenate(y_proba)
precision, recall, _ = precision_recall_curve(y_real, y_proba)
pr_auc= auc(recall, precision)
# plotting no skill line
no_skill = len(y_proba[y_proba==1]) / len(y_proba) # this is no skill
#Plot the overall average
lab = 'Overall =%.2f' % (pr_auc)
plt.plot(recall, precision,color='black', label=lab)
plt.legend(loc='lower left', fontsize='small')
#plot standard deviation # not sure if it right
mean_precision = np.mean(precision_array)
std_precision = np.std(precision_array)
plt.fill_between(recall, precision + std_precision, precision - std_precision, color='black')
# plot the roc curve for the Diagonal line
#no_skill = len(y_real[y_real==1]) / len(y_real) # this is no skill
plt.plot([0, 1], [no_skill, no_skill], linestyle='--', label='No Skill')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.legend(loc='upper right', fontsize='small')
plt.title('% d-Folds Precision-Recall Curve for % s model' % (d, model_name))
plt.gca().set_facecolor('#E8E8F0')
plt.grid(color='white', linestyle='-', linewidth=1)
plt.show()

```

```

precision_recall_kf(RandomForestClassifier(random_state= 10), X, y, 10)
precision_recall_kf(LogisticRegression(random_state= 10), X, y, 10)
precision_recall_kf(KNeighborsClassifier(n_neighbors = best_k), X, y, 10)
precision_recall_kf(XGBClassifier(random_state= 10), X, y, 10)
precision_recall_kf(GaussianNB(), X, y, 10, 'NB')
precision_recall_kf(DecisionTreeClassifier(random_state= 10), X, y, 10)
precision_recall_kf(ExtraTreesClassifier(random_state= 10), X, y, 10)

```

```
precision_recall_kf(SVC(kernel='linear', probability=True, random_state=10))
precision_recall_kf(SVC(kernel='rbf', probability=True, random_state=10))
```

Plotting recall precision curve for all models with K fold cross validation

```
# This function very important for plotting cross validation
# This function to plot the mean average AUC StratifiedKFold (K fold cross validation)

def pr_roc_models(X, y, kfold):
    plt.figure(figsize=(7, 7), dpi=150)
    # Train multiple models
    MLmodels = [
        LogisticRegression(random_state=10),
        RandomForestClassifier(random_state=10),
        XGBClassifier(random_state=10),
        GaussianNB(),
        DecisionTreeClassifier(random_state=10),
        ExtraTreesClassifier(random_state=10),
        KNeighborsClassifier(n_neighbors = best_k),
        SVC(kernel='rbf', probability=True, random_state=10)
    ]
    auc_scores = {}
    #cv = StratifiedKFold(n_splits=kfold)
    cv = StratifiedKFold(n_splits=kfold, shuffle=True, random_state=10)
    for model in MLmodels:
        y_real = []
        y_proba = []
        for i, (train_index, test_index) in enumerate(cv.split(X, y)):
            Xtrain, Xtest = X.iloc[train_index], X.iloc[test_index]
            ytrain, ytest = y.iloc[train_index], y.iloc[test_index]
            model.fit(Xtrain, ytrain)
            pred_proba = model.predict_proba(Xtest)[:,-1] if hasattr(model, 'predict_proba') else model.predict(Xtest)
            precision_fold, recall_fold, _ = precision_recall_curve(ytest, pred_proba)
            pr_auc = auc(recall_fold, precision_fold)
            lab_fold = 'Fold %d AUC=%.4f' % (i+1, pr_auc)
            #plt.plot(recall_fold, precision_fold, lw=1, label=lab_fold)
            y_real.append(ytest)
            y_proba.append(pred_proba)

    y_real = np.concatenate(y_real)
    y_proba = np.concatenate(y_proba)
    precision, recall, _ = precision_recall_curve(y_real, y_proba)
    pr_auc = auc(recall, precision)
    # plotting no skill line
    #Plot the overall average
    lab = 'Overall =%.2f' % (pr_auc)
    #plt.plot(recall, precision, label=lab)
    plt.plot(recall, precision, label= f'{type(model).__name__}')
```



```
plt.legend(fontsize='small')
plt.ylabel('Precision')
plt.xlabel('Recall')
plt.title('% d-Folds Precision-Recall Curve for several i
plt.gca().set_facecolor('#E8E8F0')
plt.grid(color='white', linestyle='-', linewidth=1)
plt.show()
```

```
pr_roc_models(X, y, 10)
```

```
=====
=====
```

```
----- . . . . .
```